

共享设备与医生约束下的超声预约优化

摘要

针对超声检查中多项目患者、设备专项能力、医生共享并发以及空腹、憋尿、床旁和跨室转运等现实约束，本文建立“需求分析与参数标定—纯住院排程—三类患者联合排程—独立验证”的患者级优化框架。数据按时间严格分层：2019年3月1日至2024年3月31日只用于识别项目目录、需求规律和医生容量，2024年4月1日至2025年3月31日只用于评价；住院评价分母保留全部70771个事件，完成定义为患者全部检查项目在开单后48小时内结束。

对于问题一，以“患者编号+精确开单时刻”构造申请事件，将源字段中以“J”连接的收费项目拆成独立任务，剔除床旁加收和图文报告等非扫查项。服务项目按题给时长和资源语义分为11类，名义时长取区间中点，5%复杂病例取区间上界。平峰、高峰和压力阈值全部由训练期日需求分位数确定：三类开单合计的平峰中位为412例/日，高峰阈值为671例/日，压力阈值为797.15例/日。时间顺序检验表明，三类开单合计的日历趋势对数岭回归WAPE为24.92%，住院和门诊分别由过去8周同星期中位数取得16.04%和14.04%的最低WAPE。设备能力由表1中的直接项目、同义项目和明确的广义项目描述确定；无法从表1确认的15个项目不进入候选设备集。床旁检查只有在专项项目能力与7号机床旁位置能力同时成立时才可排入。

对于问题二，在5分钟网格连续日历上建立纯住院患者级排程。决策同时确定患者顺序、项目内部顺序、开始时刻和设备；约束覆盖项目—设备兼容、设备互斥、医生星期一时隙并发、患者自身互斥、48小时截止、空腹10时前完成、憋尿准备、床旁位置以及跨室至少10分钟。采用共同设备优先、候选项目序列枚举和最早可行插入构成的大规模确定性算法。共享FCFS与最小松弛度保护均完成60359例，48小时完整完成率为85.29%；前者等待中位数3.58小时且排入总分钟更少，故作为问题二推荐方案。

对于问题三，将门诊和体检报告日期仅作为回顾性计划日压力代理，在同一设备—医生日历中引入背景完成率下限 α ，形成 ε 约束多目标模型。对 $\alpha = 0, 0.30, \dots, 0.70$ 计算Pareto前沿，并按到可达理想点的归一化距离选取 $\alpha = 0.30$ 。推荐点完成42730个住院事件，住院48小时率60.38%；完成57177个门诊/体检计划日事件，背景率59.53%。该背景率不被表述为“全部需求已满足”，而是当前标准班次、设备能力和医生容量下的可行服务水平。逐层现实校准显示，设备专项能力使纯住院率由100%降至85.52%，项目级准备进一步降至85.29%，三类患者竞争则降至60.38%，主要矛盾因此定位在专项设备与共享容量，而非求解器本身。

在六类代表性小窗口中，CP-SAT得到的最优完成数与构造算法完全一致；对P2、P3导出排程进行不调用调度内核的独立复核，任务键、时长、释放与截止、房间和患者互斥、转运、准备条件、医生并发及结果聚合的违规数均为0。项目级/事件级准备、医生偏紧容量、设备匹配规则边界、憋尿45/90分钟、转运5/15分钟及5个随机种子敏感性进一步界定了结论。由于附件没有真实预约时刻、扫查结束和报告提交时刻，本文将最后项目结束作为报告完成代理，结果用于预约方案与资源压力评估，不作临床因果推断。

关键词：超声预约；患者级排程；共享资源；48小时完成率；项目设备兼容；Pareto优化

1 问题重述

1.1 研究背景

某医院超声科需要同时接收住院、门诊和体检三类检查申请。每个申请可能包含一个或多个超声项目，不同项目的检查时长和准备要求不同；22 台设备也并非同质设备，部分设备只能承担特定项目。任一时刻，一台设备只能服务一名患者，一名患者也不能同时接受两个项目，检查还必须占用一名医生。住院患者从开单到全部项目完成并提交报告的时限为 48 小时，门诊和体检患者则需要在既定服务日内安排。因而，超声预约实质上是多患者、多项目、异质设备和共享医生共同作用下的资源受限排程问题，属于医疗预约与诊断资源调度的典型交叉问题 [1, 2, 14]。

1.2 问题一

利用 2019 年 3 月至 2025 年 3 月的住院、门诊、体检记录以及设备、医生资料，分析三类患者的需求变化和资源使用特征。需要识别平峰、高峰及压力日，估计不同项目的检查时长与医生工作效率，并说明时间、项目、科室和设备等因素对检查需求和供给能力的影响，为后续排程提供参数。

1.3 问题二

仅考虑住院患者，在题目规定的 48 小时完成标准下制定预约与检查顺序。方案需同时决定患者的检查时段、各项目先后次序及所用设备，并满足设备专项能力、空腹与憋尿准备、床旁检查、医生数量、患者项目互斥和跨设备转运等限制。目标是使全部项目在 48 小时内完成的住院患者比例尽可能高，并给出可执行的推荐排程。

1.4 问题三

在问题二模型基础上，把门诊和体检需求加入同一资源系统，在保证其服务水平的前提下联合安排三类患者。需要刻画门诊、体检服务水平与住院 48 小时完成率之间的矛盾，给出合理的折中方案，并评价该方案在需求波动、设备匹配边界、准备时长、医生容量和复杂病例随机性变化下的稳定性。

2 问题分析

2.1 总体分析

三个问题不能分割为彼此独立的统计、排队和优化任务。问题一产生的项目时长、项目—设备兼容集和医生并发容量，直接决定问题二、三的可行域；问题二的纯住院结果给出住院目标在不受门诊、体检竞争时的能力上限；问题三再把三类需求放入同一连续日历，才能解释服务水平之间的真实替代关系。为避免评价数据反向影响参数，本文固定训练期和留出期，并保持两阶段物理隔离。

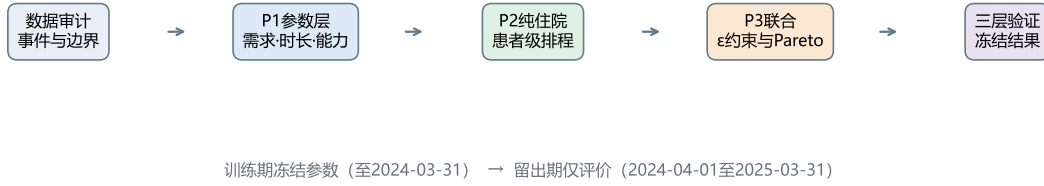


图1 统一患者—项目—设备—医生排程建模链

2.2 问题分析

附件中的一行记录不一定等于一个患者申请，也不一定只含一个项目。合理的基本单元应当是“申请事件—项目任务”两层结构：先按患者编号与精确开单时刻聚合事件，再把同一医嘱字段中以明确边界连接的项目拆开。完成率以事件为分母，工作负荷以项目任务为基础。打印、报告和床旁加收等记录虽能说明服务发生过，却不单独占用扫查设备，应从任务时长中剔除。

需求具有趋势、星期和月份效应。平峰、高峰不应由留出期结果反向挑选，因此采用训练期分位数定义运行状态，再观察留出期超过这些阈值的频率。预测部分不追求复杂模型，而比较7日季节朴素、过去8周同星期中位数和日历趋势对数岭回归；只有时间顺序检验能说明模型对未来日期的泛化。

题目没有扫查开始和结束时刻，报告时间间隔包含候检、扫查、书写和批量提交等过程，不能被当作真实服务时长。主排程应采用题给时长区间，历史相邻报告间隔仅用于比较项目族相对负荷。设备能力也不能直接从历史完成设备反推，否则会把历史临时调度固化为未来规则；候选设备必须由题目表1的项目描述确定。

2.3 问题二分析

问题二的评价对象是患者，而不是单个项目。只要某一项目未排入或越过48小时，整个住院事件均记为未完成。因此不能把每个项目独立分配后再简单相加，而应把同一患者的任务序列作为一个整体插入日历。多项目患者可能在同一设备连续完成，也可能跨设备完成；跨设备虽然扩大了可行集，却增加转运时间和患者流动，故应优先寻找共同设备。这与医疗多预约研究强调的患者级联动一致[3, 4]。

若把全年问题直接写成时间索引整数规划，变量数约为“患者数 × 项目数 × 设备数 × 时隙数”，无法在竞赛时间内求解。可先用整数规划明确约束，再设计共享同一可行性内核的构造算法：生成少量有临床含义的项目序列，按设备稀缺性和准备条件排序，在连续日历上搜索最早可行位置。为了判断构造算法是否牺牲过多，另在六类小窗口中用CP-SAT求精确解。

2.4 问题三分析

问题三中的“满足门诊、体检需求”没有给出具体完成率阈值，且附件没有真实预约日期。若擅自把某一算法所得的59%或60%称为“满足”，会把结果冒充条件。本文把门诊、体检报告日期仅作为回顾性计划日代理，用背景计划日完整完成率 R_B 刻画服务水平，并采用 ε 约束：对一组预先给定的 α ，在 $R_B \geq \alpha$ 条件下最大化住院48小时率。这样得到的是可行前沿，而不是虚构的单一“满足线”。

三类患者必须共享同一设备日历和医生并发容量。若先单独优化住院，再另算门诊、体检，会重复使用容量；若每月重置日历，则会截断跨月的 48 小时任务。故采用全年连续日历，只在留出边界前增加由 48 小时最大截止期推出的 2 日预热，并保留末端 2 日完成窗口。

3 模型假设

1、假设同一患者编号与同一精确开单时刻对应一次完整检查申请；医嘱字段中以“||”连接的内容表示分别计费的项目，应拆为同一事件下的多个任务。该假设直接对应源系统字段结构。

2、假设题目给出的项目时长区间能够代表标准检查工时；名义场景取区间中点，题目所述 5% 复杂检查取区间上界。由于附件没有复杂病例标签，主结果用固定随机种子抽取，并用 5 个额外种子检验。

3、假设工作模板为每日 08:00–12:00 和 13:00–17:00，检查不得跨越午休或日界。该模板由训练期医生首末活动记录重构，是标准日班压力场景，不解释为医院正式排班表。

4、假设任一项目同时占用一台兼容设备和一名医生；设备独占，医生以星期一 5 分钟槽总并发容量表示。题面 33 名未来医生与历史 ID 无法一一映射，因此不虚构实名医生专项资质矩阵。

5、假设项目—设备兼容只由题目表 1 的项目文本确定；允许直接名称、明确同义或包含关系以及题面明确的广义项目描述，复合项目取组成项目的同设备交集。无法从表 1 确认的项目不进入候选设备集合。

6、假设床旁表示检查地点而非项目能力。床旁任务只有在项目本身与 7 号机能力相容时才可使用 7 号机，床旁标记不能覆盖心脏、血管、产科、介入等专项要求。

7、假设腹部、胃肠和肠系膜等空腹项目须在 10:00 前完成；憋尿项目在开单后至少准备 60 分钟。项目准备类型对应公开超声检查流程中的空腹与充盈膀胱要求 [8]，具体 60 分钟提前量由敏感性界定。准备约束按项目粒度施加，事件级统一准备作为对照情景。

8、假设同一患者的两个项目不能重叠；跨设备相邻项目至少留 10 分钟转运，同设备连续项目不额外计转运。5 分钟和 15 分钟边界另行检验。

9、假设住院事件在开单后 48 小时内全部项目完成才计为达标；未排入、项目缺失或任一项目越过截止均记为未完成，分母不因能力缺失而减少。

10、假设门诊、体检报告日期可作为回顾性计划日压力代理，但不能还原真实预约日期；其最后项目在计划日 17:00 前结束记为计划日完成。本文据此讨论服务水平，不声称复原历史调度。

11、假设最后一个排程项目的结束时刻是报告完成的乐观代理。附件没有扫查结束、书写和提交时刻，故模型结果用于预约与容量评估，不能解释为真实报告时效的因果改善。

12、假设训练期结束于 2024 年 3 月 31 日，留出期从 2024 年 4 月 1 日至 2025 年 3 月 31 日。阈值、项目目录、医生容量和稀缺权重只读训练期；留出期仅用于评价，不反向调参。

4 符号说明

表 1 主要符号

符号	含义	单位/取值
$i \in \mathcal{I}$	患者申请事件；含若干检查项目	—
$j \in \mathcal{J}_i$	事件 i 的第 j 个项目任务	—
$r \in \mathcal{R}$	超声设备或对应机房	22 台
$t \in \mathcal{T}$	离散时间槽	5 分钟
p_i, d_i	事件释放时刻与截止时刻	时间戳
s_{ij}, q_{ij}	项目开始时隙与所选设备	时段、设备
τ_{ij}	项目名义或复杂病例时长	分钟
$\mathcal{R}_{ij}$	项目任务的兼容设备集合	设备集合
$C_{w,t}$	星期 w 、槽 t 的医生并发容量	人
x_{ijrt}	项目 (i, j) 是否在设备 r 、时段 t 开始	0–1
y_i	事件 i 是否全部项目按时完成	0–1
R_I	住院 48 小时完整完成率	[0, 1]
R_B	门诊、体检计划日代理完成率	[0, 1]
α	问题三背景服务率下限	[0, 1]
$g(r, r')$	相邻项目设备切换的转运时间	0 或 10 分钟

5 模型的建立与求解

5.1 问题一：需求与资源参数模型

5.1.1 事件—项目两层数据结构

设原始明细记录为 k 。先统一患者、开单、报告、项目、医生和设备字段，时间精确到秒。对同一起来源、同一患者编号和同一开单时间的记录聚合为事件 i ：

$$i = (\text{source}, \text{patientID}, \text{orderTime}).$$

若医嘱字段形如 $[a, \text{彩超}] [b, \text{彩超}]$ ，则按方括号边界拆成两个项目任务，而不按普通逗号拆分，避免把“双侧下肢动脉（股总、股浅、股深等）”错误拆成多个项目。拆分后删除“床旁 B 超检查加收”“超声计算机图文报告”等非扫查项；床旁字样仍保存在事件和项目标记中。

一个事件的全部有效项目构成集合 $\mathcal{J}_i$ 。其名义工作量和复杂病例工作量分别为

$$W_i^{(0)} = \sum_{j \in \mathcal{J}_i} \tau_{ij}^{(0)}, \quad W_i^{(1)} = \sum_{j \in \mathcal{J}_i} \tau_{ij}^{(1)},$$

其中 $\tau^{(0)}$ 取题给区间中点， $\tau^{(1)}$ 取上界。连续模拟包含 167 361 个事件、328 950 个项目任务，具体范围见表 2。住院在留出边界前保留 258 个预热事件，门诊、体检留出期背景事件为 96 046 个。

表 2 连续模拟与留出期数据规模

来源	连续模拟事件	项目任务	留出期评价事件
住院	71,029	141,429	70,771
门诊	55,512	69,756	55,226
体检	40,820	117,765	40,820
合计	167,361	328,950	166,817

5.1.2 科室需求与多项目结构

临床科室需求量按训练期申请事件统计。对同一事件包含多个科室字段的极少数情况，以源记录顺序中的首个有效检查项目归属科室；未知科室单独保留，不向高频科室摊派。表 3 给出三类患者的集中度。住院需求分散于 87 个已知科室，前 6 科室只占 35.45%；门诊前 6 科室占 62.95%，适合按重点科室的星期节律预留普通检查容量；体检数据只对应一个体检科室，但其批次规模变化大，不能用“单一科室”等同于“需求稳定”。

表 3 训练期科室需求集中度与多项目事件比例

来源	训练期事件数	已知科室数	前 6 科室占比	多项目事件占比
住院	297,656	87	35.45%	51.15%
门诊	245,614	133	62.95%	19.67%
体检	239,569	1	100.00%	67.31%

住院和体检的多项目事件比例分别为 51.15% 和 67.31%，明显高于门诊的 19.67%。因此设备配置不能只按单项目数量配额：体检批次虽可集中安排，却更需要能够覆盖多个常见部位的共同设备；住院患者则需同时考虑专项能力和减少往返。题面没有给出同次检查固定节约比例，主模型仍逐项目累加题给工时。全观察期边界分析中，若将多项目事件总工时统一乘 0.9 或 0.8，住院平均名义工作量会由 17.39 分钟降至 16.11 或 14.83 分钟；这种节约缺乏项目组合层面的测时依据，故只用于说明容量范围，不进入排程参数。

训练期项目名称按互斥优先级分类。产科 III/IV 级、产科 I/II 级和心脏项目使用题给专用时长；血管、腹部/胃肠、泌尿/经腹妇科、经阴道/腔内、浅表器官、儿科专项、介入/定位和一般其他采用题给其他项目区间。先判别专项和检查路径，再判别一般部位，可避免“胎儿心脏”被普通心脏规则覆盖、“经阴道妇科”被经腹妇科覆盖。最终参数见表 4。

表 4 服务项目时长与训练期相邻报告代理

项目族	下界/min	名义/min	上界/min	训练代理数
产科 III/IV 级	30	35	40	13,244
产科 I/II 级	15	17.5	20	20,070
心脏	10	12.5	15	140,999
介入/定位	5	7.5	10	6,767
血管	5	7.5	10	58,868
经阴道/腔内	5	7.5	10	21,233
泌尿/经腹妇科	5	7.5	10	74,934
腹部/胃肠	5	7.5	10	99,686
浅表器官	5	7.5	10	67,810
儿科专项	5	7.5	10	3,949
一般其他	5	7.5	10	3,397

5.1.3 需求趋势与峰平阈值

对来源 c 和自然日 d ，定义申请事件量

$$N_{c,d}^{\text{ord}} = |\{i : \text{source}_i = c, [p_i] = d\}|,$$

报告事件量 $N_{c,d}^{\text{rep}}$ 则以事件报告日期的众数归日。报告量可反映历史完成节奏，但问题二、三的未来需求只能由申请量驱动。

2019–2024 年住院开单日均值持续上升：2020 年为 146.12 例/日，2021 年 161.47 例/日，2022 年 170.47 例/日，2023 年 178.02 例/日，2024 年 194.44 例/日。体检呈明显批次性和年份波动，门诊相对平稳。图 2 显示三类需求在星期与月份之外还受节假日和批量体检影响，因而仅用总体均值不足以设计容量。

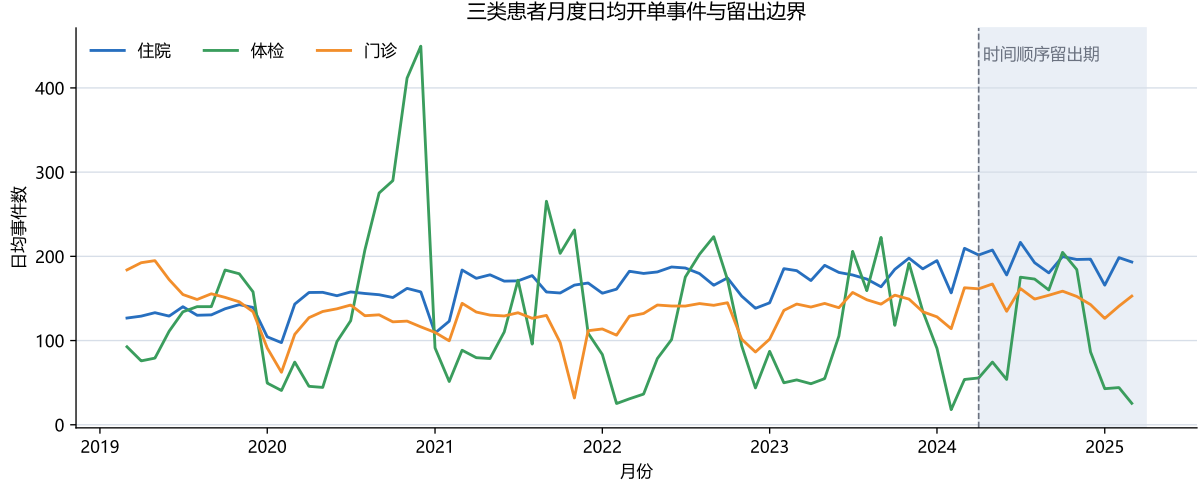


图2 三类患者月度日均开单事件与留出边界

所有运行状态阈值只由训练期确定。对训练期非空日需求序列，定义平峰区间为 $[Q_{0.45}, Q_{0.55}]$ ，高峰阈值为 $Q_{0.90}$ ，压力阈值为 $Q_{0.95}$ ：

$$\begin{aligned}\mathcal{D}_{\text{flat}} &= \{d : Q_{0.45} \leq N_d \leq Q_{0.55}\}, \\ \mathcal{D}_{\text{peak}} &= \{d : N_d \geq Q_{0.90}\}, \\ \mathcal{D}_{\text{stress}} &= \{d : N_d \geq Q_{0.95}\}.\end{aligned}$$

代表日取最接近相应阈值且时间最早的日期，保证规则可复现。三类开单合计的 $Q_{0.45}$ 、平峰中位、 $Q_{0.55}$ 分别为 388.65、412 和 434 例/日，高峰阈值 671 例/日，压力阈值 797.15 例/日。训练期峰日代表为 2019 年 11 月 6 日，共 671 例；平峰代表为 2022 年 3 月 2 日，共 412 例。留出期超过训练高峰阈值的天数占 13.15%，超过压力阈值占 7.95%，说明容量压力不是由少量异常值造成。

5.1.4 时间顺序需求检验

为考察需求规律能否外推，建立三类简单模型。7 日季节朴素为

$$\hat{N}_d = N_{d-7}.$$

过去 8 周同星期中位数为

$$\hat{N}_d = \text{median}\{N_{d-7}, N_{d-14}, \dots, N_{d-56}\}.$$

日历趋势模型在训练期拟合

$$\log(1 + N_d) = \beta_0 + \beta_1 t_d + \sum_{w=1}^6 \gamma_w I(w_d = w) + \sum_{m=2}^{12} \eta_m I(m_d = m) + \beta_2 I(d \text{ 为节假日}) + \varepsilon_d,$$

并用岭惩罚控制月份和星期虚拟变量的共线性。预测回到原尺度时使用 $\exp(\hat{z}) - 1$ ，避免把对数尺度预测直接当作人数。

评价指标为

$$\text{WAPE} = \frac{\sum_d |N_d - \hat{N}_d|}{\sum_d N_d}, \quad \text{sMAPE} = \frac{1}{n} \sum_d \frac{2|N_d - \hat{N}_d|}{|N_d| + |\hat{N}_d|}.$$

在严格时间留出期内，住院开单由过去 8 周同星期中位数取得最低 WAPE 16.04%，门诊开单为 14.04%；体检开单因批量计划变化，最好的日历趋势模型 WAPE 仍为 69.83%。三类合计由日历趋势模型取得 24.92%。这说明住院、门诊的星期规律较稳定，而体检量需要未来批次计划才能获得高精度预测。医疗需求预测和动态预约均强调预测时点与排程决策的顺序关系 [9, 10, 6]；后续排程因此使用实际留出期需求做回放评价，不把预测误差和调度误差混在一起。

表 5 开单日需求预测的留出期 WAPE 比较

开单日序列	7 日季节朴素	过去 8 周同星期中位数	日历趋势岭回归
住院	20.65%	16.04%	16.50%
门诊	19.45%	14.04%	22.16%
体检	106.43%	74.94%	69.83%
三类合计	35.06%	25.19%	24.92%

表 5 说明模型选择按来源分别进行，而不是强制三类需求共用一个预测器。住院和门诊的 8 周同星期中位数能吸收近期水平变化，又不依赖未来数据；体检的三种误差都较大，根源是团检计划具有外生批次信息。实际部署时，住院、门诊可由历史滚动预测，体检必须接入未来团检名单；若名单不可得，应以高分位压力场景而非单点预测配置容量。

5.1.5 项目时长与医生效率边界

历史报告时间不能直接给出扫查时长。影像报告用时与扫查服务时长在口径上需要区分 [7, 11]。本文只对同一医生同一报告日的相邻单项目事件计算正间隔，并截取 (0, 60] 分钟作为局部吞吐代理。有效单项目代理共 510 957 个，中位数 7 分钟；代理同时包含扫查、书写、等候和批量提交，因此只用于检查题给时长的相对排序。例如产科 III/IV 级代理中位数 21 分钟，高于一般项目 5–8 分钟，支持其更高负荷，但主排程仍使用题给 30–40 分钟而非 21 分钟。

对医生 a 在日期 d 的报告事件数 Q_{ad} 和活动跨度 H_{ad} ，构造对数生产率

$$z_{ad} = \log \frac{Q_{ad}}{H_{ad}}.$$

对有效医生日作 1% 和 99% 温莎化，并用经验贝叶斯收缩

$$\tilde{z}_a = \lambda_a \bar{z}_a + (1 - \lambda_a) \mu_0, \quad \lambda_a = \frac{n_a \sigma_b^2}{n_a \sigma_b^2 + \sigma_w^2}.$$

该指标只描述历史报告吞吐差异，不映射到未来 33 名医生。主排程中的供给参数来自表 6 工作日记录：将训练日首末活动时刻重构为活动区间，按星期和 5 分钟槽统计同时

活动人数，主情景取中位数，偏紧情景取下四分位，并截断于题面 33 名医生。主容量在 5–20 人之间，如图 3。

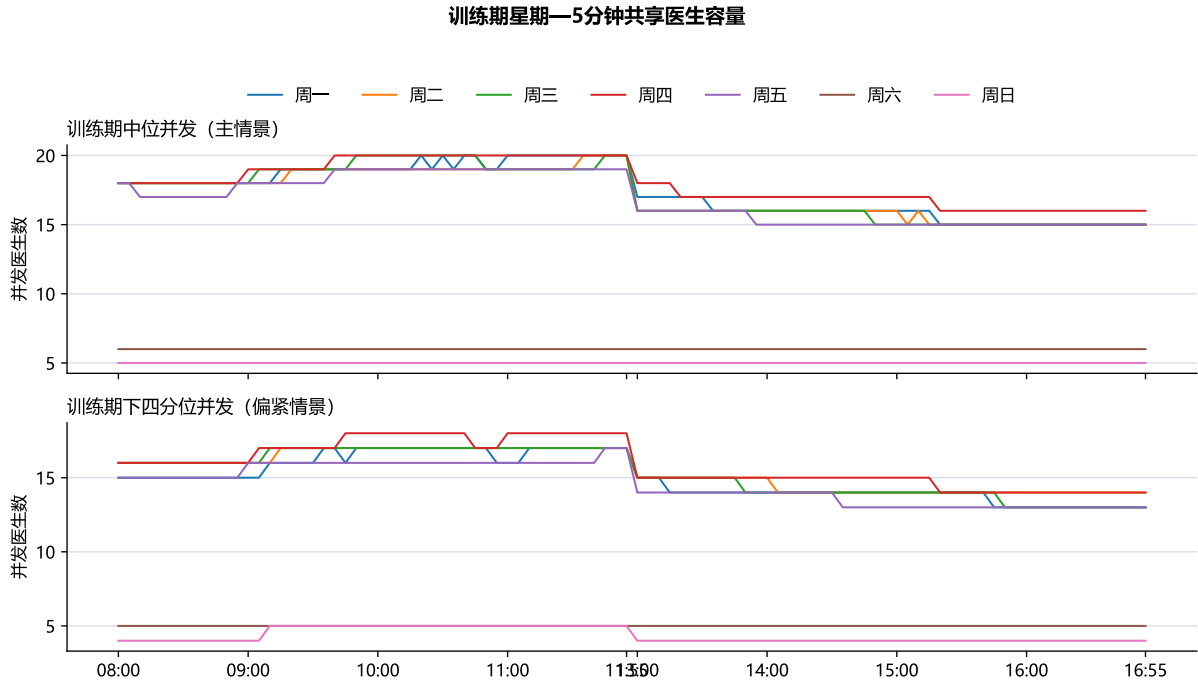


图 3 训练期星期一 5 分钟共享医生容量

5.1.6 项目—设备兼容模型

设表 1 中设备 r 的项目描述集合为 $\mathcal{E}_r$ 。对规范化项目文本 v_j ，定义匹配类型

$$m(j, r) \in \{0, 1, 2, 3\}.$$

其中 1 表示项目文本直接命中；2 表示保留解剖部位和检查路径的同义或包含关系，复合项目取组成部位的同设备交集；3 只对应表 1 明确的广义描述，例如“带有床旁的超声项目”对常规腹部、泌尿和胸水床旁任务的有限覆盖；0 表示表 1 不能确认。主模型兼容集为

$$\mathcal{R}_{ij} = \{r : m(j, r) \in \{1, 2, 3\}\}.$$

对不能逐字匹配的 50 个旧项目逐项回到表 1：33 个可通过项目级语义对应，4 个原来是由字段边界未拆开的复合名称，13 个无法确认。对纠正后的 211 个项目目录，44 个直接命中、60 个通过明确同义或包含关系匹配、92 个由题面广义描述覆盖，另有 15 个无法确认。最后一类任务仍保留在事件分母，但候选设备集为空。匹配覆盖见图 4 和表 6。

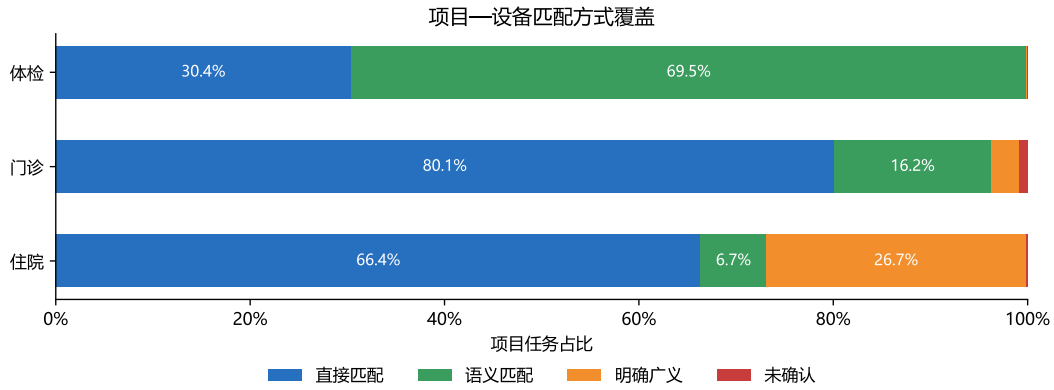


图4 项目—设备匹配方式覆盖

表6 项目—设备匹配方式统计

匹配方式	项目数	项目任务数	使用规则
直接匹配	44	185,497	表1 项目文本直接命中
语义匹配	60	102,651	同义、包含或复合部位交集
明确广义	92	39,893	表1 中的广义项目描述覆盖
未确认	15	909	表1 无法确认，主模型排除

床旁条件单独处理。令 $b_{ij} = 1$ 表示项目层床旁任务，7 号机集合为 $\mathcal{R}^{\text{bed}} = \{7\}$ ，则

$$\mathcal{R}_{ij}^{\text{final}} = \begin{cases} \mathcal{R}_{ij} \cap \mathcal{R}^{\text{bed}}, & b_{ij} = 1, \\ \mathcal{R}_{ij}, & b_{ij} = 0. \end{cases}$$

这一定义使“床旁”只改变地点，不改变专项能力。床旁心脏、床旁产科、床旁血管等若 7 号机没有对应项目能力，必须排除，而不能因含“床旁”二字直接进入 7 号机。

5.1.7 峰平状态下的机器与预约配置

项目—设备集合给出“哪些机器能够做”，峰平配置进一步回答“何时开放、先安排什么”。表 7 以训练期代表日给出三类构成。平峰代表日虽总量为 412 例，但住院占 212 例，不能被简单理解为低负荷日；高峰代表日的 671 例主要来自 321 例体检批次，调度方式应与住院高峰不同。因而调整以来源结构和专项负荷共同决定，不按总人数同比例放大每台机器。

表7 训练期峰平状态及对应资源配置原则

状态	合计/日	住院	门诊	体检	机器与预约配置
平峰代表日	412	212	161	39	专项项目固定进入相容诊室；综合项目按最早空闲分配，多项目优先共同设备。
高峰代表日	671	163	187	321	同时开放数不超过时隙医生容量（最高 20）；保留专项机容量，综合机承担可替代项目，体检批次分时进入。
压力状态	≥ 797	—	—	—	标准班次内先守住政策服务下限；其余背景预约移日或扩展工作时段，不以放宽设备资质扩大容量。

每个 5 分钟槽最多开放的诊室数为 $\min\{22, C_{w,t}\}$ ，主情景最高为 20，故“22 台机

器全部同时运行”不符合医生数量约束。心脏、产科、血管、腹部和床旁等专项任务先进入题面允许的诊室；综合机只在自己的项目集合内承担溢出量。多项目患者优先选择共同设备，无法共同完成时再跨室并加入转运。达到压力阈值后，若仍坚持标准班次和现有能力，合理措施是将可预约的体检、门诊背景需求移日，或经医院批准延长班次；不能把不相容设备临时当作通用设备。至此，问题一形成“需求预测—峰平识别—项目工时—设备兼容—时隙医生容量—预约调整”的完整参数配置。

5.2 问题二：纯住院患者级排程模型

5.2.1 排程对象与完成口径

问题二只加载住院事件，不预占任何门诊、体检容量。对事件 i ，释放时刻为开单时刻向上取整到 5 分钟，截止时刻为

$$d_i = p_i + 48 \text{ h}.$$

若所有 $j \in \mathcal{J}_i$ 均在 d_i 前结束，则 $y_i = 1$ ，否则 $y_i = 0$ 。住院 48 小时完整完成率为

$$R_I = \frac{\sum_{i \in \mathcal{I}_I^{\text{eval}}} y_i}{|\mathcal{I}_I^{\text{eval}}|}.$$

留出期评价集合包含 70 771 个事件。258 个 3 月 30–31 日预热事件可占用 4 月初资源，但既不进入分子也不进入分母；3 月 31 日事件的截止期可延伸到 4 月 2 日，故日历尾端保留 2 日而不按月重置。

5.2.2 时间索引整数规划

令 $x_{ijrt} = 1$ 表示事件 i 的项目 j 在设备 r 、时隙 t 开始， $a_{ijrtu} = 1$ 表示该选择会占用时隙 u 。时长以 5 分钟向上取整：

$$\ell_{ij} = \left\lceil \frac{\tau_{ij}}{5} \right\rceil.$$

每个项目至多安排一次：

$$\sum_{r \in \mathcal{R}_{ij}^{\text{final}}} \sum_{t \in \mathcal{T}_{ijr}} x_{ijrt} \leq 1, \quad \forall i, j.$$

事件完成与全部项目绑定：

$$y_i \leq \sum_{r,t} x_{ijrt}, \quad \forall i, j \in \mathcal{J}_i,$$

$$y_i \geq 1 - |\mathcal{J}_i| + \sum_{j \in \mathcal{J}_i} \sum_{r,t} x_{ijrt}.$$

由此，只安排部分项目不能产生住院完成收益。

设备互斥约束为

$$\sum_i \sum_j \sum_{r'=r} \sum_{t: a_{ijr'tu}=1} x_{ijr't} \leq 1, \quad \forall r, u.$$

同一患者不能同时检查：

$$\sum_j \sum_r \sum_{t: a_{ijrtu}=1} x_{ijrt} \leq 1, \quad \forall i, u.$$

医生是所有设备共享的并发资源。设时隙 u 对应星期 $w(u)$ ，训练期容量为 $C_{w(u),u}$ ，则

$$\sum_i \sum_j \sum_r \sum_{t: a_{ijrtu}=1} x_{ijrt} \leq C_{w(u),u}, \quad \forall u.$$

这比“每个房间各有若干医生”的假设更符合题面，因为 33 名医生应在 22 台设备之间共享。

释放与截止通过可选开始集合 $\mathcal{T}_{ijr}$ 保证：

$$t \geq p_i, \quad t + 5\ell_{ij} \leq d_i.$$

任一项目不得跨越 12:00–13:00 午休或日界。对空腹项目，进一步要求

$$t + 5\ell_{ij} \leq 10:00.$$

对憋尿项目，项目级最早开始为

$$t \geq p_i + 60 \text{ min}.$$

床旁项目的设备集合已由 $\mathcal{R}_{ij} \cap \{7\}$ 给出。

对同一事件内相邻项目 $j \rightarrow k$ ，若所选设备不同，设转运间隔

$$g(q_{ij}, q_{ik}) = \begin{cases} 0, & q_{ij} = q_{ik}, \\ 10 \text{ min}, & q_{ij} \neq q_{ik}. \end{cases}$$

则

$$s_{ik} \geq s_{ij} + \tau_{ij} + g(q_{ij}, q_{ik}).$$

目标首先最大化完整完成事件数，其次减少条件等待、跨室和设备碎片：

$$\max M \sum_i y_i - \lambda_1 \sum_{i: y_i=1} (s_i^{\text{first}} - p_i) - \lambda_2 \sum_i \text{switch}_i - \lambda_3 \sum_r \text{idleGap}_r,$$

其中 M 足够大，保证任何次级目标都不能交换一个完整完成事件。

5.2.3 大规模构造算法

全年时间索引模型过大，本文保留上述可行域，改用确定性患者级构造算法求解。诊断资源的动态优先级和住院患者柔性排程已有类似建模思想 [12, 15]，本文算法内核由“候选序列—共同设备—最早可行插入”组成。

步骤 1： 候选项目序列。对每个事件生成不超过五类序列：原始任务序、兼容设备数从少到多、空腹项目优先、床旁项目优先以及预计完成时间最短路。对项目数较少的事件补充可行排列。所有序列都保持同一患者任务不重叠。

步骤 2：共同设备优先。先计算

$$\mathcal{R}_i^\cap = \bigcap_{j \in \mathcal{J}_i} \mathcal{R}_{ij}^{\text{final}}.$$

若 $\mathcal{R}_i^\cap \neq \emptyset$ ，优先尝试在同一设备完成全部项目；否则才允许项目使用不同设备。选择设备时综合当前最早空闲时刻和训练期稀缺权重

$$\omega_r = \sum_j \frac{D_j \tau_j}{|\mathcal{R}_j|} \mathbb{I}(r \in \mathcal{R}_j),$$

避免普通任务过早占用少数专项设备。

步骤 3：连续日历搜索。对候选序列逐任务向前搜索。某一插入位置可行，当且仅当设备区间空闲、每个 5 分钟槽的医生负载未达上限、项目在单一工作时段内、准备条件满足且最终不越过 d_i 。若有多个位置，选择结束最早者；若结束相同，选择跨室更少和稀缺负荷更低者。

步骤 4：整体提交。只有所有项目都找到位置，才把事件写入设备和医生日历；中途失败则回滚，不允许保留部分任务。该事务式提交与患者完整完成口径一致。

步骤 5：患者优先级。比较两种住院队列：共享 FCFS 按释放时刻和事件编号排序；最小松弛度保护按

$$\text{slack}_i = d_i - p_i - W_i^{(1)}$$

升序，并用释放时刻破同序。两者使用完全相同的可行性内核，差异只来自患者队列。

5.2.4 问题二求解结果

表 8 给出两种队列结果。二者均完整完成 60 359 个留出期住院事件，完成率 85.29%，说明在当前全年需求下，单纯改变 FCFS 和松弛度次序并没有扩大可完成事件集合。共享 FCFS 的条件等待中位数为 3.58 小时，略低于最小松弛度保护的 3.67 小时，且排入总分钟少 80 分钟，因此按预先声明的“完成数最大、再比较 P90、P50 和排入分钟”规则选择共享 FCFS。

表 8 问题二纯住院策略比较

策略	48h 完成数	完成率	P50/h	P90/h	多项目跨室率
共享 FCFS	60,359	85.29%	3.58	22.08	58.12%
最小松弛度保护	60,359	85.29%	3.67	22.08	58.13%

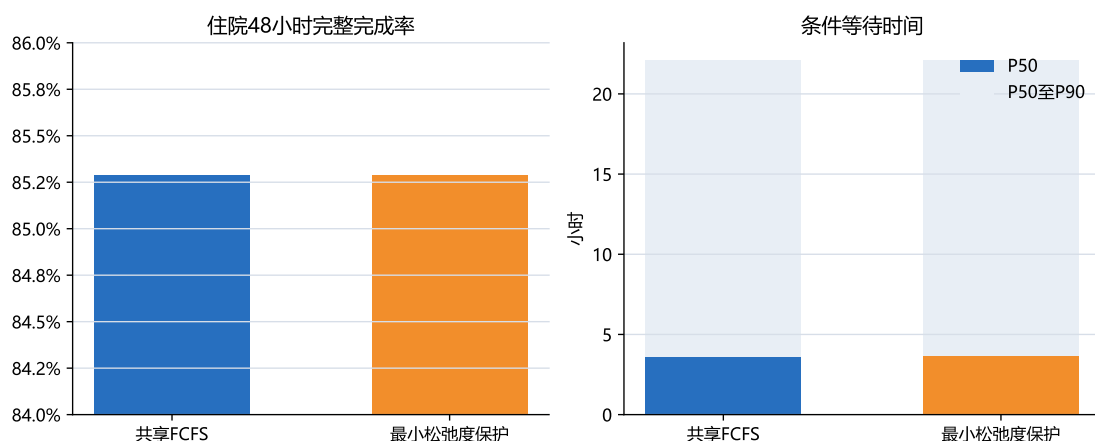


图5 问题二住院完成率与条件等待比较

在评价分母中，10412个事件未能完整排入。表9按排程器最终失败状态给出互斥分解：10245个事件至少含一个无法从表1确认相容设备的项目，另有167个事件虽各项目存在候选设备，但在48小时、准备窗口和已有负荷共同作用下没有形成完整计划。前一类不应通过删除项目、删除事件或临时扩大机器功能“修复”；应由医院确认项目资质后再更新兼容表。后一类才是调整患者顺序或增加容量能够直接改善的对象。

表9 问题二评价期未完成原因分解

评价期未完成原因	事件数	占未完成事件
无满足项目能力的设备组合	10,245	98.40%
48小时内未形成完整可行计划	167	1.60%
合计	10,412	100.00%

完整完成者的P90等待为22.08小时、P95为23.67小时。多项目患者中58.12%发生跨室，这一比例较高，原因是不同项目的专项设备集合交集不足；同一患者每次跨室均显式加入10分钟。模型没有使用与医院空间无关的几何近似，转运只由是否更换实际诊室决定。

纯住院方案共排入1344260分钟，人员在岗容量利用率为54.51%。这一总体利用率不能说明各机器均有富余：表10列出的高负荷诊室有任务日平均利用率已达53%–62%，而无法确认设备的项目即使存在空闲普通机器也不能转入。门7、门1和门6等诊室承担较多可兼容任务；床旁、血管、心脏和产科任务仍受各自候选集合限制。因此问题二的主要改进方向是补全真实项目资质和均衡相容任务，而不是只提高全科总开机率。

表10 问题二年排入分钟最高的诊室

诊室	年排入分钟	有任务日平均利用率
门7	108,685	61.70%
门1	101,280	57.49%
门6	93,840	53.27%
病4	93,810	53.25%
门11	93,810	53.25%
门5	93,810	53.25%

问题二输出为患者一项目一开始一结束一设备推荐表。表 11 展示若干行。事件内序号反映实际执行顺序，而不是把多个项目合并成一个虚构“大检查”；若两个项目跨设备，下一项目开始前已计入转运。

表 11 患者级推荐排程样例

序位	事件	项目序位	项目族	时段	设备
1	IP2024-000267	1	浅表器官	04-01 10:50–11:00	病 3(72)
2	IP2024-000284	1	腹部/胃肠	04-01 10:50–11:00	门 8(46)
2	IP2024-000284	2	腹部/胃肠	04-03 08:40–08:50	门 16(65)
3	IP2024-000265	1	浅表器官	04-01 11:00–11:10	妇门 2(75)
4	IP2024-000271	1	血管	04-01 11:00–11:10	病 1(69)
4	IP2024-000271	2	介入/定位	04-01 11:10–11:20	病 1(69)
5	IP2024-000285	1	浅表器官	04-01 11:00–11:10	妇门 3(77)
5	IP2024-000285	2	浅表器官	04-01 11:10–11:20	妇门 3(77)
5	IP2024-000285	3	腹部/胃肠	04-03 08:50–09:00	门 16(65)
5	IP2024-000285	4	泌尿/经腹妇科	04-03 09:10–09:20	门 17(160)
6	IP2024-000334	1	浅表器官	04-01 11:00–11:10	病 3(72)
6	IP2024-000334	2	浅表器官	04-01 11:10–11:20	病 3(72)
6	IP2024-000334	3	泌尿/经腹妇科	04-01 11:30–11:40	病 3(72)

因此，问题二的推荐方案为：以共享 FCFS 确定患者主序，在每名患者内部优先选择候选设备最少和准备窗口最紧的项目，先尝试同一设备连续完成，不能共同完成时才按最早完成原则跨室；所有项目作为一个事务整体提交。对应的患者级顺序、检查时刻和诊室已完整写入推荐表。该方案是在两种预先设定队列中按词典序指标选出的全年可行近似方案；其完成数在第六章六类小窗口中与精确模型一致，但不表述为全年整数规划全局最优。

5.3 问题三：三类患者联合排程

5.3.1 背景需求的可评价定义

门诊和体检附件包含开单与报告日期，却没有真实预约日期和历史预约时段。若直接读取历史报告时刻、机房和医生，排程将复现答案信息；若把报告日完全忽略，又无法定义门诊、体检服务水平。本文采用有限的回顾性压力场景：报告日期众数只确定计划日 δ_i ，释放时刻不早于真实开单和计划日 08:00，截止为计划日 17:00：

$$p_i^B = \max\{\text{orderTime}_i, \delta_i + 8\text{h}\}, \quad d_i^B = \delta_i + 17\text{h}.$$

历史报告具体时刻、历史设备和医生均不进入调度。对背景事件集合 $\mathcal{I}_B$ ，计划日代理完整完成率为

$$R_B = \frac{\sum_{i \in \mathcal{I}_B} y_i}{|\mathcal{I}_B|}.$$

该指标表示在标准班次下承接历史计划日需求的能力，不等于真实历史准点率。

5.3.2 背景服务下限多目标模型

问题三的两个主要目标为住院 48 小时率 R_I 与背景率 R_B 。超声预约与检查室指派适合在统一资源日历中评价 [16]，等待保证也需要显式的服务等级约束 [13]。加权和

$\lambda R_I + (1 - \lambda) R_B$ 对权重敏感，也会跳过非凸前沿，故本文采用 ε 约束：

$$\begin{aligned} \max \quad & \sum_{i \in \mathcal{I}_I} y_i, \\ \text{s.t.} \quad & R_B \geq \alpha, \\ & \text{问题二全部设备、医生、患者、准备与转运约束.} \end{aligned}$$

预先设置

$$\alpha \in \{0, 0.30, 0.40, 0.50, 0.55, 0.60, 0.65, 0.70\}.$$

这里 α 是全留出期背景完整完成率下限，与最终评价口径一致。算法同时记录逐日配额短缺天数，用于识别某些日期的项目结构不可行，但不以“每一天都恰好达到比例”替代全年服务约束。

5.3.3 联合构造方法

联合日历先装入 3 月 30–31 日的 258 个住院和 286 个背景预热事件。对留出期每个计划日，按训练期项目稀缺权重和释放时刻，从当日门诊、体检事件中选择至少

$$Q_d(\alpha) = \lceil \alpha |\mathcal{I}_{B,d}| \rceil$$

个背景目标事件，先尝试整体插入；随后使用问题二选定的共享 FCFS 处理所有已释放住院事件；最后再用剩余容量尝试当日未作为配额目标的背景事件。暖启动背景、住院和剩余背景均调用同一事务式患者级可行性内核，不存在三套互相矛盾的时长或能力规则。

这种次序不是把背景优先级永久置于住院之上。 α 只预留最低背景服务；达到下限后，住院以 48 小时目标竞争容量；当天最后仍有空闲时，再继续安排背景。由于项目兼容和准备约束，有些背景事件即使在配额内也无法排入，故实际 R_B 通常高于 α ，但不是机械相等。

5.3.4 Pareto 前沿与推荐点

表 12 列出八个运行点。 $\alpha = 0$ 时，住院先用容量，住院率为 85.29%，背景率仍可达到 40.44%；当 α 提高到 0.30，背景率升至 59.53%，住院率降至 60.38%。继续提高 α ，背景率逐步升至 65.19%，住院率下降到 57.01%。所有满足 $R_B \geq \alpha$ 的点均参与非支配判断； $\alpha = 0.65$ 和 0.70 因实际背景率未达到目标，不作为该下限的可行解，但保留在表中以说明容量边界。

表 12 问题三背景下限网格及联合排程结果

α	背景完成数	背景率	住院完成数	住院率	P90/h
0.00	38,842	40.44%	60,364	85.29%	22.08
0.30	57,177	59.53%	42,730	60.38%	21.75
0.40	58,592	61.00%	41,937	59.26%	25.08
0.50	60,388	62.87%	41,265	58.31%	28.92
0.55	61,248	63.77%	40,878	57.76%	29.58
0.60	61,795	64.34%	40,609	57.38%	29.92
0.65	62,234	64.80%	40,460	57.17%	30.17
0.70	62,617	65.19%	40,343	57.00%	30.33

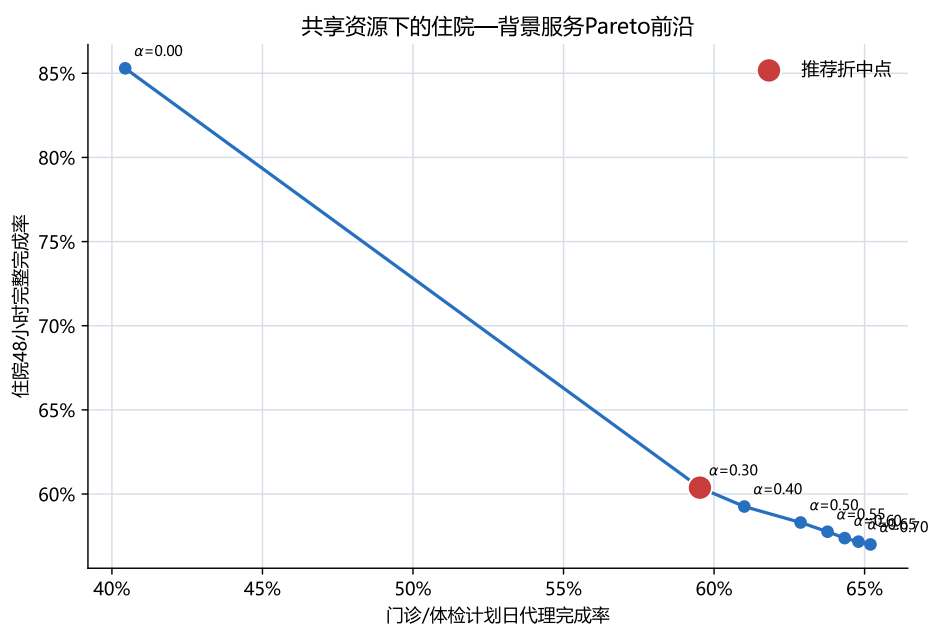


图 6 共享资源下的住院—背景服务 Pareto 前沿

为避免主观挑选，定义可达前沿上的理想点 $(R_B^{\max}, R_I^{\max})$ ，对每个非支配点计算

$$D(\alpha) = \sqrt{\left(\frac{R_B^{\max} - R_B(\alpha)}{R_B^{\max} - R_B^{\min}}\right)^2 + \left(\frac{R_I^{\max} - R_I(\alpha)}{R_I^{\max} - R_I^{\min}}\right)^2}.$$

选择距离最小者，并以背景率较高、住院率较高依次破同序。推荐点为 $\alpha = 0.30$ ：住院完整完成 42 730 例， $R_I = 60.38\%$ ；背景完整完成 57 177 例， $R_B = 59.53\%$ ；住院条件等待 P50 和 P90 分别为 3.00 和 21.75 小时，多项目跨室率 25.04%。

需要强调，59.53% 是可达折中点，不是“满足全部门诊、体检需求”的同义词。若

医院政策规定更高的最低背景率，应在前沿上选择相应点，并接受住院率下降；若要求背景事件 100% 在同日完成，则在当前标准班次、项目能力和医生容量下不可行，应增加资源、延长时段或把部分背景预约移日。

表 13 纯住院与推荐联合方案的运行形态

运行指标	P2 纯住院	P3 推荐联合方案
排入检查分钟	1,344,260	2,066,410
人员在岗容量利用率	54.51%	83.79%
住院等待 P95/小时	23.67	38.58
住院多项目跨室率	58.12%	25.04%
背景日配额未达天数	—	1/364

表 13 揭示了联合情景下降的运行原因。加入门诊、体检后，排入分钟增加 53.72%，人员在岗容量利用率由 54.51% 升至 83.79%，住院等待 P95 由 23.67 小时增至 38.58 小时，已接近 48 小时边界。另一方面，住院多项目跨室率下降到 25.04%，并非联合方案改善了设备兼容，而是背景目标先占用部分灵活容量后，只有更容易形成完整路径的住院事件被排入。故跨室率下降不能单独当作服务改善指标。

$\alpha = 0.30$ 的逐日目标覆盖 364 个有背景需求的日期，其中 363 日达到当天目标，仅 1 日短缺 7 个目标事件；全年最终背景完成 57 177 例，远高于逐日预留阶段的 28 970 例。这说明逐日配额只是构造算法的容量引导，最终可行性仍由全年 $R_B \geq \alpha$ 判定。对 $\alpha = 0.65$ 和 0.70，算法分别达到 64.80% 和 65.19%，未达到所设下限；这两点用于标识可达边界，不能作为满足相应服务政策的方案。

5.3.5 现实校准与瓶颈分解

只报告最终 60.38% 无法判断下降来自数据口径、设备能力还是算法。本文构造 C0–C7 校准链：

- 1. C0：按真实开单至报告提交计算历史留出期 48 小时率；
- 2. C1：只使用题给时长上界，不限制设备和标准班次；
- 3. C2：加入 08:00–12:00、13:00–17:00 班次和 22 并发上限；
- 4. C3：加入训练期星期一时隙医生容量；
- 5. C4：加入表 1 项目—设备兼容关系；
- 6. C5：加入项目级空腹、憋尿和床旁语义；
- 7. C6：加入门诊、体检背景竞争；
- 8. C7：使用正式确定性构造策略。

表 14 历史现实校准与约束增量

校准层	完成数/分母	完成率	较上一层/百分点
历史开单至报告	59,266/70,864	83.63%	—
仅题给时长	70,771/70,771	100.00%	+16.37
加入标准班次	70,771/70,771	100.00%	+0.00
加入医生并发	70,771/70,771	100.00%	+0.00
加入项目设备能力	60,526/70,771	85.52%	-14.48
加入项目级准备	60,359/70,771	85.29%	-0.24
加入门诊/体检竞争	42,730/70,771	60.38%	-24.91
正式构造策略	42,730/70,771	60.38%	+0.00

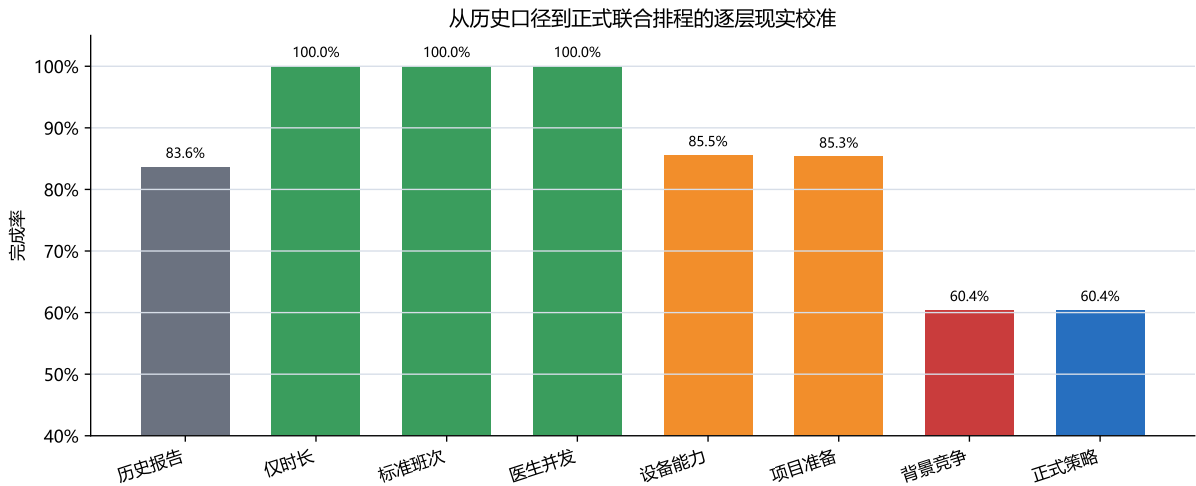


图 7 从历史口径到正式联合排程的逐层现实校准

历史开单至报告率为 83.63%，但其报告时间包含医院真实加班、排班和流程，且事件口径与排程代理不同，不能直接视为模型基准。C1–C3 均为 100%，表明若设备同质，医生总并发在标准班次内足以容纳纯住院题给工时；加入项目—设备兼容后降至 85.52%，说明专项设备集合是纯住院的首要限制。项目级准备使其再降 0.24 个百分点。加入门诊、体检竞争后下降 24.91 个百分点，说明联合场景的主要矛盾是共享容量分配，而非 FCFS 与松弛度的细微差别。

5.3.6 推荐排程的运行形态

推荐联合方案共导出 176 739 条已排任务，涉及 100 303 个完整或部分可执行事件；评价时只有完整事件进入完成分子。图 8 显示日级率受周末、节假日和项目组合影响，14 日滚动后仍存在明显波动，说明不能以总体率替代每日运营监控。

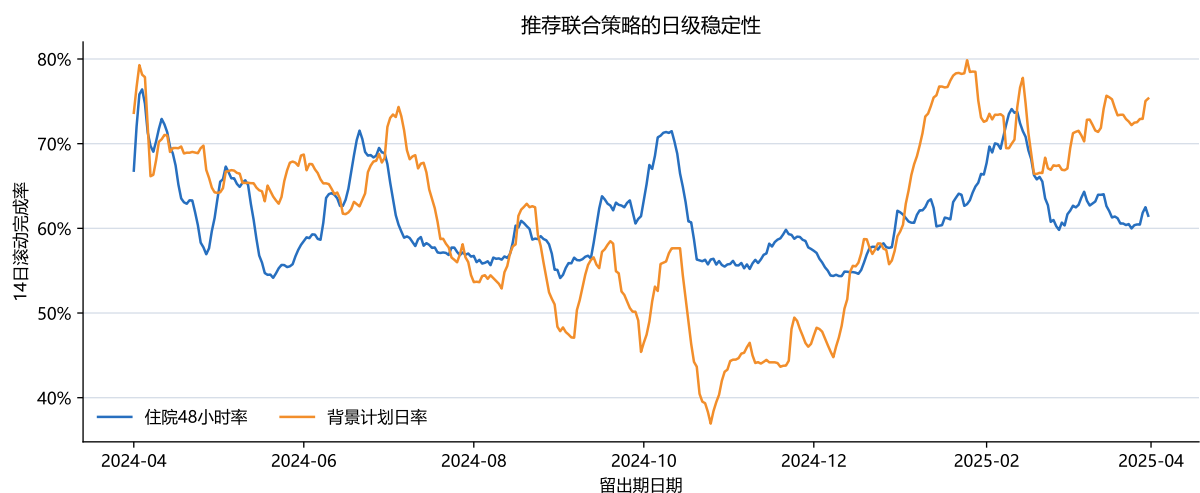


图 8 推荐联合策略的日级稳定性

图 9 给出总排入分钟最高的代表日设备甘特图。每个条块对应一个患者项目，资源结构使用题给 22 台设备、上午/下午工作块和专项兼容关系。午休时段为空白，床旁 7 号机和专项诊室的负荷由实际可行任务产生。

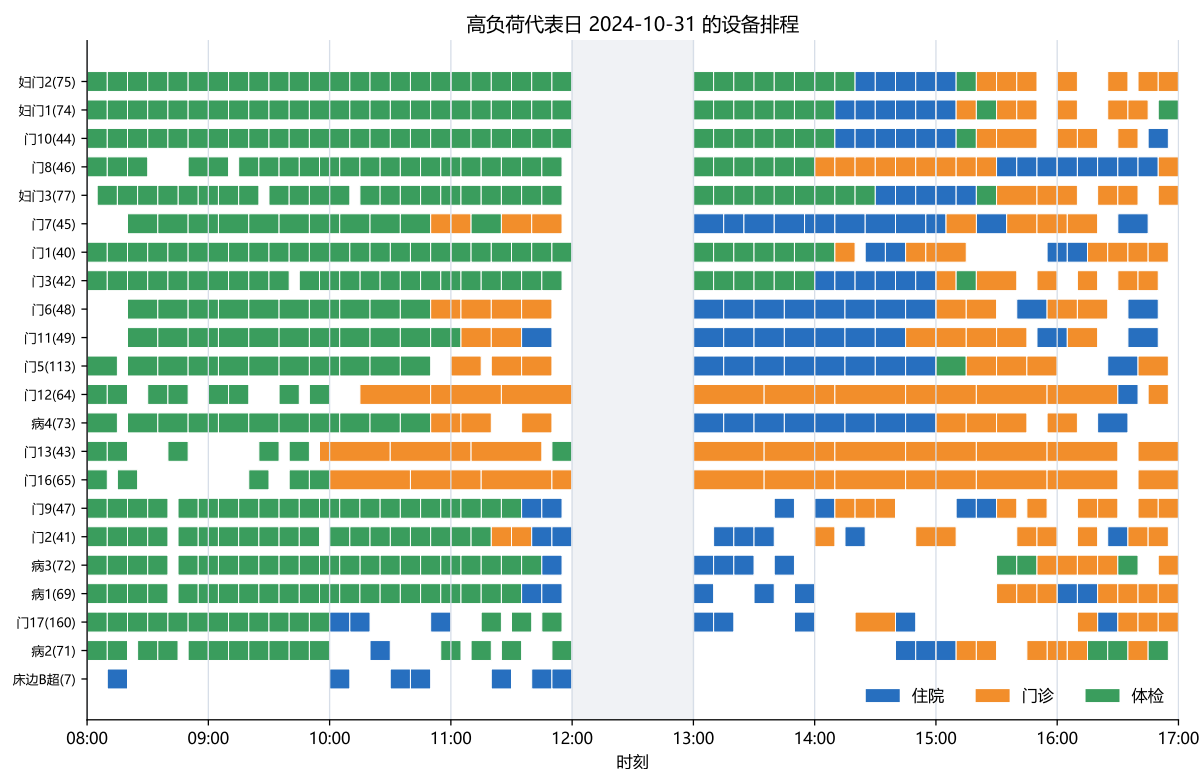


图 9 高负荷代表日的设备排程

6 模型的检验

6.1 检验框架

本文将检验分为数据口径、模型输入和排程可行性三层。第一层回答“记录如何成为事件和项目”；第二层回答“参数与设备能力是否来自题面和训练期”；第三层回答“导出结果是否真正满足约束”。三层使用不同的检查逻辑，避免用程序成功运行代替模

型正确。

6.2 事件口径检验

首先核对三类明细的日期范围、空值、重复、患者 ID、开单与报告顺序、设备和医生 ID。精确重复只删除后续副本；同一患者同一时刻的不同项目不视为重复。对事件键另做敏感性：比较“患者+精确时刻”和加入科室、医生等更严格键，检查是否发生大量拆分。最终严格键只拆分 5 个患者一时刻组，说明主键没有把明显不同申请大规模合并。

源系统项目字段检查发现少量记录使用“][”连接多个项目。若把整串名称当作一个项目，会同时低估任务数和时长，并产生不存在的“腹部+泌尿系, 彩超][前列腺 B 超”项目。拆分后所有显式边界均被识别，普通逗号和中文顿号仍保留在项目名称中；最终项目表中含“][”的规范化项目数为 0。

报告间隔检验不把非正间隔或超过 60 分钟的间隔当作扫描时长。510957 个单项目局部代理仅用于相对比较；主排程时长逐项回到题给区间。该处理避免将午休、空闲、批量提交或报告补录错误计为检查工时。

6.3 训练一留出时间边界检验

阈值、代表日、项目目录、医生容量和诊室稀缺权重均只由训练期确定；留出期只计算超过训练阈值的天数比例。日历模型只在 2019 年 3 月至 2024 年 3 月拟合，2024 年 4 月至 2025 年 3 月评价。滚动 8 周模型的每个预测只读取预测日以前的数据。

设备能力完全来自题面表 1 和预先编写的名称规则，不读取留出期历史完成设备；医生容量只读训练期表 6，不读取留出期报告量。门诊、体检的报告日期虽用于定义回顾性计划日，但报告时刻、历史设备和医生均被排除，避免把未来执行结果泄漏给调度器。

6.4 项目—设备关系检验

对 211 个当前项目逐一检查候选设备能否追溯到表 1 的直接文本、明确同义关系或题面广义描述。结果中，候选诊室超出允许集合 0 条，无法确认的项目获得非空候选集 0 条，按整个项目族整体授权 0 条。原先 50 个不能逐字匹配的名称中，33 个通过项目级语义对应，13 个因表 1 无法确认而排除，4 个复合单元按源字段边界拆分。

床旁检验分别比较项目设备集合、7 号机地点能力和最终集合。所有非空床旁集合均为 {7}，且 7 号机本身允许相应项目，违规 0 条。专项床旁项目若只有地点条件而没有项目能力，其最终集合为空。这样可防止“床旁标记覆盖专项能力”。

医生资质方面，历史记录能够形成医生—项目族出现频数，但题面未来 33 名医生没有与历史 ID 对应的名单。本文据此验证的是总并发容量，而不是虚构的个人资质指派；这是一项明确的适用边界，不通过额外设定弥补缺失数据。

6.5 小规模精确求解对照

从留出期选取普通工作日、高峰工作日、周末、多项目占比最高日、床旁量最高日和产科量最高日六类窗口。每个窗口保留 10 或 18 个住院事件，使用与全年模型相同的项目时长、设备集合、准备条件、患者互斥、转运和时变医生容量建立 CP-SAT 模型，在 60 秒内最大化完整事件数。

表 15 代表性小窗口中构造算法与 CP-SAT 对照

窗口	事件数	构造解	CP-SAT	最好界	状态
普通工作日	18	15	15	15	OPTIMAL
高峰工作日	18	16	16	16	OPTIMAL
周末	18	16	16	16	OPTIMAL
多项目高峰	10	10	10	10	OPTIMAL
床旁高峰	18	2	2	2	OPTIMAL
产科高峰	18	16	16	16	OPTIMAL

六个窗口均求得最优状态。构造算法与 CP-SAT 的完成数完全相同，相对精确解和最好界的差距均为 0。该结果不能证明全年构造解全局最优，但说明共同设备优先、稀缺项目排序和最早可行插入没有在这些典型小窗口中损失事件完成数。全年结果因此表述为“经小规模精确对照的可行近似解”，不夸大为整数规划全局最优。

6.6 导出排程的独立可行性验证

独立验证程序不导入调度器的日历和插入函数，而从 CSV 重新构造约束。检查内容包括：

1. 事件一项目键唯一，排入任务存在于输入，名义/复杂时长与 5 分钟取整一致；
2. 所选设备属于项目兼容集合，床旁任务只用 7 号机；
3. 项目不早于释放和憋尿就绪，不晚于事件截止，不跨午休或日界，空腹项目 10:00 前完成；
4. 同一设备区间不重叠，同一患者任务不重叠，跨设备间隔不少于 10 分钟；
5. 每个 5 分钟槽的总任务数不超过星期一槽医生容量；
6. 按任务表重新聚合的完整完成与截止标记和患者结果表一致。

表 16 问题二、三独立排程验证

核验组	P2 违规	P3 违规	结论
键、输入与时长	0	0	通过
设备能力与床旁	0	0	通过
时间窗与准备	0	0	通过
互斥与转运	0	0	通过
医生容量	0	0	通过
结果聚合	0	0	通过

问题二 117 802 条排入任务、问题三 176 739 条排入任务在全部检查组中的违规数均为 0。最大观察医生并发为 20，未超过主容量 20。验证器对问题二只比较住院结果事件，对问题三比较三类事件，避免把纯住院输出错误地与门诊、体检输入共同计数。

6.7 关键假设敏感性

围绕无法由附件唯一确定的边界，保持 $\alpha = 0.30$ 和同一留出期，逐项改变一个主因素。结果见表 17 和图 10。

表 17 推荐联合方案的单因素敏感性

情景	住院率	背景率	P90/h	排入分钟
主设定：项目级准备	60.38%	59.53%	21.75	2,066,410
事件级准备	59.45%	58.54%	19.08	1,848,890
医生容量下四分位	59.20%	53.22%	24.67	1,917,565
仅直接/语义匹配	44.81%	62.41%	18.42	1,856,190
憋尿 45 分钟	60.31%	59.72%	21.58	2,067,270
憋尿 90 分钟	60.48%	59.30%	21.92	2,064,825
跨室 5 分钟	60.05%	59.63%	21.42	2,047,980
跨室 15 分钟	60.51%	59.40%	21.75	2,066,540

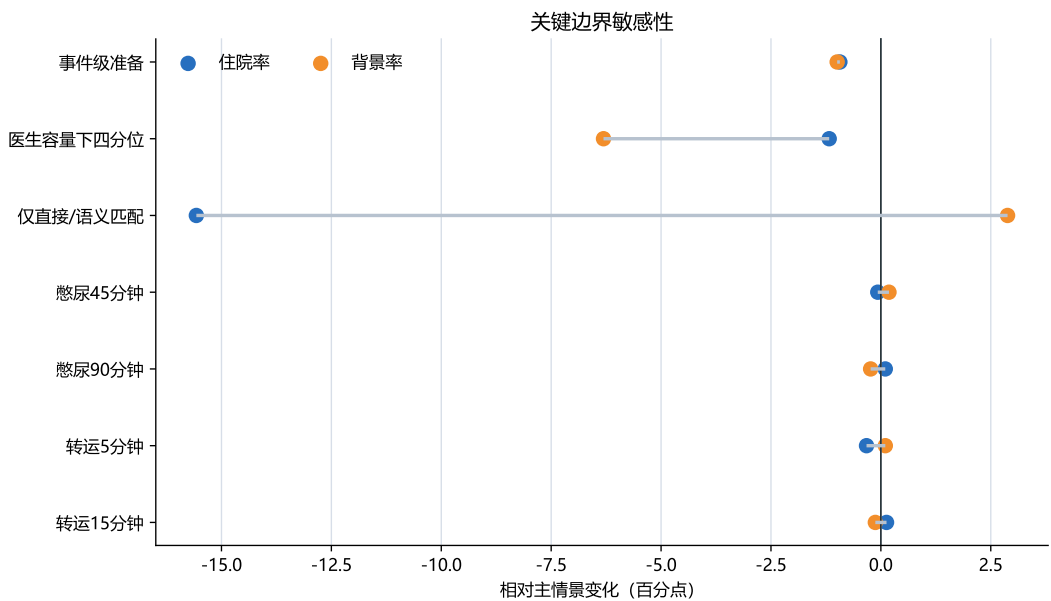


图 10 关键边界相对主情景的完成率变化

医生容量降至训练期下四分位时，住院率由 60.38% 降至 59.20%，背景率由 59.53% 降至 53.22%，说明联合方案对人员在岗容量较敏感。只保留直接和明确语义匹配时，住院率降至 44.81%、背景率升至 62.41%；背景率上升不是能力增强，而是更多住院项目因候选设备为空而退出容量竞争。因此完成率必须连同未排原因和项目覆盖一起解释。事件级准备情景的住院率为 59.45%、背景率为 58.54%，均低于项目级主情景，说明项目级施加空腹与憋尿条件更符合检查流程，且不会把无关项目一并锁入准备窗口。

憋尿 45/90 分钟和转运 5/15 分钟的结果围绕主情景小幅变化，说明主结论对这两个时间参数较稳健。转运时长从 5 分钟增加到 15 分钟时完成率并未单调下降，这是构造算法改变组合打包次序所致，不能解释成延长转运有利；三种情景相差不足 0.46 个百分点，支持的结论只是该参数在当前范围内不改变主方案量级。

表 18 5 个复杂病例随机种子的稳健性

指标	均值	标准差	最小值	最大值
住院 48 小时率	60.3626%	0.0195 个百分点	60.3326%	60.3807%
背景计划日率	59.6239%	0.0204 个百分点	59.6006%	59.6506%
等待 P90/小时	21.70	0.04	21.67	21.75
排入分钟	2,067,493	690	2,066,490	2,068,475

表 18 中，住院率标准差仅 0.0195 个百分点，背景率标准差为 0.0204 个百分点，等待 P90 限制在 21.67–21.75 小时。固定种子只用于给出可复现主方案，5% 复杂病例具体落在哪些患者上不是结论的主要随机来源。

7 模型的评价

7.1 模型优点

1. 评价口径与赛题目标一致。住院结果以患者全部项目为完成单元，任何未排项目都使事件失败，且全部 70 771 个留出期事件保留在分母中。模型没有通过只统计可排患者、只统计单项目或删除能力未确认项目来提高 48 小时率。

2. P2 与 P3 边界清晰。问题二是纯住院排程，不预占门诊、体检容量；问题三才在同日历加入背景需求。P2 的 85.29% 因此表示当前设备能力下的纯住院可达水平，P3 的 60.38% 反映三类患者竞争后的折中，两者不混用。

3. 设备能力贴合业务。22 台设备按题面表 1 逐项目建立集合，复合检查取组成项目的同设备交集，床旁位置与专项能力取交集；无法从表 1 确认的项目不进入候选集。设备在模型中保持真实的离散诊室属性，不被视为同质机器。

4. 参数来源明确。题给时长、5% 复杂比例和 48 小时规则直接进入模型；需求、项目目录和医生容量只由训练期估计；憋尿和转运等无法唯一确定的参数单独做敏感性。报告间隔只承担相对吞吐比较，不替代题给扫查时长。

5. 求解与验证分离。全年用构造算法保证计算可承受，小窗口用 CP-SAT 检验完成数损失；最终排程再由独立程序重建约束。六类窗口零差距、P2/P3 排程零违规，使“可行近似解”的表述有直接证据。

6. 多目标冲突表达透明。问题三不把某个算法结果自行定义为“满足需求”，而用 ε 约束给出完整前沿。医院可以根据政策改变 α ，明确看到背景服务提升所交换的住院完成率。

7.2 模型不足

1. 报告完成仍是代理。题目要求从开单到所有报告提交，但附件没有扫查结束和报告提交过程字段。本文用最后项目结束代理报告完成，忽略报告书写、审核和提交延迟，因此对真实 48 小时率偏乐观。历史 83.63% 与排程率的口径不同，不能直接比较优劣。

2. 背景计划日是回顾性代理。门诊、体检的报告日期并非真实预约日期。用它构造压力场景能比较容量，但不能还原患者实际到达、爽约、迟到和改约行为。59.53% 只能解释为标准班次同日承载率。

3. 医生能力只到总并发。模型知道 33 名医生总量和训练期时隙并发，却不知道未来每名医生的专项资质、班次、请假和设备偏好。若现实中某时段只有少数医生能做 IV 级产科或经食管超声，总并发模型会高估专项供给。

4. 项目级服务时长仍较粗。除产科和心脏外，多数项目使用 5–10 分钟区间；多项目之间可能存在共享准备、共享报告和同部位协同，模型按任务相加可能高估或低估实际工时。历史报告间隔又不足以安全标定绝对时长。

5. 构造算法不保证全年全局最优。小窗口零差距提高了可信度，但不能排除全年存在通过交换多个患者而增加完成数的复杂改进。特别是床旁唯一设备、跨日 48 小时和多项目共同设备选择可能形成长程组合效应。

6. 公平性目标尚未进入主目标。当前先最大化事件完成数，可能偏向项目少、设备选择多的患者。虽然未删除复杂事件，完成率目标仍可能造成科室、疾病严重程度或多项目患者之间的服务差异，部署前需要增加公平约束。

7.3 结果适用范围

推荐方案适用于题给 22 台设备、33 名医生总量、标准日班和留出期需求结构。超声质量控制强调以专业流程和质量指标管理诊疗活动 [5]；若正式排班包含夜班、周末不同工时或新增设备，应重新生成医生容量和项目兼容集，而不能沿用本次数值。表 1 无法确认的项目应由超声科核定资质后再进入系统，不能按项目大类自动扩展机器功能。

输出患者 ID 为数据集标识，实际部署还需接入预约系统的患者状态、医嘱变更和检查取消。算法可作为滚动决策支持：每天更新未完成事件和未来背景预约，在已经确认的能力与准备规则下重排；不建议把全年回放表直接当作长期不变的临床排班。

8 模型改进与推广

8.1 接入真实流程时间

若能获得预约时刻、到达时刻、扫查开始与结束、报告提交和审核时刻，可将当前单一“最后任务结束”拆为

开单 → 预约 → 到达 → 扫查 → 报告 → 审核。

用生存分析或分位数回归估计等待、服务和报告延迟分布，再将 48 小时完成写成机会约束

$$\Pr\{C_i^{\text{report}} \leq p_i + 48 \text{ h}\} \geq 1 - \epsilon_i.$$

这样可以区分预约拥堵和报告流程延迟，也能对不同项目给出具有统计覆盖率的缓冲，而不是统一取上界。

8.2 医生实名资质与排班联动

获得未来医生集合 $\mathcal{A}$ 、医生一项目资质矩阵 h_{aj} 和实名班次 B_{at} 后，引入医生指派变量 z_{ijat} ：

$$z_{ijat} \leq h_{aj}, \quad \sum_{i,j} z_{ijat} \leq B_{at}, \quad \sum_a z_{ijat} = \sum_r x_{ijrt}.$$

这会把当前总并发近似升级为设备—医生—项目三维匹配。若同一医生在一段连续时间内尽量固定设备，还可加入换房惩罚，减少人员走动和消毒切换。

8.3 滚动时域与在线重排

全年回放用于评价，而真实运行应采用滚动时域。每天或每小时重建集合：

$$\mathcal{I}^{(k)} = \mathcal{I}_{\text{unfinished}}^{(k)} \cup \mathcal{I}_{\text{booked}}^{(k:k+H)} \cup \mathcal{I}_{\text{new}}^{(k)}.$$

固定已经开始或即将开始的项目，只对窗口 H 内其余事件重排。目标中加入改约惩罚

$$\lambda_{\text{chg}} \sum_i |s_i^{(k)} - s_i^{(k-1)}|,$$

在提高完成率的同时控制患者通知成本。对于床旁急诊或设备故障，可局部释放受影响任务并重算，不必推翻全年方案。

8.4 公平与服务等级扩展

对科室 g 、多项目组和高严重程度组定义分组完成率 R_g ，增加

$$R_g \geq \rho_g, \quad \max_g R_g - \min_g R_g \leq \Delta.$$

也可采用分层词典序目标：先满足急危重和政策下限，再最大化住院总完成，随后优化门诊、体检和等待。相较把所有患者压成单一权重，这种形式更便于医院解释。

8.5 容量建设情景

校准链表明项目专项能力和三类背景竞争是主要瓶颈。可把设备增购、工作时段延长和医生增班设为上层决策：

$$\min \sum_r c_r^{\text{device}} u_r + \sum_t c_t^{\text{overtime}} v_t + \kappa(1 - R_I),$$

其中 u_r 表示新增某类设备， v_t 表示扩展时段。下层仍为本文患者级排程。通过双层或样本平均近似，可比较“增加普通设备”“增加床旁设备”“增加产科/心脏专项设备”和“增加医生班次”的边际收益。由于 C1–C3 已显示总医生并发不是纯住院首要瓶颈，优先增购普通同质设备未必有效，必须与专项能力共同评估。

8.6 向其他医疗预约问题推广

该框架可推广到 MRI、CT、内镜、放疗和手术室预约：患者事件对应诊疗申请，项目任务对应序列，设备兼容对应检查协议或手术间资质，医生容量对应技师、麻醉和术者，准备条件对应禁食、造影和消毒。推广时应保留三项原则：以患者完整流程为目标；能力集合必须由真实资质产生；统计参数和评价期严格隔离。迁移到新场景时，资源数量、资质集合、工作时段、准备和转运时间必须重新估计，不能直接复制本题参数。

9 参考文献

- [1] GUPTA D, DENTON B. Appointment scheduling in health care: Challenges and opportunities[J]. IIE Transactions, 2008, 40(9): 800–819. DOI: <https://doi.org/10.1080/07408170802165880>.
- [2] AHMADI-JAVID A, JALALI Z, KLASSEN K J. Outpatient appointment systems in healthcare: A review of optimization studies[J]. European Journal of Operational Research, 2017, 258(1): 3–34. DOI: <https://doi.org/10.1016/j.ejor.2016.06.064>.
- [3] MARYNISSEN J, DEMEULEMEESTER E. Literature review on multi-appointment scheduling problems in hospitals[J]. European Journal of Operational Research, 2019, 272(2): 407–419. DOI: <https://doi.org/10.1016/j.ejor.2018.03.001>.
- [4] APERGI L A, BARAS J S, GOLDEN B L, et al. An optimization model for multi-appointment scheduling in an outpatient cardiology setting[J]. Operations Research for Health Care, 2020, 26: 100267. DOI: <https://doi.org/10.1016/j.orhc.2020.100267>.
- [5] 国家卫生健康委员会办公厅. 国家卫生健康委办公厅关于印发超声诊断等 5 个专业医疗质量控制指标（2022 年版）的通知：国卫办医函（2022）161 号 [EB/OL]. (2022-05-27)[2026-08-24]. <https://www.nhc.gov.cn/yzygj/c100068/202205/164e6556f6d34c33a3344fec9a2c5074.shtml>.
- [6] VAN DER AALST W M P, SCHONENBERG M H, SONG M. Time prediction based on process mining[J]. Information Systems, 2011, 36(2): 450–475. DOI: <https://doi.org/10.1016/j.is.2010.09.001>.
- [7] SEXAUER R, BESTLER C. Time is money: Considerations for measuring the radiological reading time[J]. Journal of Imaging, 2022, 8(8): 208. DOI: <https://doi.org/10.3390/jimaging8080208>.
- [8] 华容区人民医院. 超声科流程及注意事项 [EB/OL]. (2025-08-21)[2026-08-24]. http://www.hbhr.gov.cn/hrqxxgk/xxgkml/ggqsydwxxgk/wsjk/hrqrmyy/202508/t20250821_720866.html.
- [9] LUO L, LUO L, ZHANG X, et al. Hospital daily outpatient visits forecasting using a combinatorial model based on ARIMA and SES models[J]. BMC Health Services Research, 2017, 17: 469. DOI: <https://doi.org/10.1186/s12913-017-2407-9>.
- [10] LIN C K Y. Dynamic appointment scheduling with forecasting and priority-specific access time service level standards[J]. Computers & Industrial Engineering, 2019, 135: 970–986. DOI: <https://doi.org/10.1016/j.cie.2019.06.049>.
- [11] LAI C H, LU Y J, CHEN P S. Using data-driven techniques to predict outpatient ultrasound examination time for the multi-clinic outpatient appointment scheduling problem[J]. Communications in Statistics-Simulation and Computation, 2025, 54(9): 3358–3376. DOI: <https://doi.org/10.1080/03610918.2024.2349168>.

- [12] PATRICK J, PUTERMAN M L, QUEYRANNE M. Dynamic multipriority patient scheduling for a diagnostic resource[J]. *Operations Research*, 2008, 56(6): 1507–1525. DOI: <https://doi.org/10.1287/opre.1080.0590>.
- [13] BAUERHENNE C, KOLISCH R, SCHULZ A S. Robust appointment scheduling with waiting time guarantees[J]. *Manufacturing & Service Operations Management*, 2026, 28(3): 995–1009. DOI: <https://doi.org/10.1287/msom.2024.0852>.
- [14] GREEN L V, SAVIN S, WANG B. Managing patient service in a diagnostic medical facility[J]. *Operations Research*, 2006, 54(1): 11–25. DOI: <https://doi.org/10.1287/opre.1060.0242>.
- [15] PATRICK J, PUTERMAN M L. Improving resource utilization for diagnostic services through flexible inpatient scheduling: A method for improving resource utilization[J]. *Journal of the Operational Research Society*, 2007, 58(2): 235–245. DOI: <https://doi.org/10.1057/palgrave.jors.2602242>.
- [16] CHEN P S, CHEN G Y H, LIU L W, et al. Using simulation optimization to solve patient appointment scheduling and examination room assignment problems for patients undergoing ultrasound examination[J]. *Healthcare*, 2022, 10(1): 164. DOI: <https://doi.org/10.3390/healthcare10010164>.

A 复现说明与权威结果

本附录不计入 30 页正文上限。唯一权威结果根目录为 **supporting_materials/results/final_frozen/**。复现顺序为：原始工作簿转存 Parquet；运行数据与问题分析；构建统一患者一项目输入；运行 P2 纯住院；并行运行 P3 的 α 网格与敏感性；运行小窗口精确对照和校准；独立验证 P2、P3；生成冻结清单；最后生成论文图表并用 XeLaTeX 编译。

冻结清单记录训练期、留出期、固定参数、主随机种子、5 个敏感性种子、P2/P3 推荐点和权威文件的一次性 SHA256。哈希只用于明确最终交付版本，不在开发过程中反复校验。

Listing 1 最终模型与结果冻结清单

```
1 {
2   "MODEL_AND_RESULTS_FROZEN": true,
3   "code_version": "working-tree pre-freeze audit; see git commit if committed",
4   "random_seed": 20260824,
5   "special_seed_experiment": [
6     11,
7     29,
8     47,
9     83,
10    131
11  ],
12  "training_period": [
13    "2019-03-01",
14    "2024-03-31"
15  ],
16  "evaluation_period": [
17    "2024-04-01",
18    "2025-03-31"
19  ],
20  "main_assumptions": {
21    "work_blocks": "08:00-12:00 and 13:00-17:00",
22    "slot_minutes": 5,
23    "transfer_minutes": 10,
24    "bladder_minutes": 60,
25    "preparation_granularity": "item",
26    "capability": "Table-1 project-level A/B/C reviewed mapping; D excluded",
27    "doctor_capacity": "training weekday-by-slot concurrent proxy",
28    "report_completion": "last scheduled task end proxy"
29  },
30  "p2": {
31    "scenario": "P2_INPATIENT_ONLY",
32    "selected_policy": "FCFS_SHARED",
33    "preparation_granularity": "item",
34    "metrics": {
35      "scenario": "P2_INPATIENT_ONLY",
36      "policy": "FCFS_SHARED",
37      "inpatient_events": 70771,
38      "inpatient_complete_48h": 60359,
39      "inpatient_48h_rate": 0.8528775911037006,
40      "inpatient_scheduled_complete": 60359,
41      "inpatient_unscheduled": 10412,
42      "inpatient_wait_p50_hours_conditional": 3.5833333333333335,
43      "inpatient_wait_p90_hours_conditional": 22.083333333333332,
44      "inpatient_wait_p95_hours_conditional": 23.666666666666668,
45      "inpatient_room_switch_rate_multi": 0.5811783598205881,
```

```

46     "background_events": 0,
47     "background_on_planned_day": 0,
48     "background_on_time_rate": NaN,
49     "background_scheduled_complete": 0,
50     "fallback_scheduled_task_share": 0.0,
51     "scheduled_minutes": 1344260.0,
52     "staffed_capacity_utilization": 0.5451010208529748,
53     "transfer_minutes_assumption": 10
54 }
55 },
56 "p3": {
57     "selection_rule": "minimum normalized Euclidean distance to the attainable Pareto
ideal",
58     "selected_alpha": 0.3,
59     "selected_metrics": {
60         "scenario": "P3_ALPHA_0.30",
61         "policy": "FCFS_SHARED",
62         "inpatient_events": 70771,
63         "inpatient_complete_48h": 42730,
64         "inpatient_48h_rate": 0.6037783838012746,
65         "inpatient_scheduled_complete": 42730,
66         "inpatient_unscheduled": 28041,
67         "inpatient_wait_p50_hours_conditional": 3.0,
68         "inpatient_wait_p90_hours_conditional": 21.75,
69         "inpatient_wait_p95_hours_conditional": 38.583333333333336,
70         "inpatient_room_switch_rate_multi": 0.2503737748491057,
71         "background_events": 96046,
72         "background_on_planned_day": 57177,
73         "background_on_time_rate": 0.5953084980113695,
74         "background_scheduled_complete": 57177,
75         "fallback_scheduled_task_share": 0.0,
76         "scheduled_minutes": 2066410.0,
77         "staffed_capacity_utilization": 0.8379347748953296,
78         "background_target_alpha": 0.3,
79         "quota_shortfall_days": 1,
80         "quota_target_events": 28977,
81         "quota_achieved_before_inpatient": 28970,
82         "preparation_granularity": "item",
83         "special_seed": 20260824,
84         "capability_mode": "hierarchical",
85         "capacity_column": "capacity_main",
86         "bladder_minutes": 60,
87         "transfer_minutes": 10,
88         "pareto_nondominated": true,
89         "distance_to_ideal": 0.915027740483536
90     }
91 },
92 "selected_alpha": 0.3,
93 "freeze_checks": {
94     "train_holdout_no_leakage": true,
95     "all_50_fallback_reviewed": true,
96     "no_blanket_category_fallback": true,
97     "bedside_semantics_passed": true,
98     "p2_exists": true,
99     "p3_pareto_exists": true,
100     "historical_calibration_exists": true,
101     "exact_benchmark_exists": true,
102     "preparation_sensitivity_exists": true,
103     "multi_seed_exists": true,
104     "input_semantics_passed": true,

```

```

105     "p2_schedule_verification_passed": true,
106     "p3_schedule_verification_passed": true,
107     "authoritative_result_unique": true
108 },
109 "authoritative_files": [
110     {
111         "path": "supporting_materials/results/final_frozen/p2_inpatient_only/summary.
112         json",
113         "sha256": "c0f56d73665de63866809d0aece0306ae6f55146b29ac0e0dca940bd4a299995"
114     },
115     {
116         "path": "supporting_materials/results/final_frozen/p2_inpatient_only/
117         final_patient_task_schedule.csv",
118         "sha256": "5041bc01d041b049da4a646c5fe85f7d216d07b1286d24d9c3711ec9b2b800ed"
119     },
120     {
121         "path": "supporting_materials/results/final_frozen/p2_inpatient_only/
122         final_patient_outcomes.csv",
123         "sha256": "dc5a8a52032d476c5f7f2b48b0e6afcb02159ed31bf2a47c0af271fb6e7699f6"
124     },
125     {
126         "path": "supporting_materials/results/final_frozen/p3_joint/selection.json",
127         "sha256": "335704ce743cade525801bbdc32e80b752d4c6e00828733429d4f266992ca12a"
128     },
129     {
130         "path": "supporting_materials/results/final_frozen/p3_joint/pareto_metrics.csv",
131         "sha256": "4ac5771da518746431c72fed0bdf7f87e5f8f25df5a1fdf46abdb8ade8d817aa"
132     },
133     {
134         "path": "supporting_materials/results/final_frozen/p3_joint/
135         final_patient_task_schedule.csv",
136         "sha256": "92063a75f7fb51ee46fef5ba13366a82a317e88fc662f1964f306defe282a3ee"
137     },
138     {
139         "path": "supporting_materials/results/final_frozen/p3_joint/
140         final_patient_outcomes.csv",
141         "sha256": "85e3d5610af39c92e86e47c9536dac47a5569abb0298435d87b1b132253ace29"
142     },
143     {
144         "path": "supporting_materials/results/final_frozen/sensitivity_metrics.csv",
145         "sha256": "dddd62023592b85f5b6e8e4dd5f064fe1a227e25a15ed7d99e007f96bbf43184"
146     },
147     {
148         "path": "supporting_materials/results/exact_benchmark/exact_vs_heuristic_metrics
149         .csv",
150         "sha256": "769db232f3f6665ffdb0858a0503f71ff11a796e539231d32e5d89d7862513d7"
151     },
152     {
153         "path": "supporting_materials/results/calibration/calibration_metrics.csv",
154         "sha256": "0dd2abe60e59f9a31e2d6127ab0fa929a9546b22c972a61255c625bfe70d22d0"
155     }
156 ]
157 }

```

B 数据审计与问题一程序

Listing 2 原始数据字段、范围与重复审计

```

1  """Field-level audit of the ultrasound scheduling competition data."""
2
3  from __future__ import annotations
4
5  import argparse
6  import gc
7  import json
8  import math
9  import re
10 from pathlib import Path
11 from typing import Any
12
13 import numpy as np
14 import pandas as pd
15
16
17 ROOT = Path(__file__).resolve().parents[2]
18 SUPPORTING_DIR = ROOT / "supporting_materials"
19 DEFAULT_RAW_DIR = SUPPORTING_DIR / "processed_data" / "raw_parquet"
20 DEFAULT_AUDIT_DIR = SUPPORTING_DIR / "tables" / "audit"
21 START_DATE = pd.Timestamp("2019-03-01")
22 END_DATE = pd.Timestamp("2025-03-31")
23 DETAIL_RENAME = {
24     "开单日期（年月日）": "order_date_raw",
25     "开单时间（00:00）": "order_time_raw",
26     "患者类型": "patient_type",
27     "病人ID": "patient_id",
28     "开单医生ID": "order_doctor_id",
29     "科室ID": "department_id",
30     "医嘱系统开单内容": "ordered_project",
31     "检查系统开单内容": "exam_project",
32     "检查报告出具日期（年月日）": "report_date_raw",
33     "检查报告出具时间（00:00）": "report_time_raw",
34     "检查人ID": "exam_doctor_id",
35     "检查超声的机器ID": "machine_id",
36 }
37
38
39 def clean_text(series: pd.Series) -> pd.Series:
40     return series.astype("string").str.strip()
41
42
43 def clean_id(series: pd.Series) -> pd.Series:
44     return (
45         clean_text(series)
46         .str.replace(r"^[\'\"]+|[\']+$", "", regex=True)
47         .str.replace(r"\.0$", "", regex=True)
48     )
49
50
51 def combine_datetime(date_series: pd.Series, time_series: pd.Series) -> pd.Series:
52     dates = pd.to_datetime(date_series, errors="coerce", format="mixed").dt.normalize()
53     parts = clean_text(time_series).str.extract(
54         r"(?P<hour>\d{1,2}):(?P<minute>\d{2}):(?: (?P<second>\d{2}(?:\.\d+)?)?)?",
55         expand=True,
56     )
57     hours = pd.to_numeric(parts["hour"], errors="coerce")
58     minutes = pd.to_numeric(parts["minute"], errors="coerce")
59     seconds = pd.to_numeric(parts["second"], errors="coerce").fillna(0)

```

```

60     elapsed = pd.to_timedelta(hours * 3600 + minutes * 60 + seconds, unit="s")
61     return dates + elapsed
62
63
64 def normalize_project(series: pd.Series) -> pd.Series:
65     normalized = clean_text(series).fillna("")
66     replacements = {
67         "(": "(",
68         ")": ")",
69         ",": ",",
70         " ": " ",
71         "\u2163": "IV",
72         "\u2162": "III",
73         "\u2161": "II",
74         "\u2160": "I",
75     }
76     for old, new in replacements.items():
77         normalized = normalized.str.replace(old, new, regex=False)
78     normalized = normalized.str.replace(r"^\[|\]$ ", "", regex=True)
79     normalized = normalized.str.replace(
80         r",(?:憋尿彩超|心脏彩超|彩超|B超)$", "", regex=True
81     )
82     return normalized.replace("", pd.NA)
83
84
85
86 def missing_counts(frame: pd.DataFrame) -> dict[str, int]:
87     result: dict[str, int] = {}
88     for column in frame.columns:
89         if column == "source_row":
90             continue
91         values = frame[column]
92         missing = values.isna()
93         if pd.api.types.is_string_dtype(values) or values.dtype == object:
94             missing = missing | clean_text(values).eq("")
95         result[column] = int(missing.sum())
96     return result
97
98
99 def finite_float(value: Any) -> float | None:
100     if value is None or pd.isna(value):
101         return None
102     number = float(value)
103     return number if math.isfinite(number) else None
104
105
106 def quantiles(series: pd.Series, probabilities: list[float]) -> dict[str, float | None]:
107     clean = series.dropna()
108     if clean.empty:
109         return {str(probability): None for probability in probabilities}
110     values = clean.quantile(probabilities)
111     return {str(probability): finite_float(values.loc[probability]) for probability in probabilities}
112
113
114 def top_values(series: pd.Series, limit: int, name: str) -> pd.DataFrame:
115     counts = clean_text(series).fillna("<缺失>").value_counts(dropna=False).head(limit)
116     return counts.rename_axis(name).reset_index(name="记录数")

```

```

117
118
119 def load_machine_map(raw_dir: Path) -> tuple[pd.DataFrame, dict[str, str]]:
120     mapping = pd.read_parquet(raw_dir / "machine_map.parquet")
121     mapping = mapping.rename(
122         columns={
123             "检查超声的机器ID": "machine_id",
124             "历史彩超诊室名称": "historical_room",
125             "现行彩超诊室名称": "current_room",
126         }
127     )
128     mapping["machine_id"] = clean_id(mapping["machine_id"])
129     mapping = mapping[mapping["machine_id"].str.fullmatch(r"\d+", na=False)].copy()
130     mapping = mapping.drop_duplicates("machine_id", keep="last")
131     room_map = dict(zip(mapping["machine_id"], clean_text(mapping["current_room"])))
132     return mapping, room_map
133
134
135 def audit_detail(
136     label: str,
137     path: Path,
138     room_map: dict[str, str],
139     tables_dir: Path,
140 ) -> tuple[dict[str, Any], pd.DataFrame, pd.DataFrame]:
141     frame = pd.read_parquet(path).rename(columns=DETAIL_RENAME)
142     for column in [
143         "patient_type",
144         "patient_id",
145         "order_doctor_id",
146         "department_id",
147         "ordered_project",
148         "exam_project",
149         "exam_doctor_id",
150         "machine_id",
151     ]:
152         frame[column] = clean_text(frame[column])
153     for column in [
154         "patient_id",
155         "order_doctor_id",
156         "department_id",
157         "exam_doctor_id",
158         "machine_id",
159     ]:
160         frame[column] = clean_id(frame[column])
161
162     frame["order_dt"] = combine_datetime(frame["order_date_raw"], frame["order_time_raw"])
163     frame["report_dt"] = combine_datetime(frame["report_date_raw"], frame["report_time_raw"])
164     frame["order_date"] = frame["order_dt"].dt.normalize()
165     frame["report_date"] = frame["report_dt"].dt.normalize()
166     frame["delay_hours"] = (frame["report_dt"] - frame["order_dt"]).dt.total_seconds() / 3600
167     frame["ordered_project_norm"] = normalize_project(frame["ordered_project"])
168     frame["exam_project_norm"] = normalize_project(frame["exam_project"])
169     frame["current_room"] = frame["machine_id"].map(room_map)
170     frame["machine_mapped"] = frame["current_room"].notna()
171     frame["machine_current_valid"] = frame["current_room"].notna() & ~frame["current_room"]
172     ].str.contains("不能对应", na=False)
173

```

```

174 official = frame["order_date"].between(START_DATE, END_DATE)
175 valid_delay = frame["delay_hours"].notna()
176 official_frame = frame.loc[official].copy()
177 duplicate_subset = [column for column in frame.columns if column != "source_row"]
178 duplicates = int(frame.duplicated(duplicate_subset, keep=False).sum())
179
180
181 project_comparison = (
182     frame["ordered_project_norm"].notna()
183     & frame["exam_project_norm"].notna()
184 )
185 project_equal = frame["ordered_project_norm"].eq(frame["exam_project_norm"])
186
187 valid_bundle_rows = official_frame[
188     official_frame["patient_id"].notna() & official_frame["order_dt"].notna()
189 ].copy()
190 bundle_keys = ["patient_id", "order_dt", "order_doctor_id", "department_id"]
191 bundles = (
192     valid_bundle_rows.groupby(bundle_keys, dropna=False)
193     .agg(
194         project_rows=("source_row", "size"),
195         report_rows=("report_dt", "count"),
196         completion_dt=("report_dt", "max"),
197         room_count=("machine_id", "nunique"),
198         exam_doctor_count=("exam_doctor_id", "nunique"),
199     )
200     .reset_index()
201 )
202 bundles["completion_hours"] = (
203     bundles["completion_dt"] - bundles["order_dt"]
204 ).dt.total_seconds() / 3600
205 bundles["all_rows_reported"] = bundles["report_rows"].eq(bundles["project_rows"])
206 bundles["complete_48h"] = (
207     bundles["all_rows_reported"]
208     & bundles["completion_hours"].between(0, 48, inclusive="both")
209 )
210 multi = bundles["project_rows"].gt(1)
211
212 daily_order = (
213     official_frame.groupby("order_date")
214     .agg(
215         detail_rows=("source_row", "size"),
216         unique_patients=("patient_id", "nunique"),
217     )
218     .reset_index()
219 )
220 daily_report = (
221     frame[frame["report_date"].between(START_DATE, END_DATE)]
222     .groupby("report_date")
223     .agg(
224         detail_rows=("source_row", "size"),
225         unique_patients=("patient_id", "nunique"),
226     )
227     .reset_index()
228     .rename(columns={"report_date": "date"})
229 )
230 bundle_daily = (
231     bundles.assign(order_date=bundles["order_dt"].dt.normalize())
232     .groupby("order_date")
233     .agg(application_bundles=("patient_id", "size"), complete_48h=("complete_48h",

```

```

234         "sum"))
235         .reset_index()
236     )
237     daily_order = daily_order.merge(bundle_daily, on="order_date", how="left")
238
239     top_values(frame["ordered_project"], 50, "医嘱系统开单内容").to_csv(
240         tables_dir / f"audit_{label}_ordered_projects_top50.csv", index=False,
241         encoding="utf-8-sig"
242     )
243     top_values(frame["exam_project"], 50, "检查系统开单内容").to_csv(
244         tables_dir / f"audit_{label}_exam_projects_top50.csv", index=False, encoding="
245         utf-8-sig"
246     )
247     top_values(frame["machine_id"], 100, "机器ID").to_csv(
248         tables_dir / f"audit_{label}_machine_ids.csv", index=False, encoding="utf-8-
249         sig"
250     )
251     if label == "inpatient":
252         top_values(official_frame["department_id"], 500, "科室ID").to_csv(
253             tables_dir / "audit_inpatient_department_demand.csv", index=False,
254             encoding="utf-8-sig"
255         )
256         bundle_distribution = (
257             bundles["project_rows"].value_counts().sort_index().rename_axis("项目记录数
258             ").reset_index(name="申请单数")
259         )
260         bundle_distribution.to_csv(
261             tables_dir / "audit_inpatient_bundle_size_distribution.csv", index=False,
262             encoding="utf-8-sig"
263         )
264
265     metrics: dict[str, Any] = {
266         "rows": int(len(frame)),
267         "columns": int(len(frame.columns)),
268         "missing": missing_counts(frame[list(DETAIL_RENAME.values())]),
269         "exact_duplicate_rows_including_all_copies": duplicates,
270         "order_date_min": str(frame["order_date"].min().date()) if frame["order_date"
271         ].notna().any() else None,
272         "order_date_max": str(frame["order_date"].max().date()) if frame["order_date"
273         ].notna().any() else None,
274         "report_date_min": str(frame["report_date"].min().date()) if frame["
275         report_date"].notna().any() else None,
276         "report_date_max": str(frame["report_date"].max().date()) if frame["
277         report_date"].notna().any() else None,
278         "rows_in_official_order_period": int(official.sum()),
279         "rows_outside_official_order_period": int((~official & frame["order_date"].
280         notna()).sum()),
281         "invalid_order_datetime": int(frame["order_dt"].isna().sum()),
282         "invalid_report_datetime": int(frame["report_dt"].isna().sum()),
283         "negative_report_delay_rows": int((frame["delay_hours"] < 0).sum()),
284         "report_delay_over_30_days_rows": int((frame["delay_hours"] > 24 * 30).sum()),
285         "delay_hours_quantiles_all_valid": quantiles(
286             frame.loc[valid_delay & frame["delay_hours"].ge(0), "delay_hours"],
287             [0, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99, 1],
288         ),
289         "delay_hours_quantiles_official_nonnegative": quantiles(
290             official_frame.loc[official_frame["delay_hours"].ge(0), "delay_hours"],
291             [0, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99, 1],
292         ),
293         "unique_patients_official_period": int(official_frame["patient_id"].nunique(

```

```

282     dropna=True)),
    "unique_order_doctors_official_period": int(official_frame["order_doctor_id"].
nunique(dropna=True)),
283     "unique_exam_doctors_official_period": int(official_frame["exam_doctor_id"].
nunique(dropna=True)),
284     "unique_departments_official_period": int(official_frame["department_id"].
nunique(dropna=True)),
285     "unique_machine_ids_all": int(frame["machine_id"].nunique(dropna=True)),
286     "machine_mapping_row_coverage": finite_float(frame["machine_mapped"].mean()),
287     "valid_current_room_row_coverage": finite_float(frame["machine_current_valid"
].mean()),
288     "ordered_project_unique_raw": int(frame["ordered_project"].nunique(dropna=True
)),
289     "exam_project_unique_raw": int(frame["exam_project"].nunique(dropna=True)),
290     "normalized_project_exact_match_rate_when_both_present": finite_float(
291         project_equal.loc[project_comparison].mean()
292     ),
293     "official_application_bundles": int(len(bundles)),
294     "official_bundle_48h_completion_rate": finite_float(bundles["complete_48h"].
mean()),
295     "official_bundle_negative_completion_hours": int((bundles["completion_hours"]
< 0).sum()),
296     "official_bundle_completion_hours_quantiles_nonnegative": quantiles(
297         bundles.loc[bundles["completion_hours"].ge(0), "completion_hours"],
298         [0, 0.25, 0.5, 0.75, 0.9, 0.95, 0.99, 1],
299     ),
300     "multi_project_bundle_count": int(multi.sum()),
301     "multi_project_single_room_rate": finite_float(bundles.loc[multi, "room_count"
].le(1).mean()),
302     "multi_project_single_exam_doctor_rate": finite_float(
303         bundles.loc[multi, "exam_doctor_count"].le(1).mean()
304     ),
305     "patients_with_multiple_application_bundles": int(
306         bundles.groupby("patient_id").size().gt(1).sum()
307     ),
308 }
309
310 del official_frame, valid_bundle_rows
311 gc.collect()
312 return metrics, daily_order, daily_report
313
314
315 def audit_daily_counts(path: Path) -> tuple[dict[str, Any], pd.DataFrame]:
316     frame = pd.read_parquet(path)
317     frame["date"] = pd.to_datetime(frame["日期"], errors="coerce", format="mixed").dt.
normalize()
318     numeric_columns = [
319         "住院病人人数",
320         "住院超声检查人次",
321         "住院超声检查项目数",
322         "门诊病人人数",
323         "门诊超声检查人次",
324         "门诊超声检查项目数",
325     ]
326     for column in numeric_columns:
327         frame[column] = pd.to_numeric(frame[column], errors="coerce")
328     expected = pd.date_range(START_DATE, END_DATE, freq="D")
329     observed = pd.DatetimeIndex(frame["date"].dropna().unique())
330     missing_dates = expected.difference(observed)
331     metrics = {

```

```

332     "rows": int(len(frame)),
333     "date_min": str(frame["date"].min().date()),
334     "date_max": str(frame["date"].max().date()),
335     "duplicate_dates": int(frame["date"].duplicated(keep=False).sum()),
336     "missing_dates_in_official_period": [str(value.date()) for value in
missing_dates],
337     "missing": missing_counts(frame),
338     "numeric_quantiles": {
339         column: quantiles(frame[column], [0, 0.25, 0.5, 0.75, 0.95, 1])
340         for column in numeric_columns
341     },
342 }
343 return metrics, frame
344
345
346 def audit_doctor_day(path: Path) -> tuple[dict[str, Any], pd.DataFrame]:
347     frame = pd.read_parquet(path)
348     frame["doctor_id"] = clean_id(frame["检查人ID"])
349     frame["date"] = pd.to_datetime(frame["检查日期"], errors="coerce", format="mixed").
dt.normalize()
350     frame["first_dt"] = pd.to_datetime(frame["最早检查时间"], errors="coerce", format="
mixed")
351     frame["last_dt"] = pd.to_datetime(frame["最晚检查时间"], errors="coerce", format="
mixed")
352     frame["project_count"] = pd.to_numeric(frame["部位数"], errors="coerce")
353     frame["span_hours"] = (frame["last_dt"] - frame["first_dt"]).dt.total_seconds() /
3600
354     daily_staff = frame.groupby("date")["doctor_id"].nunique().rename("working_doctors
").reset_index()
355     metrics = {
356         "rows": int(len(frame)),
357         "date_min": str(frame["date"].min().date()),
358         "date_max": str(frame["date"].max().date()),
359         "unique_exam_doctors": int(frame["doctor_id"].nunique(dropna=True)),
360         "duplicate_doctor_date_rows": int(frame.duplicated(["doctor_id", "date"], keep
=False).sum()),
361         "missing": missing_counts(frame[["doctor_id", "date", "project_count", "
first_dt", "last_dt"]]),
362         "negative_span_rows": int((frame["span_hours"] < 0).sum()),
363         "zero_span_rows": int(frame["span_hours"].eq(0).sum()),
364         "span_over_16_hours_rows": int((frame["span_hours"] > 16).sum()),
365         "span_hours_quantiles": quantiles(frame["span_hours"], [0, 0.25, 0.5, 0.75,
0.95, 0.99, 1]),
366         "project_count_quantiles": quantiles(frame["project_count"], [0, 0.25, 0.5,
0.75, 0.95, 0.99, 1]),
367         "working_doctors_per_day_quantiles": quantiles(
368             daily_staff["working_doctors"], [0, 0.25, 0.5, 0.75, 0.95, 1]
369         ),
370     }
371     return metrics, daily_staff
372
373
374 def audit_equipment(raw_dir: Path, mapping: pd.DataFrame) -> dict[str, Any]:
375     equipment = pd.read_parquet(raw_dir / "equipment_capability.parquet")
376     equipment["machine_id"] = clean_id(equipment["检查超声的机器ID"])
377     equipment["room"] = clean_text(equipment["彩超诊室名称"])
378     equipment["model"] = clean_text(equipment["机器型号"])
379     purchase_raw = clean_text(equipment["设备购置时间"])
380     numeric = pd.to_numeric(purchase_raw, errors="coerce")
381     parsed = pd.to_datetime(purchase_raw, errors="coerce", format="mixed")

```

```

382 serial_dates = pd.Timestamp("1899-12-30") + pd.to_timedelta(numeric, unit="D")
383 equipment["purchase_date"] = parsed.fillna(serial_dates)
384 return {
385     "equipment_rows": int(len(equipment)),
386     "unique_machine_ids": int(equipment["machine_id"].nunique(dropna=True)),
387     "unique_current_rooms": int(equipment["room"].nunique(dropna=True)),
388     "missing_machine_models": int(equipment["model"].isna().sum()),
389     "invalid_purchase_dates": int(equipment["purchase_date"].isna().sum()),
390     "machine_map_rows_after_numeric_id_filter": int(len(mapping)),
391     "machine_map_current_room_valid": int(
392         (~mapping["current_room"].str.contains("不能对应", na=False)).sum()
393     ),
394 }
395
396
397 def correlation(left: pd.Series, right: pd.Series) -> float | None:
398     valid = left.notna() & right.notna()
399     if valid.sum() < 3:
400         return None
401     return finite_float(left.loc[valid].corr(right.loc[valid]))
402
403
404 def reconcile_daily(
405     daily_counts: pd.DataFrame,
406     inpatient_order: pd.DataFrame,
407     inpatient_report: pd.DataFrame,
408     outpatient_order: pd.DataFrame,
409     outpatient_report: pd.DataFrame,
410     output: Path,
411 ) -> dict[str, Any]:
412     base = daily_counts[["date", "住院超声检查人次", "住院超声检查项目数", "门诊超声检查人次",
413         "门诊超声检查项目数"]].copy()
414     for prefix, frame, date_column in [
415         ("in_order", inpatient_order, "order_date"),
416         ("in_report", inpatient_report, "date"),
417         ("out_order", outpatient_order, "order_date"),
418         ("out_report", outpatient_report, "date"),
419     ]:
420         renamed = frame.rename(columns={date_column: "date"}).copy()
421         renamed = renamed.rename(
422             columns={column: f"{prefix}_{column}" for column in renamed.columns if
423             column != "date"}
424         )
425         base = base.merge(renamed, on="date", how="left")
426     base.to_csv(output, index=False, encoding="utf-8-sig")
427     return {
428         "inpatient_project_count_vs_order_detail_rows_correlation": correlation(
429             base["住院超声检查项目数"], base["in_order_detail_rows"]
430         ),
431         "inpatient_project_count_vs_report_detail_rows_correlation": correlation(
432             base["住院超声检查项目数"], base["in_report_detail_rows"]
433         ),
434         "inpatient_visits_vs_order_unique_patients_correlation": correlation(
435             base["住院超声检查人次"], base["in_order_unique_patients"]
436         ),
437         "inpatient_visits_vs_report_unique_patients_correlation": correlation(
438             base["住院超声检查人次"], base["in_report_unique_patients"]
439         ),
440         "outpatient_project_count_vs_order_detail_rows_correlation": correlation(
441             base["门诊超声检查项目数"], base["out_order_detail_rows"]
442         ),
443         "outpatient_visits_vs_order_unique_patients_correlation": correlation(
444             base["门诊超声检查人次"], base["out_order_unique_patients"]
445         ),
446         "outpatient_visits_vs_report_unique_patients_correlation": correlation(
447             base["门诊超声检查人次"], base["out_report_unique_patients"]
448         ),
449     }

```

```

440     ),
441     "outpatient_project_count_vs_report_detail_rows_correlation": correlation(
442         base["门诊超声检查项目数"], base["out_report_detail_rows"]
443     ),
444     "outpatient_visits_vs_order_unique_patients_correlation": correlation(
445         base["门诊超声检查人次"], base["out_order_unique_patients"]
446     ),
447     "outpatient_visits_vs_report_unique_patients_correlation": correlation(
448         base["门诊超声检查人次"], base["out_report_unique_patients"]
449     ),
450 }
451
452
453 def main() -> None:
454     parser = argparse.ArgumentParser()
455     parser.add_argument("--raw-dir", type=Path, default=DEFAULT_RAW_DIR)
456     parser.add_argument(
457         "--output-json",
458         type=Path,
459         default=DEFAULT_AUDIT_DIR / "data_audit_metrics.json",
460     )
461     parser.add_argument("--tables-dir", type=Path, default=DEFAULT_AUDIT_DIR)
462     args = parser.parse_args()
463     args.tables_dir.mkdir(parents=True, exist_ok=True)
464
465     mapping, room_map = load_machine_map(args.raw_dir)
466     results: dict[str, Any] = {"period": {"start": str(START_DATE.date()), "end": str(
467         END_DATE.date())}}
468     detail_daily: dict[str, tuple[pd.DataFrame, pd.DataFrame]] = {}
469     for label, filename in [
470         ("inpatient", "inpatient.parquet"),
471         ("physical_exam", "physical_exam.parquet"),
472         ("outpatient", "outpatient.parquet"),
473     ]:
474         print(f"Auditing {label}", flush=True)
475         metrics, daily_order, daily_report = audit_detail(
476             label, args.raw_dir / filename, room_map, args.tables_dir
477         )
478         results[label] = metrics
479         detail_daily[label] = (daily_order, daily_report)
480         gc.collect()
481
482     daily_metrics, daily_counts = audit_daily_counts(args.raw_dir / "daily_counts.
483     parquet")
484     results["daily_counts"] = daily_metrics
485     doctor_metrics, daily_staff = audit_doctor_day(args.raw_dir / "doctor_day.parquet"
486     )
487     results["doctor_day"] = doctor_metrics
488     daily_staff.to_csv(
489         args.tables_dir / "audit_daily_working_doctors.csv", index=False, encoding="
490         utf-8-sig"
491     )
492     results["equipment_and_mapping"] = audit_equipment(args.raw_dir, mapping)
493     results["daily_reconciliation"] = reconcile_daily(
494         daily_counts,
495         detail_daily["inpatient"][0],
496         detail_daily["inpatient"][1],
497         detail_daily["outpatient"][0],
498         detail_daily["outpatient"][1],
499         args.tables_dir / "audit_daily_reconciliation.csv",

```

```

496 )
497
498 args.output_json.parent.mkdir(parents=True, exist_ok=True)
499 args.output_json.write_text(
500     json.dumps(results, ensure_ascii=False, indent=2), encoding="utf-8"
501 )
502 print(f"Wrote {args.output_json}")
503
504
505 if __name__ == "__main__":
506     main()

```

Listing 3 问题一事件、需求、时长与医生效率分析

```

1  """P1 demand, project-family, and workload-proxy analysis.
2
3  The source workbooks and frozen Parquet files are read-only inputs. Results and
4  figures are written below supporting_materials by default.
5  """
6
7  from __future__ import annotations
8
9  import argparse
10 import json
11 import math
12 import re
13 from pathlib import Path
14 from typing import Any
15
16 import matplotlib
17
18 matplotlib.use("Agg")
19 import matplotlib.pyplot as plt
20 import numpy as np
21 import pandas as pd
22
23
24 ROOT = Path(__file__).resolve().parents[2]
25 SUPPORTING_DIR = ROOT / "supporting_materials"
26 SEED = 20260823
27 START_DATE = pd.Timestamp("2019-03-01")
28 END_DATE = pd.Timestamp("2025-03-31")
29 VALIDATION_START = pd.Timestamp("2024-04-01")
30 VALIDATION_END = pd.Timestamp("2025-03-31")
31
32 DETAIL_RENAME = {
33     "开单日期（年月日）": "order_date_raw",
34     "开单时间（00:00）": "order_time_raw",
35     "患者类型": "patient_type",
36     "病人ID": "patient_id",
37     "开单医生ID": "order_doctor_id",
38     "科室ID": "department_id",
39     "医嘱系统开单内容": "ordered_project",
40     "检查系统开单内容": "exam_project",
41     "检查报告出具日期（年月日）": "report_date_raw",
42     "检查报告出具时间（00:00）": "report_time_raw",
43     "检查人ID": "exam_doctor_id",
44     "检查超声的机器ID": "machine_id",
45 }
46

```

```

47 SOURCE_FILES = {
48     "住院": "inpatient.parquet",
49     "门诊": "outpatient.parquet",
50     "体检": "physical_exam.parquet",
51 }
52
53 NON_SERVICE_PATTERN = (
54     r"^(?:床旁(?:B超检查|彩色多普勒超声检查)加收|"
55     r"彩色胶片打印|超声计算机图文报告)$"
56 )
57
58 # The first matching rule wins. Obstetric rules precede cardiac because
59 # "胎儿心脏" is an obstetric level-IV item rather than a routine cardiac item.
60 CATEGORY_RULES = [
61     {
62         "category": "非服务计费/报告项",
63         "pattern": NON_SERVICE_PATTERN,
64         "basis": "整条内容仅为床旁加收、胶片打印或图文报告，不单独占用扫描时间；床旁标记仍保留",
65         "lower": 0.0,
66         "nominal": 0.0,
67         "upper": 0.0,
68     },
69     {
70         "category": "产科III/IV级",
71         "pattern": r"(?:III|IV|\u2162|\u2163|三|四)级产科|胎儿心脏|系统产前超声",
72         "basis": "题面明确III/IV级产科为30--40分钟；胎儿心脏归入IV级资源族",
73         "lower": 30.0,
74         "nominal": 35.0,
75         "upper": 40.0,
76     },
77     {
78         "category": "产科I/II级",
79         "pattern": r"(?:I|II|\u2160|\u2161|一|二)级产科|早孕|胎儿生物物理|母胎医学|宫颈管长度|产科超声",
80         "basis": "题面明确I/II级产科为15--20分钟；其余产科筛查按同资源族管理",
81         "lower": 15.0,
82         "nominal": 17.5,
83         "upper": 20.0,
84     },
85     {
86         "category": "心脏",
87         "pattern": r"心脏|心包|心动图|冠脉|室壁|瓣膜|经食道|经食管",
88         "basis": "题面明确心脏超声为10--15分钟；心脏声学造影仍属于心脏项目，故优先于一般“造影”规则",
89         "lower": 10.0,
90         "nominal": 12.5,
91         "upper": 15.0,
92     },
93     {
94         "category": "介入/定位",
95         "pattern": r"穿刺|活检|置管|造影|定位|介入|引导",
96         "basis": "在排除心脏项目后，操作性项目按介入/定位资源管理；题面未另给时长，采用其他项目区间",
97         "lower": 5.0,
98         "nominal": 7.5,
99         "upper": 10.0,
100     },
101     {
102         "category": "血管",
103         "pattern": r"血管|动脉|静脉|大隐|小隐|血流图|加血流",

```

```

104         "basis": "由项目名称识别血管专用资源族；题面未另给时长，采用其他项目区间",
105         "lower": 5.0,
106         "nominal": 7.5,
107         "upper": 10.0,
108     },
109     {
110         "category": "经阴道/腔内",
111         "pattern": r"经阴道|腔内彩超|妇科B超（经阴道）|妇科B超\\（经阴道\\)",
112         "basis": "由检查路径识别腔内资源族；题面未另给时长，采用其他项目区间",
113         "lower": 5.0,
114         "nominal": 7.5,
115         "upper": 10.0,
116     },
117     {
118         "category": "泌尿/经腹妇科",
119         "pattern": r"泌尿|输尿管|膀胱|前列腺|双肾|肾彩超|尿量|经腹妇科|妇科B超（经腹）|妇科B超\\（经腹\\）|盆底",
120         "basis": "题面要求泌尿系和经腹妇科考虑充盈膀胱约束；时长采用其他项目区间",
121         "lower": 5.0,
122         "nominal": 7.5,
123         "upper": 10.0,
124     },
125     {
126         "category": "腹部/胃肠",
127         "pattern": r"腹部|腹水|腹盆|肝|胆|胰|脾|胃|肠道|阑尾|肠系膜|腹腔|腹膜后|膜腹后|肾上腺",
128         "basis": "题面要求腹部及肠系膜项目空腹并优先上午；时长采用其他项目区间",
129         "lower": 5.0,
130         "nominal": 7.5,
131         "upper": 10.0,
132     },
133     {
134         "category": "浅表器官",
135         "pattern": r"甲状腺|甲状旁腺|乳腺|淋巴结|体表|包块|阴囊|睾丸|附睾|涎腺|浅表|腹股沟|双眼",
136         "basis": "由项目部位识别通用浅表资源族；时长采用其他项目区间",
137         "lower": 5.0,
138         "nominal": 7.5,
139         "upper": 10.0,
140     },
141     {
142         "category": "儿科专项",
143         "pattern": r"婴儿头颅|新生儿|髋关节",
144         "basis": "设备清单显示儿科专项能力差异；题面未另给时长，采用其他项目区间",
145         "lower": 5.0,
146         "nominal": 7.5,
147         "upper": 10.0,
148     },
149     {
150         "category": "一般其他",
151         "pattern": r".*",
152         "basis": "未命中前述资源族，按题面其他项目5--10分钟",
153         "lower": 5.0,
154         "nominal": 7.5,
155         "upper": 10.0,
156     },
157 ]
158
159 CATEGORY_ORDER = [rule["category"] for rule in CATEGORY_RULES]
160 CATEGORY_RANK = {category: index for index, category in enumerate(reversed(

```

```

CATEGORY_ORDER))}
161 DURATION = {
162     rule["category"]: (rule["lower"], rule["nominal"], rule["upper"])
163     for rule in CATEGORY_RULES
164 }
165
166 FLAG_PATTERNS = {
167     "床旁标记": r"床旁",
168     "空腹上午标记": r"腹部|肝胆|胃肠|肠道|阑尾|肠系膜|胰|脾",
169     "充盈膀胱标记": r"泌尿|输尿管|膀胱|前列腺|经腹妇科|妇科B超（经腹）|妇科B超\（经腹\）|盆底",
170 }
171
172
173 def clean_text(series: pd.Series) -> pd.Series:
174     return series.astype("string").str.strip()
175
176
177 def clean_id(series: pd.Series) -> pd.Series:
178     return (
179         clean_text(series)
180         .str.replace(r"^\['']+|[\']+$", "", regex=True)
181         .str.replace(r"\.0$", "", regex=True)
182     )
183
184
185 def combine_datetime(date_series: pd.Series, time_series: pd.Series) -> pd.Series:
186     dates = pd.to_datetime(date_series, errors="coerce", format="mixed").dt.normalize()
187     parts = clean_text(time_series).str.extract(
188         r"(?P<hour>\d{1,2}):(?P<minute>\d{2})(?::(?P<second>\d{2}(?:\.\d+)??))?",
189         expand=True,
190     )
191     seconds = (
192         pd.to_numeric(parts["hour"], errors="coerce") * 3600
193         + pd.to_numeric(parts["minute"], errors="coerce") * 60
194         + pd.to_numeric(parts["second"], errors="coerce").fillna(0)
195     )
196     return dates + pd.to_timedelta(seconds, unit="s")
197
198
199 def normalize_project(series: pd.Series) -> pd.Series:
200     result = clean_text(series).fillna("")
201     for old, new in {
202         "(": "(",
203         ")": ")",
204         ",": ",",
205         " ": "",
206         " ": "",
207         "\u2163": "IV",
208         "\u2162": "III",
209         "\u2161": "II",
210         "\u2160": "I",
211     }.items():
212         result = result.str.replace(old, new, regex=False)
213     result = result.str.replace(r"^\[|\]$ ", "", regex=True)
214     result = result.str.replace(r",(?:憋尿彩超|心脏彩超|彩超|B超)$", "", regex=True)
215     return result.replace("", pd.NA)
216
217
218 def expand_embedded_project_lists(frame: pd.DataFrame) -> pd.DataFrame:

```

```

219     """Split the source system's explicit ```` multi-item encoding.
220
221     A single order field can contain several separately billed examinations,
222     encoded as ``[item,modality][item,modality]``. Only the bracket boundary
223     is used as a separator; commas and Chinese enumeration marks inside an
224     examination name are retained. This preserves compound anatomy names
225     while making the scheduling task count agree with the source semantics.
226     """
227     expanded = frame.copy()
228     expanded["ordered_project_original"] = expanded["ordered_project"]
229     expanded["ordered_project"] = clean_text(expanded["ordered_project"]).str.split(
230         r"\s*\[", regex=True
231     )
232     expanded = expanded.explode("ordered_project", ignore_index=True)
233     expanded["ordered_project"] = (
234         clean_text(expanded["ordered_project"])
235         .str.replace(r"^\[\],]+|[\[\],]+$", "", regex=True)
236     )
237     expanded["source_component_index"] = expanded.groupby("source_row").cumcount() + 1
238     return expanded
239
240
241 def classify_projects(series: pd.Series) -> pd.DataFrame:
242     normalized = normalize_project(series)
243     category = pd.Series("一般其他", index=series.index, dtype="string")
244     unmatched = pd.Series(True, index=series.index)
245     for rule in CATEGORY_RULES[:-1]:
246         hit = normalized.str.contains(rule["pattern"], case=False, regex=True, na=
False) & unmatched
247         category.loc[hit] = rule["category"]
248         unmatched.loc[hit] = False
249     result = pd.DataFrame({"project_norm": normalized, "category": category})
250     result["non_service_item"] = normalized.str.fullmatch(NON_SERVICE_PATTERN, na=
False)
251     for flag, pattern in FLAG_PATTERNS.items():
252         result[flag] = normalized.str.contains(pattern, regex=True, na=False)
253     result["duration_lower_min"] = result["category"].map(lambda x: DURATION[x][0])
254     result["duration_nominal_min"] = result["category"].map(lambda x: DURATION[x][1])
255     result["duration_upper_min"] = result["category"].map(lambda x: DURATION[x][2])
256     return result
257
258
259 def load_detail(path: Path, source: str) -> pd.DataFrame:
260     frame = pd.read_parquet(path).rename(columns=DETAIL_RENAME)
261     frame = expand_embedded_project_lists(frame)
262     for column in [
263         "patient_id",
264         "order_doctor_id",
265         "department_id",
266         "exam_doctor_id",
267         "machine_id",
268     ]:
269         frame[column] = clean_id(frame[column])
270     frame["order_dt"] = combine_datetime(frame["order_date_raw"], frame["
order_time_raw"])
271     frame["report_dt"] = combine_datetime(frame["report_date_raw"], frame["
report_time_raw"])
272     frame["source"] = source
273     classes = classify_projects(frame["ordered_project"])
274     frame = pd.concat([frame, classes], axis=1)

```

```

275     frame["category_rank"] = frame["category"].map(CATEGORY_RANK).fillna(0).astype(int
276 )
277     frame["category_score"] = frame["duration_nominal_min"] * 100 + frame["
278 category_rank"]
279     return frame
280
281 def make_project_catalog(
282     frame: pd.DataFrame,
283     start_date: pd.Timestamp = START_DATE,
284     end_date: pd.Timestamp = END_DATE,
285 ) -> pd.DataFrame:
286     frame = frame[
287         frame["order_dt"].notna()
288         & frame["order_dt"].dt.normalize().between(start_date, end_date)
289     ].copy()
290     columns = [
291         "source",
292         "ordered_project",
293         "project_norm",
294         "category",
295         "床旁标记",
296         "空腹上午标记",
297         "充盈膀胱标记",
298         "non_service_item",
299         "duration_lower_min",
300         "duration_nominal_min",
301         "duration_upper_min",
302     ]
303     return (
304         frame.groupby(columns, dropna=False)
305         .size()
306         .rename("record_count")
307         .reset_index()
308         .sort_values(["source", "record_count"], ascending=[True, False])
309     )
310
311 def make_order_episodes(frame: pd.DataFrame) -> pd.DataFrame:
312     valid = frame[
313         frame["order_dt"].notna()
314         & frame["order_dt"].dt.normalize().between(START_DATE, END_DATE)
315     ].copy()
316     valid["patient_key"] = valid["patient_id"].fillna("缺失_")
317     missing = valid["patient_id"].isna()
318     valid.loc[missing, "patient_key"] = "缺失_" + valid.loc[missing, "source_row"].
319     astype(str)
320     keys = ["source", "patient_key", "order_dt"]
321     summary = (
322         valid.groupby(keys, dropna=False)
323         .agg(
324             project_row_count=("source_row", "size"),
325             category_count=("category", "nunique"),
326             nonservice_record_count=("non_service_item", "sum"),
327             bedside=("床旁标记", "max"),
328             fasting_before_10=("空腹上午标记", "max"),
329             requires_bladder=("充盈膀胱标记", "max"),
330         )
331         .reset_index()
332     )

```

```

332 unique_projects = valid.drop_duplicates(keys + ["project_norm"]).copy()
333 service_counts = (
334     unique_projects[~unique_projects["non_service_item"]]
335     .groupby(keys, dropna=False)
336     .size()
337     .rename("unique_project_count")
338     .reset_index()
339 )
340 nonservice_counts = (
341     unique_projects[unique_projects["non_service_item"]]
342     .groupby(keys, dropna=False)
343     .size()
344     .rename("unique_nonservice_item_count")
345     .reset_index()
346 )
347 workload = (
348     unique_projects.groupby(keys, dropna=False)
349     .agg(
350         workload_lower_min=("duration_lower_min", "sum"),
351         nominal_workload_min=("duration_nominal_min", "sum"),
352         workload_upper_min=("duration_upper_min", "sum"),
353     )
354     .reset_index()
355 )
356 primary = (
357     valid.sort_values("category_score")
358     .drop_duplicates(keys, keep="last")[keys + ["category"]]
359     .rename(columns={"category": "primary_category"})
360 )
361 result = (
362     summary.merge(service_counts, on=keys, how="left")
363     .merge(nonservice_counts, on=keys, how="left")
364     .merge(workload, on=keys, how="left")
365     .merge(primary, on=keys, how="left")
366 )
367 result[["unique_project_count", "unique_nonservice_item_count"]] = result[
368     ["unique_project_count", "unique_nonservice_item_count"]
369 ].fillna(0).astype(int)
370 return result
371
372
373 def make_report_episodes(frame: pd.DataFrame) -> pd.DataFrame:
374     valid = frame[
375         frame["order_dt"].notna()
376         & frame["report_dt"].notna()
377         & frame["exam_doctor_id"].notna()
378         & frame["machine_id"].notna()
379     ].copy()
380     valid["patient_key"] = valid["patient_id"].fillna("缺失_")
381     missing = valid["patient_id"].isna()
382     valid.loc[missing, "patient_key"] = "缺失_" + valid.loc[missing, "source_row"].
383     astype(str)
384     keys = ["source", "patient_key", "order_dt", "exam_doctor_id", "machine_id"]
385     summary = (
386         valid.groupby(keys, dropna=False)
387         .agg(
388             report_dt=("report_dt", "max"),
389             project_row_count=("source_row", "size"),
390             category_count=("category", "nunique"),
391             nonservice_record_count=("non_service_item", "sum"),

```

```

391         bedside=("床旁标记", "max"),
392         fasting_before_10=("空腹上午标记", "max"),
393         requires_bladder=("充盈膀胱标记", "max"),
394     )
395     .reset_index()
396 )
397 unique_projects = valid.drop_duplicates(keys + ["project_norm"]).copy()
398 service_counts = (
399     unique_projects[~unique_projects["non_service_item"]]
400     .groupby(keys, dropna=False)
401     .size()
402     .rename("unique_project_count")
403     .reset_index()
404 )
405 nonservice_counts = (
406     unique_projects[unique_projects["non_service_item"]]
407     .groupby(keys, dropna=False)
408     .size()
409     .rename("unique_nonservice_item_count")
410     .reset_index()
411 )
412 workload = (
413     unique_projects.groupby(keys, dropna=False)
414     .agg(
415         workload_lower_min=("duration_lower_min", "sum"),
416         nominal_workload_min=("duration_nominal_min", "sum"),
417         workload_upper_min=("duration_upper_min", "sum"),
418     )
419     .reset_index()
420 )
421 primary = (
422     valid.sort_values("category_score")
423     .drop_duplicates(keys, keep="last")[keys + ["category"]]
424     .rename(columns={"category": "primary_category"})
425 )
426 result = (
427     summary.merge(service_counts, on=keys, how="left")
428     .merge(nonservice_counts, on=keys, how="left")
429     .merge(workload, on=keys, how="left")
430     .merge(primary, on=keys, how="left")
431 )
432 result[["unique_project_count", "unique_nonservice_item_count"]] = result[
433     ["unique_project_count", "unique_nonservice_item_count"]
434 ].fillna(0).astype(int)
435 return result
436
437
438 def multiproject_sensitivity(episodes: pd.DataFrame, basis: str) -> pd.DataFrame:
439     rows: list[dict[str, Any]] = []
440     for (source,), group in episodes.groupby(["source"]):
441         multiple = group["unique_project_count"].gt(1)
442         for coefficient in [1.0, 0.9, 0.8]:
443             adjusted = group["nominal_workload_min"].where(
444                 ~multiple,
445                 group["nominal_workload_min"] * coefficient,
446             )
447             rows.append(
448                 {
449                     "source": source,
450                     "basis": basis,

```

```

451         "multiproject_coefficient": coefficient,
452         "episode_count": int(len(group)),
453         "multiproject_episode_count": int(multiple.sum()),
454         "multiproject_episode_share": float(multiple.mean()),
455         "total_nominal_workload_min": float(adjusted.sum()),
456         "mean_nominal_workload_per_episode_min": float(adjusted.mean()),
457     }
458 )
459 return pd.DataFrame(rows)
460
461
462 def complete_daily(
463     daily: pd.DataFrame,
464     source: str,
465     basis: str,
466 ) -> pd.DataFrame:
467     calendar = pd.DataFrame({"date": pd.date_range(START_DATE, END_DATE, freq="D")})
468     out = calendar.merge(daily, on="date", how="left")
469     out["episode_count"] = out["episode_count"].fillna(0).astype(int)
470     out["project_row_count"] = out["project_row_count"].fillna(0).astype(int)
471     out.insert(0, "basis", basis)
472     out.insert(0, "source", source)
473     return out
474
475
476 def aggregate_trends(daily: pd.DataFrame, frequency: str) -> pd.DataFrame:
477     frame = daily.copy()
478     if frequency == "weekly":
479         frame["period"] = frame["date"].dt.to_period("W-SUN").dt.start_time
480     elif frequency == "monthly":
481         frame["period"] = frame["date"].dt.to_period("M").dt.start_time
482     elif frequency == "annual":
483         frame["period"] = frame["date"].dt.to_period("Y").dt.start_time
484     else:
485         raise ValueError(frequency)
486     result = (
487         frame.groupby(["source", "basis", "period"], as_index=False)
488         .agg(
489             episode_count=("episode_count", "sum"),
490             project_row_count=("project_row_count", "sum"),
491             calendar_days=("date", "size"),
492             mean_daily_episodes=("episode_count", "mean"),
493             median_daily_episodes=("episode_count", "median"),
494             p90_daily_episodes=("episode_count", lambda x: x.quantile(0.9)),
495         )
496     )
497     if frequency == "annual":
498         result["full_calendar_year"] = result["period"].dt.year.between(2020, 2024)
499     return result
500
501
502 def calendar_design(dates: pd.Series, origin: pd.Timestamp) -> np.ndarray:
503     dates = pd.Series(pd.to_datetime(dates)).reset_index(drop=True)
504     n = len(dates)
505     trend = (dates - origin).dt.days.to_numpy(dtype=float) / 365.25
506     day_of_year = dates.dt.dayofyear.to_numpy(dtype=float)
507     columns: list[np.ndarray] = [
508         np.ones(n),
509         trend,
510         np.sin(2 * np.pi * day_of_year / 365.25),

```

```

511     np.cos(2 * np.pi * day_of_year / 365.25),
512 ]
513 weekday = dates.dt.dayofweek.to_numpy()
514 month = dates.dt.month.to_numpy()
515 columns.extend((weekday == value).astype(float) for value in range(1, 7))
516 columns.extend((month == value).astype(float) for value in range(2, 13))
517 return np.column_stack(columns)
518
519
520 def metric_row(actual: np.ndarray, predicted: np.ndarray) -> dict[str, float]:
521     residual = predicted - actual
522     abs_error = np.abs(residual)
523     denominator = np.maximum(np.abs(actual) + np.abs(predicted), 1e-9)
524     return {
525         "MAE": float(np.mean(abs_error)),
526         "RMSE": float(np.sqrt(np.mean(residual**2))),
527         "WAPE": float(abs_error.sum() / max(np.abs(actual).sum(), 1e-9)),
528         "sMAPE": float(np.mean(2 * abs_error / denominator)),
529         "mean_bias": float(np.mean(residual)),
530     }
531
532
533 def chronological_validation(daily: pd.DataFrame) -> pd.DataFrame:
534     results: list[dict[str, Any]] = []
535     series_frames: list[tuple[str, str, pd.DataFrame]] = []
536     for (source, basis), group in daily.groupby(["source", "basis"]):
537         series_frames.append((source, basis, group.sort_values("date").copy()))
538     for basis, group in daily.groupby("basis"):
539         combined = (
540             group.groupby("date", as_index=False)["episode_count"].sum().sort_values("
541             date")
542             )
543         series_frames.append(("三类合计", basis, combined))
544
545     for source, basis, frame in series_frames:
546         frame = frame.sort_values("date").reset_index(drop=True)
547         y = frame["episode_count"].to_numpy(dtype=float)
548         dates = frame["date"]
549         train_mask = dates.lt(VALIDATION_START).to_numpy()
550         test_mask = dates.between(VALIDATION_START, VALIDATION_END).to_numpy()
551         actual = y[train_mask]
552
553         predictions: dict[str, np.ndarray] = {}
554         predictions["7日季节朴素"] = pd.Series(y).shift(7).to_numpy()[test_mask]
555
556         same_weekday_median = np.full(len(frame), np.nan)
557         for index in np.flatnonzero(test_mask):
558             history_index = [index - 7 * lag for lag in range(1, 9) if index - 7 * lag
559             >= 0]
560             same_weekday_median[index] = np.median(y[history_index])
561         predictions["过去8周同星期中位数"] = same_weekday_median[test_mask]
562
563         design_train = calendar_design(dates[train_mask], START_DATE)
564         response_train = np.log1p(y[train_mask])
565         ridge = 1e-4 * np.eye(design_train.shape[1])
566         ridge[0, 0] = 0
567         coefficient = np.linalg.solve(
568             design_train.T @ design_train + ridge,
569             design_train.T @ response_train,
570         )

```

```

569     design_test = calendar_design(dates[test_mask], START_DATE)
570     predictions["日历趋势对数岭回归"] = np.maximum(np.expml(design_test @
coefficient), 0)
571
572     for model, predicted in predictions.items():
573         finite = np.isfinite(predicted) & np.isfinite(actual)
574         metrics = metric_row(actual[finite], predicted[finite])
575         results.append(
576             {
577                 "source": source,
578                 "basis": basis,
579                 "model": model,
580                 "train_start": str(dates[train_mask].min().date()),
581                 "train_end": str(dates[train_mask].max().date()),
582                 "validation_start": str(dates[test_mask].min().date()),
583                 "validation_end": str(dates[test_mask].max().date()),
584                 "validation_days": int(finite.sum()),
585                 **metrics,
586             }
587         )
588     result = pd.DataFrame(results)
589     result["best_WAPE_within_series"] = result.groupby(["source", "basis"])["WAPE"].
transform("min").eq(result["WAPE"])
590     return result
591
592
593 def peak_flat_summary(daily: pd.DataFrame) -> pd.DataFrame:
594     frames: list[tuple[str, str, pd.DataFrame]] = []
595     for (source, basis), group in daily.groupby(["source", "basis"]):
596         frames.append((source, basis, group.copy()))
597     for basis, group in daily.groupby("basis"):
598         frames.append(
599             (
600                 "三类合计",
601                 basis,
602                 group.groupby("date", as_index=False)["episode_count"].sum(),
603             )
604         )
605     rows: list[dict[str, Any]] = []
606     weekdays = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
607     for source, basis, frame in frames:
608         training = frame[
609             frame["date"].between(START_DATE, VALIDATION_START - pd.Timedelta(days=1))
610         ].copy()
611         holdout = frame[frame["date"].between(VALIDATION_START, VALIDATION_END)].copy()
612
613         training["weekday"] = training["date"].dt.dayofweek
614         training["month"] = training["date"].dt.month
615         values = training["episode_count"]
616         q45, q50, q55, q90, q95 = values.quantile([0.45, 0.5, 0.55, 0.9, 0.95])
617         peak_candidates = training[training["episode_count"].ge(q90)].copy()
618         peak_candidates["distance"] = (peak_candidates["episode_count"] - q90).abs()
619         flat_candidates = training[
620             training["episode_count"].between(q45, q55, inclusive="both")
621         ].copy()
622         flat_candidates["distance"] = (flat_candidates["episode_count"] - q50).abs()
623         peak_row = peak_candidates.sort_values(["distance", "date"]).iloc[0]
624         flat_row = flat_candidates.sort_values(["distance", "date"]).iloc[0]
625         weekday_median = training.groupby("weekday")["episode_count"].median()
        month_median = training.groupby("month")["episode_count"].median()

```

```

626         rows.append(
627             {
628                 "source": source,
629                 "basis": basis,
630                 "period": f"{START_DATE.date()}至{(VALIDATION_START - pd.Timedelta(
days=1)).date()}",
631                 "threshold_data_role": "training_only",
632                 "q45": float(q45),
633                 "median_flat_level": float(q50),
634                 "q55": float(q55),
635                 "q90_peak_threshold": float(q90),
636                 "q95_stress_threshold": float(q95),
637                 "representative_peak_date": str(peak_row["date"].date()),
638                 "representative_peak_count": int(peak_row["episode_count"]),
639                 "representative_flat_date": str(flat_row["date"].date()),
640                 "representative_flat_count": int(flat_row["episode_count"]),
641                 "peak_weekday": weekdays[int(weekday_median.idxmax())],
642                 "peak_weekday_median": float(weekday_median.max()),
643                 "peak_calendar_month": int(month_median.idxmax()),
644                 "peak_month_daily_median": float(month_median.max()),
645                 "holdout_peak_day_share": float(holdout["episode_count"].ge(q90).mean
()),
646                 "holdout_stress_day_share": float(holdout["episode_count"].ge(q95).
mean()),
647                 "holdout_flat_day_share": float(
648                     holdout["episode_count"].between(q45, q55, inclusive="both").mean
649                     ),
650             }
651         )
652     return pd.DataFrame(rows)
653
654
655 def median_absolute_deviation(series: pd.Series) -> float:
656     values = series.dropna().to_numpy(dtype=float)
657     if len(values) == 0:
658         return math.nan
659     median = np.median(values)
660     return float(np.median(np.abs(values - median)))
661
662
663 def report_gap_proxy(episodes: pd.DataFrame) -> tuple[pd.DataFrame, dict[str, Any], pd
.DataFrame]:
664     frame = episodes[
665         episodes["report_dt"].dt.normalize().between(START_DATE, END_DATE)
666         & episodes["exam_doctor_id"].notna()
667         & episodes["nominal_workload_min"].gt(0)
668     ].copy()
669     frame["report_date"] = frame["report_dt"].dt.normalize()
670     frame = frame.sort_values(["exam_doctor_id", "report_date", "report_dt"])
671     frame["previous_report_dt"] = frame.groupby(["exam_doctor_id", "report_date"])["
report_dt"].shift(1)
672     frame["report_gap_min"] = (frame["report_dt"] - frame["previous_report_dt"]).dt.
total_seconds() / 60
673     hour = frame["report_dt"].dt.hour + frame["report_dt"].dt.minute / 60
674     valid = frame["report_gap_min"].gt(0) & frame["report_gap_min"].le(60) & hour.
between(6, 22)
675     frame["valid_gap_proxy"] = valid
676     valid_frame = frame[valid].copy()
677     category_proxy_frame = valid_frame[valid_frame["unique_project_count"].eq(1)].copy

```

```

678 grouped = (
679     category_proxy_frame.groupby("primary_category")["report_gap_min"]
680     .agg(
681         proxy_n="size",
682         proxy_median_min="median",
683         proxy_q25_min=lambda x: x.quantile(0.25),
684         proxy_q75_min=lambda x: x.quantile(0.75),
685         proxy_mad_min=median_absolute_deviation,
686     )
687     .reset_index()
688 )
689 overall_median = float(category_proxy_frame["report_gap_min"].median())
690 grouped["relative_burden_index"] = grouped["proxy_median_min"] / overall_median
691 diagnostics = {
692     "all_report_episodes": int(len(frame)),
693     "episodes_with_previous_same_doctor_day": int(frame["report_gap_min"].notna().
694 sum()),
695     "zero_or_negative_gap_count": int(frame["report_gap_min"].le(0).sum()),
696     "gap_over_60_min_count": int(frame["report_gap_min"].gt(60).sum()),
697     "valid_gap_proxy_count": int(valid.sum()),
698     "valid_gap_proxy_share_of_all_episodes": float(valid.mean()),
699     "single_project_valid_gap_proxy_count": int(len(category_proxy_frame)),
700     "single_project_valid_gap_proxy_share": float(len(category_proxy_frame) / max(
701 valid.sum(), 1)),
702     "overall_single_project_valid_gap_median_min": overall_median,
703     "interpretation": "类别代理仅使用单项目事件；相邻报告间隔仍含扫查、书写、等待和批量出报
704 告效应，只能作为相对负荷代理，不能解释为服务时长。",
705 }
706 return grouped, diagnostics, frame
707
708 def doctor_efficiency(
709     episodes_with_gaps: pd.DataFrame,
710     doctor_day_path: Path,
711 ) -> tuple[pd.DataFrame, dict[str, Any], pd.DataFrame]:
712     frame = episodes_with_gaps.copy()
713     frame["report_date"] = frame["report_dt"].dt.normalize()
714     frame = frame.sort_values(["exam_doctor_id", "report_date", "report_dt"])
715     day = (
716         frame.groupby(["exam_doctor_id", "report_date"], as_index=False)
717         .agg(
718             episode_count=("patient_key", "size"),
719             project_row_count=("project_row_count", "sum"),
720             nominal_workload_min=("nominal_workload_min", "sum"),
721             first_report=("report_dt", "min"),
722             last_report=("report_dt", "max"),
723             first_episode_nominal=("nominal_workload_min", "first"),
724         )
725     )
726     day["report_span_min"] = (day["last_report"] - day["first_report"]).dt.
727 total_seconds() / 60
728     day["effective_window_min"] = day["report_span_min"] + day["first_episode_nominal"]
729     day["proxy_productivity"] = day["nominal_workload_min"] / day["
730 effective_window_min"]
731     valid = (
732         day["episode_count"].ge(3)
733         & day["report_span_min"].gt(0)
734         & day["report_span_min"].le(16 * 60)

```

```

731         & np.isfinite(day["proxy_productivity"])
732         & day["proxy_productivity"].gt(0)
733     )
734     valid_day = day[valid].copy()
735     raw_log = np.log(valid_day["proxy_productivity"])
736     low, high = raw_log.quantile([0.01, 0.99])
737     valid_day["log_productivity_winsor"] = raw_log.clip(low, high)
738
739     pooled_within_variance = float(
740         valid_day.groupby("exam_doctor_id")["log_productivity_winsor"].var().median()
741     )
742     if not math.isfinite(pooled_within_variance) or pooled_within_variance <= 0:
743         pooled_within_variance = float(valid_day["log_productivity_winsor"].var())
744     doctors = (
745         valid_day.groupby("exam_doctor_id")
746         .agg(
747             valid_days=("report_date", "size"),
748             total_episodes=("episode_count", "sum"),
749             mean_log_productivity=("log_productivity_winsor", "mean"),
750             variance_log_productivity=("log_productivity_winsor", "var"),
751             raw_proxy_productivity=("proxy_productivity", "median"),
752         )
753         .reset_index()
754     )
755     doctors["variance_log_productivity"] = doctors["variance_log_productivity"].fillna(
        pooled_within_variance
    )
756     doctors["sampling_variance"] = doctors["variance_log_productivity"] / doctors["
        valid_days"]
757     mu0 = float(np.average(doctors["mean_log_productivity"], weights=doctors["
        valid_days"])))
758     between_observed = float(doctors["mean_log_productivity"].var(ddof=1))
759     mean_sampling = float(doctors["sampling_variance"].mean())
760     tau2 = max(between_observed - mean_sampling, 1e-6)
761     doctors["shrinkage_weight"] = tau2 / (tau2 + doctors["sampling_variance"])
762     doctors["posterior_log_productivity"] = (
763         doctors["shrinkage_weight"] * doctors["mean_log_productivity"]
764         + (1 - doctors["shrinkage_weight"]) * mu0
765     )
766     doctors["posterior_variance"] = 1 / (1 / tau2 + 1 / doctors["sampling_variance"])
767     doctors["raw_relative_throughput_factor"] = np.exp(doctors["mean_log_productivity"]
        - mu0)
768     doctors["shrunk_relative_throughput_factor"] = np.exp(doctors["
        posterior_log_productivity"] - mu0)
769     posterior_sd = np.sqrt(doctors["posterior_variance"])
770     doctors["factor_ci95_lower"] = np.exp(doctors["posterior_log_productivity"] - mu0
        - 1.96 * posterior_sd)
771     doctors["factor_ci95_upper"] = np.exp(doctors["posterior_log_productivity"] - mu0
        + 1.96 * posterior_sd)
772     doctors = doctors.sort_values("shrunk_relative_throughput_factor", ascending=False)
773
774     raw_doctor_day = pd.read_parquet(doctor_day_path).rename(
775         columns={
776             "检查人ID": "exam_doctor_id",
777             "检查日期": "report_date",
778             "部位数": "listed_project_count",
779             "最早检查时间": "listed_first_report",
780             "最晚检查时间": "listed_last_report",
781         }
782     )

```

```

783 raw_doctor_day["exam_doctor_id"] = clean_id(raw_doctor_day["exam_doctor_id"])
784 raw_doctor_day["report_date"] = pd.to_datetime(raw_doctor_day["report_date"],
785 errors="coerce", format="mixed").dt.normalize()
786 raw_doctor_day["listed_project_count"] = pd.to_numeric(raw_doctor_day["
listed_project_count"], errors="coerce")
787 raw_doctor_day["listed_first_report"] = pd.to_datetime(raw_doctor_day["
listed_first_report"], errors="coerce", format="mixed")
788 raw_doctor_day["listed_last_report"] = pd.to_datetime(raw_doctor_day["
listed_last_report"], errors="coerce", format="mixed")
789 reconciliation = raw_doctor_day.merge(
790     day[
791         "exam_doctor_id",
792         "report_date",
793         "project_row_count",
794         "episode_count",
795         "first_report",
796         "last_report",
797     ]
798 ],
799 on=["exam_doctor_id", "report_date"],
800 how="left",
801 )
802 matched = reconciliation[
803     reconciliation["listed_project_count"].notna()
804     & reconciliation["project_row_count"].notna()
805 ].copy()
806 first_abs_minutes = (matched["listed_first_report"] - matched["first_report"]).dt.
total_seconds().abs() / 60
807 last_abs_minutes = (matched["listed_last_report"] - matched["last_report"]).dt.
total_seconds().abs() / 60
808 diagnostics = {
809     "doctor_day_rows": int(len(day)),
810     "valid_doctor_days_for_efficiency": int(len(valid_day)),
811     "doctor_count_estimated": int(len(doctors)),
812     "daily_validity_rule": "至少3个报告事件，报告跨度大于0且不超过16小时；日对数生产率在1%
与99%分位温莎化。",
813     "empirical_bayes_prior_log_mean": mu0,
814     "empirical_bayes_between_doctor_variance": tau2,
815     "table6_matched_doctor_days": int(len(matched)),
816     "table6_project_count_correlation": float(matched["listed_project_count"].corr
(matched["project_row_count"])),
817     "table6_project_count_MAE": float((matched["listed_project_count"] - matched["
project_row_count"]).abs().mean()),
818     "table6_first_time_median_abs_difference_min": float(first_abs_minutes.median
()),
819     "table6_last_time_median_abs_difference_min": float(last_abs_minutes.median())
820 ,
821     "interpretation": "因报告时刻包含非扫查时间，因子仅表示历史报告吞吐代理；当前医生名单未
给出时不能直接映射为未来个人速度。",
822 }
823 return doctors, diagnostics, reconciliation
824
825 def duration_table(gap_proxy: pd.DataFrame, category_volume: pd.DataFrame) -> pd.
DataFrame:
826     rows = []
827     for priority, rule in enumerate(CATEGORY_RULES, start=1):
828         rows.append(
829             {

```

```

830         "priority": priority,
831         "category": rule["category"],
832         "keyword_regex": rule["pattern"],
833         "classification_basis": rule["basis"],
834         "duration_lower_min": rule["lower"],
835         "duration_nominal_min": rule["nominal"],
836         "duration_upper_min": rule["upper"],
837         "robust_schedule_parameter_min": rule["upper"],
838         "special_case_frequency_from_problem": 0.05,
839         "special_case_extra_duration_known": False,
840         "multiproject_default_coefficient": 1.0,
841         "multiproject_sensitivity_lower": 0.8,
842         "multiproject_sensitivity_upper": 1.0,
843     }
844 )
845 result = pd.DataFrame(rows)
846 total_volume = category_volume.groupby("category")["record_count"].sum().rename("
all_source_record_count")
847 result = result.merge(total_volume, on="category", how="left")
848 result = result.merge(gap_proxy, left_on="category", right_on="primary_category",
how="left").drop(columns="primary_category")
849 return result
850
851
852 def finite_json(value: Any) -> Any:
853     if isinstance(value, dict):
854         return {str(key): finite_json(item) for key, item in value.items()}
855     if isinstance(value, list):
856         return [finite_json(item) for item in value]
857     if isinstance(value, (np.integer,)):
858         return int(value)
859     if isinstance(value, (np.floating, float)):
860         return float(value) if math.isfinite(float(value)) else None
861     if isinstance(value, (pd.Timestamp,)):
862         return value.isoformat()
863     return value
864
865
866 def set_plot_style() -> None:
867     plt.rcParams.update(
868         {
869             "font.sans-serif": ["SimHei", "Microsoft YaHei", "Arial Unicode MS", "
DejaVu Sans"],
870             "axes.unicode_minus": False,
871             "font.size": 10,
872             "axes.spines.top": False,
873             "axes.spines.right": False,
874             "figure.dpi": 150,
875             "savefig.dpi": 220,
876         }
877     )
878
879
880 def make_figures(
881     daily: pd.DataFrame,
882     monthly: pd.DataFrame,
883     peak_flat: pd.DataFrame,
884     durations: pd.DataFrame,
885     doctors: pd.DataFrame,
886     figures_dir: Path,

```

```

887 ) -> None:
888     set_plot_style()
889     colors = {"住院": "#1f77b4", "门诊": "#d95f02", "体检": "#2a9d8f"}
890
891     fig, ax = plt.subplots(figsize=(11, 5.2))
892     order = daily[daily["basis"].eq("开单日")].copy()
893     for source, group in order.groupby("source"):
894         group = group.sort_values("date")
895         rolling = group["episode_count"].rolling(28, center=True, min_periods=14).mean()
896         ax.plot(group["date"], rolling, label=f"{source} (28日中心均值)", color=colors[source], linewidth=1.4)
897     ax.axvline(VALIDATION_START, color="#555555", linestyle="--", linewidth=1, label="时间顺序验证起点")
898     ax.set_ylabel("每日开单事件数")
899     ax.set_title("三类患者开单需求长期趋势")
900     ax.legend(ncol=2, frameon=False)
901     ax.grid(axis="y", alpha=0.2)
902     fig.tight_layout()
903     fig.savefig(figures_dir / "daily_order_trend.png", bbox_inches="tight")
904     plt.close(fig)
905
906     fig, axes = plt.subplots(2, 1, figsize=(11, 7.2), sharex=True)
907     for axis, basis in zip(axes, ["开单日", "报告日"]):
908         selected = monthly[monthly["basis"].eq(basis)]
909         for source, group in selected.groupby("source"):
910             axis.plot(group["period"], group["mean_daily_episodes"], label=source, color=colors[source], linewidth=1.3)
911             axis.set_ylabel("月内日均事件数")
912             axis.set_title(f"{basis}口径")
913             axis.grid(axis="y", alpha=0.2)
914     axes[0].legend(ncol=3, frameon=False)
915     axes[-1].set_xlabel("月份")
916     fig.suptitle("开单需求与报告吞吐的月度趋势（口径分离）", y=0.995)
917     fig.tight_layout()
918     fig.savefig(figures_dir / "monthly_order_report_trend.png", bbox_inches="tight")
919     plt.close(fig)
920
921     recent_order = daily[
922         daily["basis"].eq("开单日") & daily["date"].between(VALIDATION_START, VALIDATION_END)
923     ].copy()
924     recent_order["weekday"] = recent_order["date"].dt.dayofweek
925     profile = recent_order.groupby(["source", "weekday"])["episode_count"].median().reset_index()
926     fig, ax = plt.subplots(figsize=(9.5, 4.8))
927     labels = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
928     for source, group in profile.groupby("source"):
929         ax.plot(group["weekday"], group["episode_count"], marker="o", label=source, color=colors[source])
930     ax.set_xticks(range(7), labels)
931     ax.set_ylabel("日开单事件中位数")
932     ax.set_title("验证期星期负荷剖面")
933     ax.legend(frameon=False)
934     ax.grid(axis="y", alpha=0.2)
935     fig.tight_layout()
936     fig.savefig(figures_dir / "weekday_order_profile.png", bbox_inches="tight")
937     plt.close(fig)
938
939     plot_duration = durations.sort_values("duration_nominal_min")

```

```

940 fig, ax = plt.subplots(figsize=(10, 6))
941 y = np.arange(len(plot_duration))
942 error = np.vstack(
943     [
944         plot_duration["duration_nominal_min"] - plot_duration["duration_lower_min"]
945     ],
946     [
947         plot_duration["duration_upper_min"] - plot_duration["duration_nominal_min"]
948     ]
949 )
950 ax.errorbar(
951     plot_duration["duration_nominal_min"],
952     y,
953     xerr=error,
954     fmt="o",
955     color="#1f77b4",
956     ecolor="#7f8c8d",
957     capsize=3,
958     label="题面下界/中点/上界",
959 )
960 proxy_scaled = plot_duration["relative_burden_index"] * 7.5
961 ax.scatter(proxy_scaled, y, marker="x", color="#d95f02", label="报告间隔相对指数×7.5 (非时长)")
962 ax.set_yticks(y, plot_duration["category"])
963 ax.set_xlabel("分钟或相对指数缩放值")
964 ax.set_title("调度时长参数与报告间隔代理的边界")
965 ax.legend(frameon=False)
966 ax.grid(axis="x", alpha=0.2)
967 fig.tight_layout()
968 fig.savefig(figures_dir / "duration_and_gap_proxy.png", bbox_inches="tight")
969 plt.close(fig)
970
971 fig, ax = plt.subplots(figsize=(9.5, 4.8))
972 factors = doctors["shrunk_relative_throughput_factor"].sort_values().reset_index(drop=True)
973 ax.plot(np.arange(1, len(factors) + 1), factors, marker="o", markersize=3, linewidth=1, color="#2a9d8f")
974 ax.axhline(1, color="#555555", linestyle="--", linewidth=1)
975 ax.set_xlabel("历史医生（按收缩因子排序）")
976 ax.set_ylabel("相对报告吞吐因子")
977 ax.set_title("经验贝叶斯收缩后的医生历史吞吐代理")
978 ax.grid(axis="y", alpha=0.2)
979 fig.tight_layout()
980 fig.savefig(figures_dir / "doctor_throughput_factors.png", bbox_inches="tight")
981 plt.close(fig)
982
983 def markdown_table(frame: pd.DataFrame, columns: list[str], limit: int | None = None)
984     -> str:
985     selected = frame[columns].copy()
986     if limit is not None:
987         selected = selected.head(limit)
988     header = "|" + "|".join(columns) + "|"
989     divider = "|" + "|".join(["---"] * len(columns)) + "|"
990     rows = []
991     for _, row in selected.iterrows():
992         values = []
993         for value in row:
994             if pd.isna(value):
995                 values.append("不适用")

```

```

994         elif isinstance(value, float):
995             values.append(f"{value:.3f}")
996         else:
997             values.append(str(value))
998         rows.append("|" + "|".join(values) + "|")
999     return "\n".join([header, divider, *rows])
1000
1001
1002 def write_report(
1003     output_path: Path,
1004     summary: dict[str, Any],
1005     peak_flat: pd.DataFrame,
1006     validation: pd.DataFrame,
1007     durations: pd.DataFrame,
1008     doctors: pd.DataFrame,
1009     weekly: pd.DataFrame,
1010     annual: pd.DataFrame,
1011     multiproject: pd.DataFrame,
1012     nonservice: pd.DataFrame,
1013 ) -> None:
1014     combined_order = peak_flat[
1015         peak_flat["source"].eq("三类合计") & peak_flat["basis"].eq("开单日")
1016     ].iloc[0]
1017     best = validation[validation["best_WAPE_within_series"]].sort_values(["source", "basis"])
1018     duration_view = durations[
1019         [
1020             "category",
1021             "duration_lower_min",
1022             "duration_nominal_min",
1023             "duration_upper_min",
1024             "proxy_n",
1025             "proxy_median_min",
1026             "relative_burden_index",
1027         ]
1028     ]
1029     factor_quantiles = doctors["shrunk_relative_throughput_factor"].quantile([0.05, 0.5, 0.95])
1030     full_years = annual[annual["full_calendar_year"]].copy()
1031     full_years["year"] = full_years["period"].dt.year
1032     annual_view = full_years[
1033         ["source", "basis", "year", "episode_count", "mean_daily_episodes"]
1034     ]
1035     weekly_peak = (
1036         weekly.sort_values("mean_daily_episodes", ascending=False)
1037         .groupby(["source", "basis"], as_index=False)
1038         .first()[["source", "basis", "period", "episode_count", "mean_daily_episodes"]
1039     ])
1040     weekly_peak["period"] = weekly_peak["period"].dt.date.astype(str)
1041     multi_view = multiproject[
1042         [
1043             "source",
1044             "basis",
1045             "multiproject_coefficient",
1046             "multiproject_episode_share",
1047             "total_nominal_workload_min",
1048             "mean_nominal_workload_per_episode_min",
1049         ]
1050     ]

```

```

1051     nonservice_view = nonservice[
1052         ["source", "ordered_project", "record_count", "bedside_record_count"]
1053     ]
1054     text = f"""\# P1 数据分析技术报告
1055
1056 ## 1. 结论摘要
1057
1058 本分析覆盖 2019-03-01 至 2025-03-31。三类患者按开单日合计的训练期平峰中位数为 {combined_order
    ['median_flat_level']:.0f} 个开单事件/日，90% 高峰阈值为 {combined_order['
    q90_peak_threshold']:.0f} 个/日，95% 压力阈值为 {combined_order['
    q95_stress_threshold']:.0f} 个/日；代表性平峰日为 {combined_order['
    representative_flat_date']} ({combined_order['representative_flat_count']} 个)，代
    表性高峰日为 {combined_order['representative_peak_date']} ({combined_order['
    representative_peak_count']} 个)。阈值和代表日只由 2019-03-01 至 2024-03-31 的训练期确
    定，最后12个月仅检验这些阈值在未来时段的表现。
1059
1060 项目分类采用“互斥项目族 + 非互斥资源标记”：项目族决定基础时长与设备能力，床旁、空腹上午、充盈膀胱
    三个标记决定地点或时段约束。规则逐条按优先级匹配，项目字典保留原名、规范名、命中类别和记录量，能
    够逐项追溯。
1061
1062 独立计费/报告条目不作为检查项目排程。只有规范化文本完整匹配“床旁B超检查加收”“床旁彩色多普勒超声检
    查加收”“彩色胶片打印”或“超声计算机图文报告”时才置0分钟；组合文本只要还含真实扫查项目，就按
    真实项目分类和计时。床旁加收虽然为0分钟，仍保留床旁资源标记并与同开单事件中的真实项目绑定。正式
    期中数量如下：
1063
1064 {markdown_table(nonservice_view, ['source', 'ordered_project', 'record_count', '
    bedside_record_count'])}
1065
1066 题目数据没有检查开始时刻。开单到报告时长包含排队、预约、检查、书写和报告发布，不能当作服务时长。本分
    析只把单项目事件中、同一医生同日报告事件的相邻间隔作为类别相对工作量代理，并将其限制在大于0且不
    超过60分钟；最终调度分钟参数仍直接取题面时长区间的中点，稳健情景取上界。5%特殊病例频率来自题
    面，但题面未给额外时长倍率，因此不人为设置倍率。
1067
1068 历史医生的收缩后相对报告吞吐因子 5%、50%、95% 分位分别为 {factor_quantiles.loc[0.05]:.3f}、{
    factor_quantiles.loc[0.5]:.3f}、{factor_quantiles.loc[0.95]:.3f}。该因子已按有效医生
    日做经验贝叶斯收缩，但仍是报告吞吐代理，不是临床扫查速度，也不能在缺少当前医生ID映射时直接用于点
    名排班。
1069
1070 ## 2. 数据口径
1071
1072 - 开单事件：同一患者、同一精确开单时刻合并为一个事件；同一事件的服务项目按规范名称去重，纯计费/报告
    条目不计服务项目数。
1073 - 报告事件：同一患者、同一开单时刻、同一检查医生和同一机器构成一个连续检查事件，完成时刻取该组最后报
    告时刻；事件内按规范项目名称去重后逐项累加时长。
1074 - 开单日序列描述到达需求；报告日序列描述历史完成吞吐。两者分别输出，未把报告日吞吐误称为到达需求。
1075 - 日序列补齐完整自然日，未出现记录的日期按0处理；周、月、年均由完整日序列汇总。
1076 - 2019年仅从3月开始，2025年仅到3月结束，年度总量横向比较应限于2020--2024完整年份。
1077
1078 ## 3. 项目分类与调度时长
1079
1080 {markdown_table(duration_view, list(duration_view.columns))}
1081
1082 说明：`proxy_median_min` 是单项目报告事件间隔的稳健中位数，含等待和书写；`
    relative_burden_index` 仅用于检验类别相对负荷，不替代题面时长。多项目事件默认按规范项目去重
    后逐项累加题面参数，并在同一兼容检查室连续安排。题面没有给出同次检查的固定节约比例，因此默认合并
    系数为1.0；0.8和0.9只作为敏感性情景：
1083
1084 {markdown_table(multi_view, ['source', 'basis', 'multiproject_coefficient', '
    multiproject_episode_share', 'total_nominal_workload_min', '
    mean_nominal_workload_per_episode_min'])}

```

1085

1086 **## 4. 高峰和平峰识别**

1087

1088 `{markdown_table(peak_flat, ['source', 'basis', 'median_flat_level', 'q90_peak_threshold', 'q95_stress_threshold', 'representative_flat_date', 'representative_peak_date', 'peak_weekday', 'peak_calendar_month'])}`

1089

1090 高峰阈值取训练期日事件数**90%**分位，压力阈值取**95%**分位；平峰水平取训练期中位数，代表日也只从训练期选择。留出期仅统计落入训练期平峰、高峰和压力阈值的日期比例。月份用月内“每日事件数中位数”排序，避免月份天数不同造成总量偏差。

1091

1092 完整日序列同时按周、月、年汇总。各序列在全观察期内的日均负荷最高周如下；周首日期为周一：

1093

1094 `{markdown_table(weekly_peak, ['source', 'basis', 'period', 'episode_count', 'mean_daily_episodes'])}`

1095

1096 完整年份的年度结果如下。**2019**年从**3**月开始、**2025**年到**3**月结束，因此未混入完整年份比较：

1097

1098 `{markdown_table(annual_view, ['source', 'basis', 'year', 'episode_count', 'mean_daily_episodes'])}`

1099

1100 **## 5. 时间顺序验证**

1101

1102 训练集固定为 **2019-03-01** 至 **2024-03-31**，验证集固定为 **2024-04-01** 至 **2025-03-31**，未随机打乱。比较**7**日季节朴素、滚动一步的过去**8**周同星期中位数、只使用日历和长期趋势的对数岭回归。滚动一步模型在每个验证日只使用该日以前的真实观测。下表列出每条序列按**WAPE**最优的基线：

1103

1104 `{markdown_table(best, ['source', 'basis', 'model', 'MAE', 'RMSE', 'WAPE', 'sMAPE', 'mean_bias'])}`

1105

1106 此验证用于判断季节/星期结构能否在未来时间段复现，不把其中任何一个基线当成最终排程需求预测器。最终模型应至少以最优基线为对照，并在滚动窗口中重估。

1107

1108 **## 6. 工作量代理与医生收缩估计**

1109

1110 报告间隔代理先按“同一医生、同一报告日”排序，仅保留 `**[0, 60]**` 分钟间隔，并只用单项目事件形成类别统计。医生日生产率定义为去重项目题面名义时长之和除以有效报告窗口；多项目默认系数为**1.0**。有效窗口为最晚减最早报告时刻，再补首个事件名义时长。至少**3**个事件、跨度不超过**16**小时的医生日进入估计，日对数生产率在**1%**和**99%**分位温莎化。

1111

1112 对每名医生的平均对数生产率采用正态层次模型的经验贝叶斯收缩：先由医生间方差扣除平均抽样方差估计先验方差，再按各医生的日内波动和有效天数形成数据权重。输出包括原始相对因子、收缩因子和**95%**区间。该处理降低少量医生日导致的极端值，但不消除报告流程和病例组合造成的混杂。

1113

1114 表6复核结果：匹配医生日 `{summary['doctor_efficiency']['table6_matched_doctor_days']}` 条，明细项目记录数与表6部位数相关系数为 `{summary['doctor_efficiency']['table6_project_count_correlation']:.4f}`，两端报告时刻中位绝对差分别为 `{summary['doctor_efficiency']['table6_first_time_median_abs_difference_min']:.2f}` 和 `{summary['doctor_efficiency']['table6_last_time_median_abs_difference_min']:.2f}` 分钟。该一致性支持“报告事件重建”用于吞吐分析，但不改变其非服务时长属性。

1115

1116 **## 7. 可用于后续模型的参数与边界**

1117

1118 1. 基础时长使用 ``service_duration_parameters.csv`` 的中点；同一事件各去重服务项目默认逐项求和并连续同室安排，容量压力测试使用逐项上界之和；``non_service_item=True`` 的独立条目为**0**分钟。

1119 2. 多项目合并系数默认**1.0**，**0.8--1.0**只用于敏感性分析，不将“同室连续”解释为零增量同时扫描。

1120 3. **5%**特殊病例单独占用上界槽位或设情景变量；在没有额外时长证据时不外推倍率。

1121 4. 医生收缩因子只适合历史敏感性分析。若后续获得当前**33**名医生**ID**对应和班表，才可逐人应用；否则使用总体因子**1**并以因子分布做情景。

1122 5. 体检历史机器**ID**几乎不能对应现行诊室时，项目族分析仍可用于总工作量，但不能据此恢复旧房间的精确能

力。

1123 6. 报告事件分组依赖“患者+精确开单时刻+医生+机器”。同分钟多张申请单可能被合并，故排程应对事件粒度另做口径敏感性分析。

1124

1125 **## 8. 复现**

1126

1127 在工作区根目录执行：

1128

1129 ```powershell

1130 python supporting_materials/code/analyze_p1.py

1131 ```

1132

1133 随机种子固定为 {SEED}。脚本只读取 `supporting\_materials/processed\_data/raw\_parquet/`，结果写入 `supporting\_materials/results/p1/`，图件写入 `supporting\_materials/figures/p1/`。

1134 """

1135 output_path.write_text(text, encoding="utf-8")

1136

1137

1138 def main() -> None:

1139 parser = argparse.ArgumentParser()

1140 parser.add_argument(

1141 "--raw-dir",

1142 type=Path,

1143 default=SUPPORTING_DIR / "processed_data" / "raw_parquet",

1144)

1145 parser.add_argument(

1146 "--output-dir",

1147 type=Path,

1148 default=SUPPORTING_DIR / "results" / "p1",

1149)

1150 parser.add_argument(

1151 "--figures-dir",

1152 type=Path,

1153 default=SUPPORTING_DIR / "figures" / "p1",

1154)

1155 args = parser.parse_args()

1156 np.random.seed(SEED)

1157 results_dir = args.output_dir

1158 figures_dir = args.figures_dir

1159 results_dir.mkdir(parents=True, exist_ok=True)

1160 figures_dir.mkdir(parents=True, exist_ok=True)

1161

1162 all_catalogs_train: list[pd.DataFrame] = []

1163 all_catalogs_full: list[pd.DataFrame] = []

1164 all_daily: list[pd.DataFrame] = []

1165 all_report_episodes: list[pd.DataFrame] = []

1166 all_multiproject_sensitivity: list[pd.DataFrame] = []

1167 source_diagnostics: dict[str, Any] = {}

1168

1169 for source, filename in SOURCE_FILES.items():

1170 frame = load_detail(args.raw_dir / filename, source)

1171 official_frame = frame[

1172 frame["order_dt"].notna()

1173 & frame["order_dt"].dt.normalize().between(START_DATE, END_DATE)

1174].copy()

1175 all_catalogs_train.append(

1176 make_project_catalog(frame, START_DATE, VALIDATION_START - pd.Timedelta(

1177 days=1))

1177)

```

1178     all_catalogs_full.append(make_project_catalog(frame, START_DATE, END_DATE))
1179     order_episodes = make_order_episodes(frame)
1180     report_episodes = make_report_episodes(frame)
1181     order_episodes = order_episodes[order_episodes["nominal_workload_min"].gt(0)].
copy()
1182     report_episodes = report_episodes[report_episodes["nominal_workload_min"].gt
(0)].copy()
1183     all_report_episodes.append(report_episodes)
1184     all_multiproject_sensitivity.append(multiproject_sensitivity(order_episodes, "
开单事件"))
1185     official_report_episodes = report_episodes[
1186         report_episodes["report_dt"].dt.normalize().between(START_DATE, END_DATE)
1187     ].copy()
1188     all_multiproject_sensitivity.append(multiproject_sensitivity(
official_report_episodes, "报告事件"))
1189
1190     order_daily = (
1191         order_episodes.assign(date=order_episodes["order_dt"].dt.normalize())
1192         .groupby("date", as_index=False)
1193         .agg(episode_count=("patient_key", "size"), project_row_count=("
project_row_count", "sum"))
1194     )
1195     report_daily = (
1196         report_episodes.assign(date=report_episodes["report_dt"].dt.normalize())
1197         .loc[lambda x: x["date"].between(START_DATE, END_DATE)]
1198         .groupby("date", as_index=False)
1199         .agg(episode_count=("patient_key", "size"), project_row_count=("
project_row_count", "sum"))
1200     )
1201     all_daily.append(complete_daily(order_daily, source, "开单日"))
1202     all_daily.append(complete_daily(report_daily, source, "报告日"))
1203     source_diagnostics[source] = {
1204         "detail_rows": int(len(frame)),
1205         "official_detail_rows": int(len(official_frame)),
1206         "official_order_events": int(len(order_episodes)),
1207         "official_report_events": int(len(official_report_episodes)),
1208         "unique_ordered_project_names": int(official_frame["ordered_project"].
nunique(dropna=True)),
1209         "unclassified_general_other_share": float(official_frame["category"].eq("
一般其他").mean()),
1210         "non_service_item_rows": int(official_frame["non_service_item"].sum()),
1211         "non_service_item_share": float(official_frame["non_service_item"].mean())
1212     },
1213     "bedside_row_share": float(official_frame["床旁标记"].mean()),
1214     "fasting_constraint_row_share": float(official_frame["空腹上午标记"].mean()
),
1215     "bladder_constraint_row_share": float(official_frame["充盈膀胱标记"].mean()
),
1216     }
1217     del frame, official_frame, order_episodes, official_report_episodes
1218
1219     catalog = pd.concat(all_catalogs_train, ignore_index=True)
1220     catalog_full = pd.concat(all_catalogs_full, ignore_index=True)
1221     daily = pd.concat(all_daily, ignore_index=True)
1222     report_episodes_all = pd.concat(all_report_episodes, ignore_index=True)
1223     report_episodes_training = report_episodes_all[
report_episodes_all["report_dt"].dt.normalize().lt(VALIDATION_START)
1224     ].copy()
1225     multiproject = pd.concat(all_multiproject_sensitivity, ignore_index=True)
1226

```

```

1227 category_volume = (
1228     catalog_full.groupby(["source", "category"], as_index=False)["record_count"].
sum()
1229 )
1230 category_volume["share_within_source"] = category_volume["record_count"] /
category_volume.groupby("source")["record_count"].transform("sum")
1231 nonservice = (
1232     catalog_full[catalog_full["non_service_item"]]
1233     .groupby(["source", "ordered_project"], as_index=False)
1234     .agg(
1235         record_count=("record_count", "sum"),
1236         bedside_record_count=("record_count", lambda x: x.sum()),
1237     )
1238 )
1239 nonservice["bedside_record_count"] = np.where(
1240     nonservice["ordered_project"].str.contains("床旁", na=False),
1241     nonservice["record_count"],
1242     0,
1243 )
1244
1245 weekly = aggregate_trends(daily, "weekly")
1246 monthly = aggregate_trends(daily, "monthly")
1247 annual = aggregate_trends(daily, "annual")
1248 validation = chronological_validation(daily)
1249 peak_flat = peak_flat_summary(daily)
1250 # Scheduling-time and doctor-efficiency estimates are model inputs, so they
1251 # must be frozen before the chronological holdout begins. Full-period
1252 # report episodes remain available only for descriptive trend outputs.
1253 gap_proxy, gap_diagnostics, episode_gap_frame = report_gap_proxy(
report_episodes_training)
1254 doctors, doctor_diagnostics, reconciliation = doctor_efficiency(
episode_gap_frame,
1255     args.raw_dir / "doctor_day.parquet",
1256 )
1257
1258 durations = duration_table(gap_proxy, category_volume)
1259
1260 rules = pd.DataFrame(
1261     [
1262         {
1263             "priority": index,
1264             "category": rule["category"],
1265             "keyword_regex": rule["pattern"],
1266             "classification_basis": rule["basis"],
1267             "duration_lower_min": rule["lower"],
1268             "duration_nominal_min": rule["nominal"],
1269             "duration_upper_min": rule["upper"],
1270         }
1271         for index, rule in enumerate(CATEGORY_RULES, start=1)
1272     ]
1273 )
1274 flags = pd.DataFrame(
1275     [
1276         {"flag": flag, "keyword_regex": pattern}
1277         for flag, pattern in FLAG_PATTERNS.items()
1278     ]
1279 )
1280
1281 # The authoritative catalog stops before the chronological holdout. The
1282 # full-period catalog is descriptive only and must not be joined into
1283 # holdout dispatch inputs.

```

```

1284 catalog.to_csv(results_dir / "project_category_catalog.csv", index=False, encoding
="utf-8-sig")
1285 catalog.to_csv(results_dir / "project_category_catalog_train.csv", index=False,
encoding="utf-8-sig")
1286 catalog_full.to_csv(
1287     results_dir / "project_category_catalog_full_audit.csv",
1288     index=False,
1289     encoding="utf-8-sig",
1290 )
1291 rules.to_csv(results_dir / "category_rules.csv", index=False, encoding="utf-8-sig"
)
1292 flags.to_csv(results_dir / "resource_flag_rules.csv", index=False, encoding="utf
-8-sig")
1293 category_volume.to_csv(results_dir / "category_volume_summary.csv", index=False,
encoding="utf-8-sig")
1294 daily.to_csv(results_dir / "daily_trends.csv", index=False, encoding="utf-8-sig")
1295 weekly.to_csv(results_dir / "weekly_trends.csv", index=False, encoding="utf-8-sig"
)
1296 monthly.to_csv(results_dir / "monthly_trends.csv", index=False, encoding="utf-8-
sig")
1297 annual.to_csv(results_dir / "annual_trends.csv", index=False, encoding="utf-8-sig"
)
1298 validation.to_csv(results_dir / "chronological_validation.csv", index=False,
encoding="utf-8-sig")
1299 peak_flat.to_csv(results_dir / "peak_flat_summary.csv", index=False, encoding="utf
-8-sig")
1300 gap_proxy.to_csv(results_dir / "report_gap_proxy_by_category.csv", index=False,
encoding="utf-8-sig")
1301 durations.to_csv(results_dir / "service_duration_parameters.csv", index=False,
encoding="utf-8-sig")
1302 doctors.to_csv(results_dir / "doctor_efficiency_shrunk.csv", index=False, encoding
="utf-8-sig")
1303 reconciliation.to_csv(results_dir / "doctor_day_reconciliation.csv", index=False,
encoding="utf-8-sig")
1304 multiproject.to_csv(results_dir / "multiproject_coefficient_sensitivity.csv",
index=False, encoding="utf-8-sig")
1305 nonservice.to_csv(results_dir / "non_service_item_summary.csv", index=False,
encoding="utf-8-sig")
1306
1307 summary = {
1308     "seed": SEED,
1309     "period": {"start": str(START_DATE.date()), "end": str(END_DATE.date())},
1310     "validation_period": {
1311         "start": str(VALIDATION_START.date()),
1312         "end": str(VALIDATION_END.date()),
1313         "split_type": "strict chronological holdout",
1314     },
1315     "catalog_boundary": {
1316         "authoritative_project_catalog_end": str((VALIDATION_START - pd.Timedelta(
days=1)).date()),
1317         "full_period_catalog_output": "project_category_catalog_full_audit.csv",
1318         "holdout_names_used_for_dispatch_join": False,
1319     },
1320     "doctor_time_boundary": {
1321         "authoritative_report_gap_end": str((VALIDATION_START - pd.Timedelta(days
=1)).date()),
1322         "holdout_report_times_used_for_doctor_or_duration_estimation": False,
1323     },
1324     "source_diagnostics": source_diagnostics,
1325     "report_gap_proxy": gap_diagnostics,

```

```

1326     "doctor_efficiency": doctor_diagnostics,
1327     "multiproject_sensitivity": multiproject.to_dict(orient="records"),
1328     "non_service_items": nonservice.to_dict(orient="records"),
1329     "modeling_boundary": {
1330         "has_exam_start_timestamp": False,
1331         "turnaround_used_as_service_time": False,
1332         "duration_parameter_basis": "problem-statement intervals; midpoint for
nominal, upper bound for robust scenario",
1333         "special_case_frequency": 0.05,
1334         "special_case_extra_duration_multiplier": None,
1335     },
1336 }
1337 (results_dir / "analysis_summary.json").write_text(
1338     json.dumps(finite_json(summary), ensure_ascii=False, indent=2),
1339     encoding="utf-8",
1340 )
1341 make_figures(daily, monthly, peak_flat, durations, doctors, figures_dir)
1342 write_report(
1343     results_dir / "P1_DATA_ANALYSIS_REPORT.md",
1344     summary,
1345     peak_flat,
1346     validation,
1347     durations,
1348     doctors,
1349     weekly,
1350     annual,
1351     multiproject,
1352     nonservice,
1353 )
1354
1355 print(json.dumps(finite_json(summary), ensure_ascii=False, indent=2))
1356
1357
1358 if __name__ == "__main__":
1359     main()

```

Listing 4 训练期科室需求分析

```

1  """Build a reproducible department-level demand summary for Problem 1.
2
3  The source workbooks are not modified. Department names are unavailable, so
4  all outputs retain the anonymous department identifiers supplied in the data.
5  """
6
7  from __future__ import annotations
8
9  import argparse
10 import json
11 from pathlib import Path
12
13 import pandas as pd
14
15 from analyze_p1 import END_DATE, SOURCE_FILES, START_DATE, VALIDATION_START,
    load_detail
16
17
18 def choose_event_department(service_rows: pd.DataFrame) -> tuple[pd.DataFrame, int]:
19     """Assign each application proxy to one department without double counting."""
20     event_keys = ["source", "patient_key", "order_dt"]
21     candidate_counts = service_rows.groupby(event_keys, dropna=False)["department_id"

```

```

].nunique()
mixed_department_events = int((candidate_counts > 1).sum())

# Preserve the source ordering and use the first actual service project's
# department. This avoids letting exact duplicate rows vote multiple times.
chosen = (
    service_rows.sort_values(event_keys + ["source_row"])
    .drop_duplicates(event_keys, keep="first")
    [event_keys + ["department_id"]]
)
return chosen, mixed_department_events

def build_summary(
    raw_dir: Path, end_date: pd.Timestamp
) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    all_departments: list[pd.DataFrame] = []
    overview_rows: list[dict] = []
    metadata: dict = {
        "date_range": [START_DATE.date().isoformat(), end_date.date().isoformat()],
        "event_proxy": "source + patient_id + exact order datetime",
        "department_attribution": "department on the first source-ordered service
project row",
        "sources": {},
    }

    for source, filename in SOURCE_FILES.items():
        frame = load_detail(raw_dir / filename, source)
        frame = frame[
            frame["order_dt"].notna()
            & frame["order_dt"].dt.normalize().between(START_DATE, end_date)
        ].copy()
        frame["patient_key"] = frame["patient_id"]
        missing_patient = frame["patient_key"].isna()
        frame.loc[missing_patient, "patient_key"] = (
            "missing_row_" + frame.index[missing_patient].astype(str)
        )
        frame["department_id"] = frame["department_id"].fillna("缺失")

        event_keys = ["source", "patient_key", "order_dt"]
        service_rows = frame[~frame["non_service_item"]].copy()
        chosen, mixed_count = choose_event_department(service_rows)

        unique_projects = service_rows.drop_duplicates(event_keys + ["project_norm"])
        event_workload = (
            unique_projects.groupby(event_keys, dropna=False)
            .agg(
                unique_service_projects=("project_norm", "size"),
                nominal_workload_min=("duration_nominal_min", "sum"),
                upper_workload_min=("duration_upper_min", "sum"),
            )
            .reset_index()
        )
        events = chosen.merge(event_workload, on=event_keys, how="inner", validate="
one_to_one")

        summary = (
            events.groupby(["source", "department_id"], dropna=False)
            .agg(
                service_event_count=("order_dt", "size"),

```

```

79         unique_service_projects=("unique_service_projects", "sum"),
80         nominal_workload_min=("nominal_workload_min", "sum"),
81         upper_workload_min=("upper_workload_min", "sum"),
82     )
83     .reset_index()
84 )
85 total_events = int(summary["service_event_count"].sum())
86 summary["event_share"] = summary["service_event_count"] / total_events
87 summary["mean_nominal_min_per_event"] = (
88     summary["nominal_workload_min"] / summary["service_event_count"]
89 )
90 summary = summary.sort_values(
91     ["service_event_count", "department_id"], ascending=[False, True]
92 ).reset_index(drop=True)
93 summary["rank"] = summary.index + 1
94 summary["cumulative_event_share"] = summary["event_share"].cumsum()
95 summary = summary[
96     [
97         "source",
98         "rank",
99         "department_id",
100        "service_event_count",
101        "event_share",
102        "cumulative_event_share",
103        "unique_service_projects",
104        "nominal_workload_min",
105        "upper_workload_min",
106        "mean_nominal_min_per_event",
107    ]
108 ]
109 all_departments.append(summary)
110
111 top = summary.head(12)
112 known_departments = summary[summary["department_id"] != "缺失"]
113 missing_department_events = int(
114     summary.loc[summary["department_id"] == "缺失", "service_event_count"].sum
115 )
116 overview_rows.append(
117     {
118         "source": source,
119         "service_event_count": total_events,
120         "known_department_count": int(known_departments["department_id"].
nunique()),
121         "missing_department_event_count": missing_department_events,
122         "top_6_event_share": float(summary.head(6)["event_share"].sum()),
123         "top_12_event_share": float(top["event_share"].sum()),
124         "top_6_department_ids": ", ".join(summary.head(6)["department_id"].
astype(str)),
125         "top_6_event_counts": ", ".join(
126             summary.head(6)["service_event_count"].astype(str)
127         ),
128         "mixed_department_event_count": mixed_count,
129     }
130 )
131 metadata["sources"][source] = {
132     "official_rows": int(len(frame)),
133     "service_rows": int(len(service_rows)),
134     "service_events": total_events,
135     "known_department_count": int(known_departments["department_id"].nunique())

```

```

136     ),
137     "missing_department_events": missing_department_events,
138     "mixed_department_events": mixed_count,
139 }
140
141 return pd.concat(all_departments, ignore_index=True), pd.DataFrame(overview_rows),
142 metadata
143
144 def main() -> None:
145     parser = argparse.ArgumentParser()
146     root_default = Path(__file__).resolve().parents[2]
147     parser.add_argument("--raw-dir", type=Path, default=root_default / "
148 supporting_materials/processed_data/raw_parquet")
149     parser.add_argument("--output-dir", type=Path, default=root_default / "
150 supporting_materials/results/p1")
151     args = parser.parse_args()
152     args.output_dir.mkdir(parents=True, exist_ok=True)
153
154     training_end = VALIDATION_START - pd.Timedelta(days=1)
155     summary, overview, metadata = build_summary(args.raw_dir, training_end)
156     full_summary, full_overview, full_metadata = build_summary(args.raw_dir, END_DATE)
157     summary.to_csv(args.output_dir / "department_demand_summary.csv", index=False,
158 encoding="utf-8-sig")
159     overview.to_csv(args.output_dir / "department_demand_overview.csv", index=False,
160 encoding="utf-8-sig")
161     summary.to_csv(
162         args.output_dir / "department_demand_summary_train.csv", index=False, encoding
163 ="utf-8-sig"
164     )
165     overview.to_csv(
166         args.output_dir / "department_demand_overview_train.csv", index=False,
167 encoding="utf-8-sig"
168     )
169     full_summary.to_csv(
170         args.output_dir / "department_demand_summary_full_audit.csv",
171 index=False,
172 encoding="utf-8-sig",
173     )
174     full_overview.to_csv(
175         args.output_dir / "department_demand_overview_full_audit.csv",
176 index=False,
177 encoding="utf-8-sig",
178     )
179     metadata["boundary"] = {
180         "authoritative_training_end": training_end.date().isoformat(),
181         "full_audit_end": END_DATE.date().isoformat(),
182         "holdout_used_for_configuration": False,
183     }
184     metadata["full_period_audit"] = full_metadata
185     with (args.output_dir / "department_demand_metadata.json").open("w", encoding="utf
186 -8") as handle:
187         json.dump(metadata, handle, ensure_ascii=False, indent=2)
188
189     print(overview.to_string(index=False))
190
191 if __name__ == "__main__":
192     main()

```

C 统一输入与能力审计程序

Listing 5 统一患者事件、项目设备证据和医生容量构建

```
1 """Build leakage-audited patient-level inputs for the unified scheduler.
2
3 The output has three deliberate boundaries:
4
5 1. Inpatient jobs are released at their order timestamp and have a 48-hour
6   deadline.
7 2. Outpatient and physical-exam jobs use the observed report *date* as a
8   retrospective plan-day scenario and cannot be released before their order
9   time. Historical report time, room and doctor never enter dispatch fields.
10 3. Machine feasibility is recorded per normalized project with an explicit
11   evidence level. A family match is never labelled as an exact project match.
12 """
13
14 from __future__ import annotations
15
16 import argparse
17 import json
18 import math
19 import re
20 from collections import defaultdict
21 from pathlib import Path
22 from typing import Any, Iterable
23
24 import numpy as np
25 import pandas as pd
26
27 import analyze_p1 as p1
28 import audit_capability as capability
29
30
31 ROOT = Path(__file__).resolve().parents[2]
32 SUPPORTING = ROOT / "supporting_materials"
33 RAW = SUPPORTING / "processed_data" / "raw_parquet"
34 P1_RESULTS = SUPPORTING / "results" / "p1"
35 CAPABILITY_RESULTS = SUPPORTING / "results" / "capability"
36 OUT = SUPPORTING / "processed_data" / "unified_holdout"
37
38 TRAIN_START = pd.Timestamp("2019-03-01")
39 HOLDOUT_START = pd.Timestamp("2024-04-01")
40 HOLDOUT_END_EXCLUSIVE = pd.Timestamp("2025-04-01")
41 # A job can occupy resources for at most 48 hours after release. Therefore
42 # only the two calendar days immediately before the holdout can affect the
43 # state at 2024-04-01; a longer warm-up would add computation without adding a
44 # physically possible carry-over.
45 SIMULATION_START = HOLDOUT_START - pd.Timedelta(hours=48)
46 DAY_START_HOUR = 8
47 DAY_END_HOUR = 17
48 SLOT_MINUTES = 5
49 BLADDER_PREPARATION_MINUTES = 60
50 SEED = 20260824
51
52 SOURCE_CODES = {"住院": "IP", "门诊": "OP", "体检": "PE"}
53
54 EQUIPMENT_RENAME = {
55     "检查超声的机器ID": "machine_id",
56     "彩超诊室名称": "current_room",
```

```

57     "机器型号": "machine_model",
58     "设备购置时间": "purchase_date",
59     "检查项目": "capability_text",
60 }
61
62 DOCTOR_DAY_RENAME = {
63     "检查人ID": "doctor_id",
64     "检查日期": "date",
65     "部位数": "project_count",
66     "最早检查时间": "first_report",
67     "最晚检查时间": "last_report",
68 }
69
70 # These descriptors explicitly denote a whole clinically named class in table 1.
71 # They are broader than exact strings, but narrower and better evidenced than a
72 # blanket family matrix.
73 GENERIC_EVIDENCE = {
74     "心脏": ("心脏彩超",),
75     "产科III/IV级": ("III级产科", "IV级产科", "III级产科超声检查", "IV级产科超声检查"),
76     "产科I/II级": ("I级产科", "II级产科", "I级产科超声检查", "II级产科超声检查"),
77     "腹部/胃肠": ("腹部彩超",),
78     "血管": ("血管彩超",),
79 }
80
81
82 def as_bool(series: pd.Series) -> pd.Series:
83     if pd.api.types.is_bool_dtype(series):
84         return series.fillna(False)
85     return series.astype("string").str.lower().isin(["true", "1", "yes"])
86
87
88 def clean_id(series: pd.Series) -> pd.Series:
89     return pl.clean_id(series).replace("", pd.NA)
90
91
92 def round_up_slot(timestamp: pd.Timestamp) -> pd.Timestamp:
93     if pd.isna(timestamp):
94         return timestamp
95     return timestamp.ceil(f"{SLOT_MINUTES}min")
96
97
98 def release_times(
99     events: pd.DataFrame,
100     bladder_minutes: int,
101     apply_event_bladder: bool = True,
102 ) -> pd.Series:
103     """Return the earliest physically available time for each event.
104
105     A background patient whose order already exists can complete preparation
106     before the 08:00 planning window opens. Therefore readiness is
107     max(planning-window start, order time + preparation), not a blanket delay
108     applied after 08:00.
109     """
110     inpatient = events["source"].eq("住院")
111     release = pd.to_datetime(events["order_dt"]).map(round_up_slot)
112     background_start = pd.to_datetime(events["planned_date"]) + pd.Timedelta(hours=
DAY_START_HOUR)
113     release.loc[~inpatient] = np.maximum(
114         release.loc[~inpatient].values.astype("datetime64[ns]"),
115         background_start.loc[~inpatient].values.astype("datetime64[ns]"),

```

```

116 )
117 prepared = (pd.to_datetime(events["order_dt"]) + pd.Timedelta(minutes=
bladder_minutes)).map(
118     round_up_slot
119 )
120 if apply_event_bladder:
121     bladder = events["bladder"].astype(bool)
122     release.loc[bladder] = np.maximum(
123         release.loc[bladder].values.astype("datetime64[ns]"),
124         prepared.loc[bladder].values.astype("datetime64[ns]"),
125     )
126 return pd.to_datetime(release)
127
128
129 def mode_date(values: pd.Series) -> pd.Timestamp:
130     dates = pd.to_datetime(values, errors="coerce").dropna().dt.normalize()
131     if dates.empty:
132         return pd.NaT
133     counts = dates.value_counts()
134     return pd.Timestamp(counts[counts.eq(counts.max())].index.min())
135
136
137 def unique_join(values: Iterable[Any]) -> str:
138     return "|".join(sorted({str(value) for value in values if pd.notna(value) and str(
value)}}))
139
140
141 def load_source(source: str, filename: str) -> pd.DataFrame:
142     frame = p1.load_detail(RAW / filename, source)
143     frame["item_bedside"] = as_bool(frame["床旁标记"])
144     frame["item_fasting"] = as_bool(frame["空腹上午标记"])
145     frame["item_bladder"] = as_bool(frame["充盈膀胱标记"])
146     frame["patient_id"] = clean_id(frame["patient_id"])
147     missing = frame["patient_id"].isna()
148     frame.loc[missing, "patient_id"] = "MISSING_" + frame.loc[missing, "source_row"].
astype(str)
149     frame["report_date"] = frame["report_dt"].dt.normalize()
150
151     frame = frame.sort_values("source_row", kind="stable")
152
153     duplicate_columns = [
154         "patient_id",
155         "order_dt",
156         "ordered_project",
157         "report_dt",
158         "exam_doctor_id",
159         "machine_id",
160     ]
161     frame["is_exact_duplicate_extra"] = frame.duplicated(duplicate_columns, keep="
first")
162     frame = frame[~frame["is_exact_duplicate_extra"]].copy()
163
164     event_keys = ["source", "patient_id", "order_dt"]
165     event_flags = (
166         frame.groupby(event_keys, dropna=False)
167         .agg(
168             bedside=("床旁标记", "max"),
169             fasting=("空腹上午标记", "max"),
170             bladder=("充盈膀胱标记", "max"),
171             planned_date=("report_date", mode_date),

```

```

172     )
173     .reset_index()
174 )
175 frame = frame.merge(event_flags, on=event_keys, how="left", validate="many_to_one"
176 )
177 frame = frame[~frame["non_service_item"] & frame["project_norm"].notna()].copy()
178 frame = frame.drop_duplicates(event_keys + ["project_norm"], keep="first")
179
180 # Construct the complete application event before applying the simulation
181 # boundary. Filtering detail rows first can truncate a multi-project event
182 # whose reports fall on different dates and can change its modal plan date.
183 if source == "住院":
184     keep = frame["order_dt"].between(
185         SIMULATION_START, HOLDOUT_END_EXCLUSIVE, inclusive="left"
186     )
187 else:
188     keep = frame["planned_date"].between(
189         SIMULATION_START, HOLDOUT_END_EXCLUSIVE, inclusive="left"
190     )
191 frame = frame[keep].copy()
192
193 keys = (
194     frame[event_keys]
195     .drop_duplicates()
196     .sort_values(["order_dt", "patient_id"], kind="stable")
197     .reset_index(drop=True)
198 )
199 code = SOURCE_CODES[source]
200 keys["event_id"] = [f"{code}2024-{index:06d}" for index in range(1, len(keys) + 1)]
201 frame = frame.merge(keys, on=event_keys, how="left", validate="many_to_one")
202 return frame
203
204 def build_patient_tables() -> tuple[pd.DataFrame, pd.DataFrame, dict[str, Any]]:
205     source_frames = [
206         load_source(source, filename) for source, filename in p1.SOURCE_FILES.items()
207     ]
208     items = pd.concat(source_frames, ignore_index=True)
209     for column in ["item_bedside", "item_fasting", "item_bladder", "bedside", "fasting",
210         "bladder"]:
211         items[column] = as_bool(items[column])
212     items["item_index"] = items.groupby("event_id").cumcount() + 1
213
214     events = (
215         items.groupby(["event_id", "source", "patient_id", "order_dt"], as_index=False)
216         .agg(
217             planned_date=("planned_date", "first"),
218             bedside=("bedside", "max"),
219             fasting=("fasting", "max"),
220             bladder=("bladder", "max"),
221             service_project_count=("project_norm", "size"),
222             nominal_minutes=("duration_nominal_min", "sum"),
223             upper_minutes=("duration_upper_min", "sum"),
224             project_names=("project_norm", unique_join),
225             categories=("category", unique_join),
226         )
227     )

```

```

228 inpatient = events["source"].eq("住院")
229 events["mandatory_background"] = ~inpatient
230 events["deadline_dt"] = events["order_dt"] + pd.Timedelta(hours=48)
231 events.loc[~inpatient, "deadline_dt"] = (
232     events.loc[~inpatient, "planned_date"] + pd.Timedelta(hours=DAY_END_HOUR)
233 )
234 events["base_release_dt"] = release_times(
235     events, BLADDER_PREPARATION_MINUTES, apply_event_bladder=False
236 )
237 events["bladder_ready_dt"] = (
238     pd.to_datetime(events["order_dt"]) + pd.Timedelta(minutes=
BLADDER_PREPARATION_MINUTES)
239 ).map(round_up_slot)
240 events["release_dt"] = release_times(events, BLADDER_PREPARATION_MINUTES)
241 events["deadline_dt"] = pd.to_datetime(events["deadline_dt"])
242
243 events["evaluation_cohort"] = np.where(
244     inpatient,
245     events["order_dt"].between(HOLDOUT_START, HOLDOUT_END_EXCLUSIVE, inclusive="
left"),
246     events["planned_date"].between(HOLDOUT_START, HOLDOUT_END_EXCLUSIVE, inclusive
="left"),
247 )
248 rng = np.random.default_rng(SEED)
249 events["special_case"] = False
250 # Keep the required 5% share independent of the technical warm-up window.
251 # The attachment identifies no individual complex cases, so the selected
252 # event IDs are a fixed scenario sample rather than inferred diagnoses.
253 for cohort_value in [False, True]:
254     eligible = events.index[events["evaluation_cohort"].eq(cohort_value)].to_numpy
()
255     special_count = int(round(0.05 * len(eligible)))
256     if special_count:
257         chosen = rng.choice(eligible, size=special_count, replace=False)
258         events.loc[chosen, "special_case"] = True
259 events["duration_scenario"] = np.where(events["special_case"], "upper", "nominal")
260 event_columns = [
261     "event_id",
262     "source",
263     "patient_id",
264     "order_dt",
265     "planned_date",
266     "base_release_dt",
267     "bladder_ready_dt",
268     "release_dt",
269     "deadline_dt",
270     "mandatory_background",
271     "special_case",
272     "duration_scenario",
273     "evaluation_cohort",
274     "bedside",
275     "fasting",
276     "bladder",
277     "service_project_count",
278     "nominal_minutes",
279     "upper_minutes",
280     "project_names",
281     "categories",
282 ]
283 events = events[event_columns].sort_values(["release_dt", "event_id"], kind="

```

```

stable")
284
285 items = items[
286     [
287         "event_id",
288         "item_index",
289         "source_row",
290         "project_norm",
291         "category",
292         "duration_lower_min",
293         "duration_nominal_min",
294         "duration_upper_min",
295         "item_bedside",
296         "item_fasting",
297         "item_bladder",
298         "bedside",
299         "fasting",
300         "bladder",
301     ]
302 ].copy()
303 items = items.rename(
304     columns={
305         "bedside": "event_bedside",
306         "fasting": "event_fasting",
307         "bladder": "event_bladder",
308     }
309 )
310 diagnostics = {
311     "events": {source: int(count) for source, count in events["source"].
value_counts().items()},
312     "items": {source: int(count) for source, count in items.merge(events[["
event_id", "source"]], on="event_id")["source"].value_counts().items()},
313     "special_case_count": int(events["special_case"].sum()),
314     "special_case_share": float(events["special_case"].mean()),
315     "background_scenario_basis": "report date defines the retrospective plan day;
release is no earlier than observed order time; report time, room and doctor are
excluded",
316     "bladder_preparation_minutes": BLADDER_PREPARATION_MINUTES,
317     "bladder_readiness_rule": "max(plan-day start, order time plus preparation)
for background; order time plus preparation for inpatient",
318 }
319 return events, items, diagnostics
320
321
322 def load_equipment() -> pd.DataFrame:
323     frame = pd.read_parquet(RAW / "equipment_capability.parquet").rename(columns=
EQUIPMENT_RENAME)
324     frame["machine_id"] = clean_id(frame["machine_id"])
325     frame["current_room"] = p1.clean_text(frame["current_room"])
326     frame = frame[frame["machine_id"].str.fullmatch(r"\d+", na=False)].copy()
327     if frame["machine_id"].nunique() != 22:
328         raise ValueError("表1现行机器数量不是22台")
329     return frame
330
331
332 def evidence_table(equipment: pd.DataFrame) -> pd.DataFrame:
333     rules = capability.load_rules(P1_RESULTS / "category_rules.csv")
334     rows: list[dict[str, Any]] = []
335     for machine in equipment.iteruples(index=False):
336         for raw_token in capability.split_capabilities(machine.capability_text):

```

```

337         token = capability.normalize_project_text(raw_token)
338         category_name, _, _ = capability.classify_token(token, rules)
339         rows.append(
340             {
341                 "machine_id": str(machine.machine_id),
342                 "current_room": machine.current_room,
343                 "evidence_project": raw_token,
344                 "evidence_norm": token,
345                 "evidence_category": category_name,
346             }
347         )
348     return pd.DataFrame(rows)
349
350
351 def explicit_generic_match(project: str, category_name: str, evidence: str) -> bool:
352     candidates = GENERIC_EVIDENCE.get(category_name, ())
353     return any(candidate in evidence for candidate in candidates)
354
355
356 def semantic_core(value: str) -> str:
357     """Remove presentation words while retaining anatomy/procedure semantics."""
358     text = capability.normalize_project_text(value)
359     text = re.sub(
360         r"彩色多普勒|浅表器官|超声常规检查|常规检查|超声检查|彩超|B超|"
361         r"临床操作的|主任检查|主任|加收|减免",
362         "",
363         text,
364         flags=re.IGNORECASE,
365     )
366     return re.sub(r"[\[\]\(\)\(\),\:\: , \. +\-\_/\s]", "", text)
367
368
369 def reviewed_alias_rule(project: str) -> tuple[str, list[str], str, str]:
370     """Map a formerly category-fallback project to auditable Table-1 evidence."""
371     if project == "B":
372         return "D", [], "invalid_label", "单字符不能确定检查部位或路径"
373     if re.search(r"穿刺|活检|介入引导|临床操作的.*引导", project):
374         return "D", [], "specialized_procedure_not_listed", "表1没有对应穿刺、活检或介入项目能力"
375     if "肺部超声" in project:
376         return "D", [], "pediatric_lung_not_listed", "表1儿科专项只列头颅和髋关节，胸水项目不能替代新生儿肺部超声"
377     if "胡桃夹" in project or "精囊" in project:
378         return "D", [], "anatomy_not_listed", "表1没有足够接近的专项部位证据"
379     if project.startswith("床旁"):
380         if re.search(r"腹部|腹水|泌尿系|前列腺|胸部|胸水", project):
381             return "C", [r"带有床旁的超声项目"], "explicit_bedside_generalization", "7号机原文只对常规床旁部位作有限泛化"
382         return "D", [], "bedside_location_without_specialty", "床旁地点证据不足以覆盖该专项项目"
383     if re.search(r"男性B超", project):
384         return "B", [r"腹部", r"泌尿系", r"前列腺"], "reviewed_compound_intersection", "腹部、泌尿系与前列腺能力取同机交集"
385     if "腹部+泌尿系" in project:
386         patterns = [r"腹部", r"泌尿系"]
387         if "前列腺" in project:
388             patterns.append(r"前列腺")
389         return "B", patterns, "reviewed_compound_intersection", "组合项目按组成部位能力取同机交集"
390     if re.search(r"腹部常规|彩超：肝|腹部\（肝|腹膜后", project):

```

```

391     pattern = r"膜腹后|腹膜后" if "腹膜后" in project else r"腹部"
392     return "B", [pattern], "reviewed_anatomy_alias", "保留腹部或腹膜后解剖语义的名称变
体"
393     if "胃肠道、阑尾" in project:
394         return "B", [r"胃肠道", r"阑尾"], "reviewed_compound_intersection", "表1胃肠道和
阑尾证据取同机交集"
395     if "胃十二指肠充盈声学造影" in project:
396         return "C", [r"胃肠道"], "explicit_anatomy_generalization", "充盈声学造影沿用胃肠
道超声设备能力"
397     if project == "腹水":
398         return "B", [r"腹水"], "reviewed_anatomy_alias", "与表1腹盆腔腹水为同一部位项目"
399     if "腹水加定位" in project:
400         return "B", [r"腹水", r"定位"], "reviewed_compound_intersection", "同机须同时具有
腹水和定位证据"
401     if re.search(r"双肾彩超|泌尿系B超|前列腺彩超|妇科B超\((经腹\)\"", project):
402         pattern = r"前列腺" if "前列腺" in project else (r"经腹妇科" if "妇科" in project
else r"双肾|泌尿系")
403         return "B", [pattern], "reviewed_anatomy_alias", "B超/彩超后缀差异不改变检查部位"
404     if re.search(r"泌尿系彩超.*前列腺", project):
405         return "B", [r"泌尿系", r"前列腺"], "reviewed_compound_intersection", "泌尿系与前
列腺能力取同机交集"
406     if re.search(r"经阴道妇科|阴道彩超", project):
407         return "B", [r"经阴道|腔内彩超"], "reviewed_path_alias", "保留经阴道检查路径的同义
名称"
408     if re.search(r"双乳腺|甲状腺|双眼|关节|软组织", project):
409         pattern = r"乳腺" if "乳腺" in project else r"甲状腺" if "甲状腺" in project
else r"双眼" if "双眼" in project else r"关节" if "关节" in project else r"上肢或下肢
.*软组织"
410         return "B", [pattern], "reviewed_anatomy_alias", "保留浅表器官的具体解剖部位"
411     if "子宫、输卵管超声造影" in project:
412         return "B", [r"输卵管超声造影"], "reviewed_specific_alias", "与表1输卵管超声造影为
明确包含关系"
413     return "D", [], "no_sufficient_table1_evidence", "未找到可保留部位和检查路径的表1证据"
414
415
416 def machines_matching_patterns(evidence: pd.DataFrame, patterns: list[str]) -> pd.
DataFrame:
417     if not patterns:
418         return pd.DataFrame(columns=["machine_id", "current_room", "evidence_text", "
evidence_norm_text"])
419     machine_text = evidence.groupby(["machine_id", "current_room"], as_index=False).
agg(
420         evidence_text=("evidence_project", lambda values: "; ".join(map(str, values)))
421         ,
422         evidence_norm_text=("evidence_norm", lambda values: "; ".join(map(str, values)
)),
423     )
424     mask = pd.Series(True, index=machine_text.index)
425     for pattern in patterns:
426         mask &= machine_text["evidence_norm_text"].str.contains(pattern, regex=True,
na=False)
427     return machine_text[mask].copy()
428
429 def project_machine_matches(
430     items: pd.DataFrame,
431 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
432     equipment = load_equipment()
433     evidence = evidence_table(equipment)
434     projects = items[["project_norm", "category"]].drop_duplicates().sort_values("

```

```

project_norm")
matches: list[dict[str, Any]] = []

for project_row in projects.itertuples(index=False):
    project = capability.normalize_project_text(project_row.project_norm)
    category_name = str(project_row.category)
    category_evidence = evidence[evidence["evidence_category"].eq(category_name)]
    for row in category_evidence.itertuples(index=False):
        token = str(row.evidence_norm)
        level: str | None = None
        rule: str | None = None
        if project == token:
            level, rule = "A", "exact_project"
        elif min(len(project), len(token)) >= 6 and (project in token or token in
project):
            level, rule = "B", "specific_text_containment"
        elif min(len(semantic_core(project)), len(semantic_core(token))) >= 4 and
(
            semantic_core(project) in semantic_core(token)
            or semantic_core(token) in semantic_core(project)
        ):
            level, rule = "B", "semantic_core_match"
        elif explicit_generic_match(project, category_name, token):
            level, rule = "C", "explicit_generic_descriptor"
        if rule:
            matches.append(
                {
                    "project_norm": project_row.project_norm,
                    "category": category_name,
                    "machine_id": str(row.machine_id),
                    "current_room": row.current_room,
                    "match_level": level,
                    "match_rule": rule,
                    "evidence_project": row.evidence_project,
                }
            )
        bedside_level, _, bedside_rule, bedside_reason = reviewed_alias_rule(project)
        if project.startswith("床旁") and bedside_level == "C":
            matches.append(
                {
                    "project_norm": project_row.project_norm,
                    "category": category_name,
                    "machine_id": "7",
                    "current_room": equipment.loc[equipment["machine_id"].eq("7"), "
current_room"].iloc[0],
                    "match_level": "C",
                    "match_rule": bedside_rule,
                    "evidence_project": f"带有床旁的超声项目; {bedside_reason}",
                }
            )

direct = pd.DataFrame(matches)
direct_projects = set(direct["project_norm"]) if not direct.empty else set()
unmatched = projects[~projects["project_norm"].isin(direct_projects)]
review_rows: list[dict[str, Any]] = []
for project_row in unmatched.itertuples(index=False):
    project = str(project_row.project_norm)
    level, patterns, rule, reason = reviewed_alias_rule(project)
    selected = machines_matching_patterns(evidence, patterns)
    if level == "C" and project.startswith("床旁"):

```

```

491         selected = selected[selected["machine_id"].eq("7")]
492     if level != "D" and selected.empty:
493         level, rule = "D", "review_rule_without_machine"
494         reason = f"{reason}, 但没有同一设备满足全部证据"
495     category_candidates = evidence[evidence["evidence_category"].eq(project_row.
category)]
496     if level != "D":
497         for row in selected.itertuples(index=False):
498             matches.append(
499                 {
500                     "project_norm": project,
501                     "category": project_row.category,
502                     "machine_id": str(row.machine_id),
503                     "current_room": row.current_room,
504                     "match_level": level,
505                     "match_rule": rule,
506                     "evidence_project": row.evidence_text,
507                 }
508             )
509     review_rows.append(
510         {
511             "project_norm": project,
512             "category": project_row.category,
513             "current_category_fallback_candidates": "|".join(
514                 sorted(category_candidates["machine_id"].astype(str).unique(), key
= int)
515             ),
516             "review_level": level,
517             "semantic_evidence_sufficient": level != "D",
518             "fallback_reasonable": level != "D",
519             "risk_level": "high" if level == "D" else ("medium" if level == "C"
else "low"),
520             "final_recommended_treatment": "exclude_from_main_hard_capability" if
level == "D" else "use_reviewed_project_mapping",
521             "final_rooms": "|".join(sorted(selected["machine_id"].astype(str).
unique(), key= int)) if level != "D" else "",
522             "table1_evidence_by_machine": " || ".join(
523                 f"{row.machine_id}:{row.evidence_text}" for row in selected.
itertuples(index=False)
524             ) if level != "D" else "",
525             "review_rule": rule,
526             "review_reason": reason,
527         }
528     )
529
530     match_frame = pd.DataFrame(matches).drop_duplicates(
531         ["project_norm", "machine_id", "match_level"], keep="first"
532     )
533     summary_rows: list[dict[str, Any]] = []
534     for project_row in projects.itertuples(index=False):
535         group = match_frame[match_frame["project_norm"].eq(project_row.project_norm)]
536         rooms = sorted(group["machine_id"].astype(str).unique(), key= int) if len(group)
else []
537         levels = sorted(set(group["match_level"]), key= lambda value: "ABC".index(value)
) if len(group) else ["D"]
538         summary_rows.append(
539             {
540                 "project_norm": project_row.project_norm,
541                 "category": project_row.category,
542                 "strict_rooms": "|".join(rooms),

```

```

543         "fallback_rooms": "",
544         "strict_room_count": len(rooms),
545         "fallback_room_count": 0,
546         "uses_fallback_if_nonbedside": False,
547         "final_rooms": "|".join(rooms),
548         "final_room_count": len(rooms),
549         "evidence_level": levels[0],
550         "semantic_status": "confirmed" if rooms else "level_D_excluded",
551     }
552 )
553 return (
554     match_frame.sort_values(["project_norm", "match_level", "machine_id"]),
555     pd.DataFrame(summary_rows),
556     pd.DataFrame(review_rows),
557 )
558
559
560 def build_training_room_scarcity() -> pd.DataFrame:
561     """Estimate room demand pressure from the frozen training catalog only."""
562     catalog = pd.read_csv(
563         P1_RESULTS / "project_category_catalog.csv", encoding="utf-8-sig", low_memory=
564         False
565     )
566     non_service = catalog["non_service_item"].astype(str).str.lower().eq("true")
567     catalog = catalog[~non_service].copy()
568     _, project_summary, _ = project_machine_matches(catalog[["project_norm", "category
569     "]])
570     demand = (
571         catalog.groupby(
572             ["project_norm", "category", "duration_nominal_min"], as_index=False
573         )["record_count"]
574         .sum()
575         .merge(
576             project_summary,
577             on=["project_norm", "category"],
578             how="left",
579             validate="many_to_one",
580         )
581     )
582     room_weight: dict[str, float] = defaultdict(float)
583     for row in demand.itertuples(index=False):
584         rooms = [part for part in str(row.final_rooms).split("|") if part and part !=
585         "nan"]
586         if not rooms:
587             continue
588         contribution = float(row.record_count) * float(row.duration_nominal_min) / len
589         (rooms)
590         for room in rooms:
591             room_weight[room] += contribution
592     return pd.DataFrame(
593         [
594             {
595                 "room_id": room,
596                 "training_weighted_demand_minutes": weight,
597                 "evidence_period_end": str((HOLDOUT_START - pd.Timedelta(days=1)).date
598                 ()),
599             }
600             for room, weight in sorted(room_weight.items(), key=lambda pair: int(pair
601             [0]))
602         ]

```

```

597 )
598
599
600 def slot_clock(slot_index: int) -> tuple[int, int]:
601     if slot_index < 48:
602         minute = 8 * 60 + slot_index * SLOT_MINUTES
603     else:
604         minute = 13 * 60 + (slot_index - 48) * SLOT_MINUTES
605     return minute // 60, minute % 60
606
607
608 def clock_to_slot(timestamp: pd.Timestamp, side: str) -> int:
609     minute = timestamp.hour * 60 + timestamp.minute + timestamp.second / 60
610     if minute < 12 * 60:
611         raw = (minute - 8 * 60) / SLOT_MINUTES
612         base = 0
613     else:
614         raw = (minute - 13 * 60) / SLOT_MINUTES
615         base = 48
616     value = math.floor(raw) if side == "end" else math.ceil(raw)
617     return int(np.clip(base + value, base, base + 48))
618
619
620 def build_doctor_slot_capacity() -> tuple[pd.DataFrame, pd.DataFrame]:
621     frame = pd.read_parquet(RAW / "doctor_day.parquet").rename(columns=
DOCTOR_DAY_RENAME)
622     frame["doctor_id"] = clean_id(frame["doctor_id"])
623     frame["date"] = pd.to_datetime(frame["date"], errors="coerce", format="mixed").dt.
normalize()
624     frame["first_report"] = pd.to_datetime(frame["first_report"], errors="coerce",
format="mixed")
625     frame["last_report"] = pd.to_datetime(frame["last_report"], errors="coerce",
format="mixed")
626     frame = frame[
627         frame["date"].between(TRAIN_START, HOLDOUT_START, inclusive="left")
628         & frame["doctor_id"].notna()
629         & frame["first_report"].notna()
630         & frame["last_report"].notna()
631     ].copy()
632     frame = (
633         frame.groupby(["date", "doctor_id"], as_index=False)
634         .agg(first_report=("first_report", "min"), last_report=("last_report", "max"))
635     )
636
637     calendar = pd.date_range(TRAIN_START, HOLDOUT_START - pd.Timedelta(days=1), freq="
D")
638     daily_arrays = {date: np.zeros(96, dtype=np.int16) for date in calendar}
639     for row in frame.itertuples(index=False):
640         date = pd.Timestamp(row.date)
641         first_observed = pd.Timestamp(row.first_report)
642         last_observed = pd.Timestamp(row.last_report)
643         first_minute = first_observed.hour * 60 + first_observed.minute
644         last_minute = last_observed.hour * 60 + last_observed.minute
645         if 7 * 60 <= first_minute <= 9 * 60:
646             start = date + pd.Timedelta(hours=8)
647         elif 12 * 60 + 30 <= first_minute <= 14 * 60:
648             start = date + pd.Timedelta(hours=13)
649         else:
650             start = first_observed - pd.Timedelta(minutes=10)
651         if 11 * 60 <= last_minute <= 12 * 60 + 30 and first_minute < 12 * 60:

```

```

652         end = date + pd.Timedelta(hours=12)
653     elif 16 * 60 <= last_minute <= 19 * 60:
654         end = date + pd.Timedelta(hours=17)
655     else:
656         end = last_observed
657     for block_start, block_end, offset in [(8, 12, 0), (13, 17, 48)]:
658         block_start_dt = date + pd.Timedelta(hours=block_start)
659         block_end_dt = date + pd.Timedelta(hours=block_end)
660         overlap_start = max(start, block_start_dt)
661         overlap_end = min(end, block_end_dt)
662         if overlap_end <= overlap_start:
663             continue
664         first_slot = clock_to_slot(overlap_start, "start")
665         last_slot = clock_to_slot(overlap_end, "end")
666         first_slot = max(first_slot, offset)
667         last_slot = min(last_slot, offset + 48)
668         if last_slot > first_slot:
669             daily_arrays[date][first_slot:last_slot] += 1
670
671     daily_rows: list[dict[str, Any]] = []
672     for date, counts in daily_arrays.items():
673         for slot_index, count in enumerate(counts):
674             hour, minute = slot_clock(slot_index)
675             daily_rows.append(
676                 {
677                     "date": date,
678                     "weekday": date.dayofweek,
679                     "slot_index": slot_index,
680                     "slot_time": f"{hour:02d}:{minute:02d}",
681                     "active_doctor_proxy": int(count),
682                 }
683             )
684     daily = pd.DataFrame(daily_rows)
685     slot = (
686         daily.groupby(["weekday", "slot_index", "slot_time"], as_index=False)
687         .agg(
688             training_days=("date", "nunique"),
689             median_active_doctors=("active_doctor_proxy", "median"),
690             q25_active_doctors=("active_doctor_proxy", lambda values: values.quantile
(0.25)),
691             q75_active_doctors=("active_doctor_proxy", lambda values: values.quantile
(0.75)),
692             mean_active_doctors=("active_doctor_proxy", "mean"),
693         )
694     )
695     slot["capacity_main"] = np.minimum(
696         np.rint(slot["median_active_doctors"]).astype(int), 22
697     )
698     slot["capacity_low"] = np.minimum(
699         np.floor(slot["q25_active_doctors"]).astype(int), 22
700     )
701     return slot, daily
702
703
704 def attach_room_sets(items: pd.DataFrame, project_summary: pd.DataFrame) -> pd.
DataFrame:
705     merged = items.merge(
706         project_summary,
707         on=["project_norm", "category"],
708         how="left",

```

```

709         validate="many_to_one",
710     )
711     if merged["final_room_count"].isna().any():
712         raise ValueError("存在未进入能力匹配的项目")
713     merged["project_compatible_rooms"] = merged["final_rooms"].fillna("")
714     merged["bedside_compatible_rooms"] = np.where(
715         merged["project_compatible_rooms"].str.split("|").map(lambda values: "7" in
values),
716         "7",
717         "",
718     )
719     merged["compatible_rooms"] = np.where(
720         merged["item_bedside"],
721         merged["bedside_compatible_rooms"],
722         merged["project_compatible_rooms"],
723     )
724     merged["capability_level"] = merged["evidence_level"]
725     return merged
726
727
728 def main() -> None:
729     parser = argparse.ArgumentParser()
730     parser.add_argument("--output-dir", type=Path, default=OUT)
731     args = parser.parse_args()
732     args.output_dir.mkdir(parents=True, exist_ok=True)
733
734     events, items, diagnostics = build_patient_tables()
735     match_rows, project_summary, fallback_review = project_machine_matches(items)
736     items = attach_room_sets(items, project_summary)
737     slot_capacity, daily_capacity = build_doctor_slot_capacity()
738     room_scarcity = build_training_room_scarcity()
739
740     events.to_csv(args.output_dir / "events.csv", index=False, encoding="utf-8-sig")
741     items.to_csv(args.output_dir / "items.csv", index=False, encoding="utf-8-sig")
742     match_rows.to_csv(
743         args.output_dir / "project_machine_evidence.csv", index=False, encoding="utf
-8-sig"
744     )
745     project_summary.to_csv(
746         args.output_dir / "project_machine_summary.csv", index=False, encoding="utf-8-
sig"
747     )
748     source_lookup = events[["event_id", "source"]]
749     review_counts = (
750         items.merge(source_lookup, on="event_id", how="left")
751         .groupby(["project_norm", "source"])
752         .size()
753         .unstack(fill_value=0)
754         .reset_index()
755     )
756     fallback_review = fallback_review.merge(review_counts, on="project_norm", how="
left")
757     for source in ["住院", "门诊", "体检"]:
758         if source not in fallback_review:
759             fallback_review[source] = 0
760     fallback_review["historical_occurrences"] = fallback_review[["住院", "门诊", "体检"
]].sum(axis=1)
761     fallback_review["source_coverage"] = fallback_review.apply(
762         lambda row: "|".join(source for source in ["住院", "门诊", "体检"] if row[
source] > 0), axis=1

```

```

763 )
764 fallback_review.to_csv(
765     CAPABILITY_RESULTS / "CURRENT_UNMATCHED_PROJECT_REVIEW.csv", index=False,
766     encoding="utf-8-sig"
767 )
768 # Retain this compatibility filename for downstream readers; the explicit
769 # legacy 50-row ledger is produced by build_legacy_fallback_audit.py.
770 fallback_review.to_csv(
771     CAPABILITY_RESULTS / "FALLBACK_PROJECT_REVIEW.csv", index=False, encoding="utf
772     -8-sig"
773 )
774 bedside_audit = (
775     items[items["item_bedside"]]
776     .merge(source_lookup, on="event_id", how="left")
777     .groupby(
778         ["project_norm", "category", "source", "evidence_level", "
779         project_compatible_rooms", "bedside_compatible_rooms"],
780         dropna=False,
781         as_index=False,
782     )
783     .size()
784     .rename(columns={"size": "item_count"})
785 )
786 bedside_audit["machine7_has_project_evidence"] = bedside_audit["
787 bedside_compatible_rooms"].eq("7")
788 bedside_audit["semantic_status"] = np.where(
789     bedside_audit["machine7_has_project_evidence"], "confirmed_project_and_bedside
790 ", "bedside_without_specialty_excluded"
791 )
792 bedside_audit.to_csv(
793     CAPABILITY_RESULTS / "bedside_project_capability_audit.csv", index=False,
794     encoding="utf-8-sig"
795 )
796 slot_capacity.to_csv(
797     args.output_dir / "doctor_slot_capacity.csv", index=False, encoding="utf-8-sig
798 "
799 )
800 daily_capacity.to_csv(
801     args.output_dir / "doctor_slot_capacity_training_days.csv",
802     index=False,
803     encoding="utf-8-sig",
804 )
805 room_scarcity.to_csv(
806     args.output_dir / "room_scarcity_training.csv", index=False, encoding="utf-8-
807 sig"
808 )
809
810 event_source = events[["event_id", "source"]]
811 item_levels = items.merge(event_source, on="event_id", how="left")
812 diagnostics["capability"] = {
813     "unique_projects": int(project_summary["project_norm"].nunique()),
814     "projects_by_evidence_level": {
815         level: int(count)
816         for level, count in project_summary["evidence_level"].value_counts().items
817     }
818 },
819     "projects_excluded_level_D": int(project_summary["evidence_level"].eq("D").sum
820 ()),
821     "blanket_category_fallback_used": False,
822     "schedulable_item_share": float(item_levels["compatible_rooms"].fillna("").ne(

```

```

813     "").mean()),
814     "level_D_item_share_by_source": {
815         source: float(group["capability_level"].eq("D").mean())
816         for source, group in item_levels.groupby("source")
817     },
818     "reviewed_former_fallback_project_count": int(len(fallback_review)),
819     "bedside_item_count": int(items["item_bedside"].sum()),
820     "bedside_item_machine7_semantic_coverage": float(
821         items.loc[items["item_bedside"], "bedside_compatible_rooms"].eq("7").mean()
822     ),
823     }
824     diagnostics["doctor_capacity"] = {
825         "method": "weekday-by-5-minute median of training-day active session proxy",
826         "session_reconstruction": "first activity at 07:00-09:00 snaps to 08:00;
827         12:30-14:00 snaps to 13:00; last activity at 11:00-12:30 snaps to 12:00 and
828         16:00-19:00 snaps to 17:00; otherwise retain observed interval with a 10-minute
829         start expansion",
830         "room_cap": 22,
831         "current_doctor_count_from_problem": 33,
832         "main_capacity_min": int(slot_capacity["capacity_main"].min()),
833         "main_capacity_max": int(slot_capacity["capacity_main"].max()),
834     }
835     diagnostics["time_scope"] = {
836         "training_start": str(TRAIN_START.date()),
837         "training_end_exclusive": str(HOLDOUT_START.date()),
838         "holdout_start": str(HOLDOUT_START.date()),
839         "holdout_end_exclusive": str(HOLDOUT_END_EXCLUSIVE.date()),
840         "simulation_start": str(SIMULATION_START.date()),
841         "warmup_days": int((HOLDOUT_START - SIMULATION_START).days),
842     }
843     diagnostics["room_scarcity_boundary"] = {
844         "source": "2019-03-01 to 2024-03-31 frozen training project catalog",
845         "holdout_project_mix_used": False,
846         "measure": "record_count times nominal_minutes divided equally across
847         compatible rooms",
848     }
849     (args.output_dir / "summary.json").write_text(
850         json.dumps(diagnostics, ensure_ascii=False, indent=2), encoding="utf-8"
851     )
852     print(json.dumps(diagnostics, ensure_ascii=False, indent=2), flush=True)
853
854 if __name__ == "__main__":
855     main()

```

Listing 6 旧版 50 个类别回退项目逐项复核

```

1  """Close the audit loop for the 50 projects formerly assigned by category fallback."""
2
3  from __future__ import annotations
4
5  import re
6  from pathlib import Path
7
8  import pandas as pd
9
10 import analyze_p1 as p1
11
12

```

```

13 ROOT = Path(__file__).resolve().parents[2]
14 INPUT = ROOT / "supporting_materials" / "processed_data" / "unified_holdout"
15 CAP = ROOT / "supporting_materials" / "results" / "capability"
16
17 LEGACY_FALLBACK_PROJECTS = [
18     "B", "临床操作的彩色多普勒超声引导(超声科)", "前列腺彩超", "双乳腺及腋窝淋巴结",
19     "双肾彩超", "妇科B超(经腹)", "子宫、输卵管超声造影检查", "左肾胡桃夹现象,主任检查",
20     "床旁前列腺彩超", "床旁彩超介入引导(半小时)", "床旁泌尿系彩超(双肾、输尿管、膀胱)",
21     "床旁泌尿系彩超(双肾、输尿管、膀胱),床旁彩超][床旁腹部常规(肝胆胰脾),床旁彩超][床旁腹水,床旁彩
    超][床旁胸部(床旁胸水),床旁彩超",
22     "床旁经腹诊断性羊膜腔穿刺术(羊水穿刺)", "床旁经腹诊断性羊膜腔穿刺术(羊水穿刺)(减免)",
23     "床旁肺部超声(新生儿)", "床旁腹水", "床旁腹水加定位", "床旁腹部常规(肝胆胰脾)",
24     "彩色多普勒超声常规检查(腹膜后肿物加收)", "彩色多普勒超声常规检查(腹膜后间隙软组织)",
25     "彩超:肝、胆、胰、脾、肾", "泌尿系B超", "泌尿系彩超(双肾输尿管膀胱),男性:前列腺",
26     "浅表器官彩色多普勒超声检查(上肢或下肢软组织)", "浅表器官彩色多普勒超声检查(关节)",
27     "浅表器官彩色多普勒超声检查(双眼及附属器)", "甲状腺B超", "甲状腺及周围淋巴结B超",
28     "男性B超(腹部+泌尿系+前列腺)", "男性B超(腹部、膀胱、前列腺)", "精囊彩超",
29     "经阴道妇科(子宫、附件)", "经阴道妇科(子宫、附件)彩超", "肺部超声(新生儿)",
30     "胃十二指肠充盈声学造影", "胃肠道、阑尾彩超", "腹水", "腹水加定位", "腹膜后组织",
31     "腹部(肝胆胰脾肾)彩超", "腹部(肝胆胰脾肾)彩超,彩超][双乳腺及腋窝淋巴结",
32     "腹部(肝胆胰脾肾)彩超,彩超][甲状腺及颈部淋巴结", "腹部+泌尿系",
33     "腹部+泌尿系,彩超][前列腺B超", "腹部常规(肝、胆、胰、脾、肾)", "腹部常规(肝胆胰脾)",
34     "超声引导下浅表器官穿刺活检术(每处)(超声诊断科)", "超声引导下肾脏穿刺活检术(超声诊断科)",
35     "超声引导下肿瘤穿刺活检术(超声诊断科)", "阴道彩超",
36 ]
37
38
39 def split_legacy(value: str) -> list[str]:
40     parts = re.split(r"\]\s*\[", value)
41     return pl.normalize_project(pd.Series(parts, dtype="string")).dropna().astype(str)
    .tolist()
42
43
44 def main() -> None:
45     CAP.mkdir(parents=True, exist_ok=True)
46     summary = pd.read_csv(INPUT / "project_machine_summary.csv", encoding="utf-8-sig")
47     evidence = pd.read_csv(INPUT / "project_machine_evidence.csv", encoding="utf-8-sig",
48                             dtype={"machine_id": "string"})
49     items = pd.read_csv(INPUT / "items.csv", encoding="utf-8-sig", low_memory=False)
50     current_review_path = CAP / "CURRENT_UNMATCHED_PROJECT_REVIEW.csv"
51     if not current_review_path.exists():
52         current_review_path = CAP / "FALLBACK_PROJECT_REVIEW.csv"
53     current_review = pd.read_csv(current_review_path, encoding="utf-8-sig")
54     summary_by_project = summary.set_index("project_norm")
55     counts = items["project_norm"].value_counts().to_dict()
56     rows: list[dict[str, object]] = []
57
58     for legacy in LEGACY_FALLBACK_PROJECTS:
59         components = split_legacy(legacy)
60         component_rows = summary[summary["project_norm"].isin(components)]
61         if len(components) > 1:
62             disposition = "split_source_bundle_into_items"
63             levels = "|".join(
64                 f"{row.project_norm}:{row.evidence_level}" for row in component_rows.
65                 itertuples(index=False)
66             )
67             rooms = " || ".join(
68                 f"{row.project_norm}:{row.final_rooms}" for row in component_rows.
69                 itertuples(index=False)
70             )

```

```

68         rationale = "旧字段把多个收费项目串成一个名称；已按][边界拆为独立任务并逐项使用A/B/
C/D证据。"
69         elif legacy in summary_by_project.index:
70             row = summary_by_project.loc[legacy]
71             level = str(row["evidence_level"])
72             disposition = "exclude_level_D" if level == "D" else "
use_project_level_A_B_C"
73             levels = level
74             rooms = "" if pd.isna(row["final_rooms"]) else str(row["final_rooms"])
75             matched = evidence[evidence["project_norm"].eq(legacy)]
76             rules = ",".join(sorted(matched["match_rule"].dropna().astype(str)).unique
77             ()))
78             rationale = f"当前项目级证据规则：{rules or 'D级无充分表1证据'}。"
79         else:
80             disposition = "not_in_holdout_after_cleaning"
81             levels = "not_applicable"
82             rooms = ""
83             rationale = "清洗后的留出期任务中不再出现；不再给予任何类别级设备授权。"
84
85         rows.append(
86             {
87                 "legacy_project": legacy,
88                 "legacy_issue": "same_category_blanket_fallback",
89                 "current_disposition": disposition,
90                 "components_after_source_fix": "|".join(components),
91                 "current_evidence_level": levels,
92                 "current_rooms": rooms,
93                 "current_holdout_item_count": int(sum(counts.get(component, 0) for
component in components)),
94                 "category_fallback_retained": False,
95                 "audit_rationale": rationale,
96             }
97         )
98
99         frame = pd.DataFrame(rows)
100         if len(frame) != 50 or frame["legacy_project"].duplicated().any():
101             raise RuntimeError("legacy fallback audit must contain 50 unique projects")
102         frame.to_csv(CAP / "LEGACY_50_FALLBACK_PROJECT_AUDIT.csv", index=False, encoding="
utf-8-sig")
103         view_columns = ["legacy_project", "current_disposition", "current_evidence_level",
"current_holdout_item_count"]
104         markdown_rows = [
105             "|" + "|".join(view_columns) + "|",
106             "|" + "|".join(["---"] * len(view_columns)) + "|",
107         ]
108         for row in frame[view_columns].itertuples(index=False, name=None):
109             markdown_rows.append("|" + "|".join(str(value).replace("|", "/") for value in
row) + "|")
110         report = [
111             "# 旧版50个类别回退项目逐项审计",
112             "",
113             "旧版同类设备兜底已全部撤销。项目只允许A/B/C级项目证据；D级不进入硬约束排程。旧字段中含][
的复合单元按源系统边界拆为独立项目。",
114             "",
115             "\n".join(markdown_rows),
116         ]
117         (CAP / "LEGACY_50_FALLBACK_PROJECT_AUDIT.md").write_text("\n".join(report),
encoding="utf-8")
118         print(frame["current_disposition"].value_counts().to_string())

```

```

119
120 if __name__ == "__main__":
121     main()

```

Listing 7 模型输入语义与训练留出边界验证

```

1  """Independent semantic checks for the frozen scheduling inputs."""
2
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  import pandas as pd
9
10 import analyze_p1 as p1
11
12
13 ROOT = Path(__file__).resolve().parents[2]
14 INPUT = ROOT / "supporting_materials" / "processed_data" / "unified_holdout"
15 CAP = ROOT / "supporting_materials" / "results" / "capability"
16 OUT = ROOT / "supporting_materials" / "results" / "validation"
17 RAW = ROOT / "supporting_materials" / "processed_data" / "raw_parquet"
18
19
20 def room_set(value: object) -> set[str]:
21     if pd.isna(value) or not str(value):
22         return set()
23     result: set[str] = set()
24     for part in str(value).split("|"):
25         if not part:
26             continue
27         number = float(part)
28         result.add(str(int(number)) if number.is_integer() else part)
29     return result
30
31
32 def main() -> None:
33     OUT.mkdir(parents=True, exist_ok=True)
34     events = pd.read_csv(INPUT / "events.csv", encoding="utf-8-sig", low_memory=False)
35     items = pd.read_csv(INPUT / "items.csv", encoding="utf-8-sig", low_memory=False)
36     evidence = pd.read_csv(INPUT / "project_machine_evidence.csv", encoding="utf-8-sig",
37                             dtype={"machine_id": "string"})
38     summary = pd.read_csv(INPUT / "project_machine_summary.csv", encoding="utf-8-sig")
39     review = pd.read_csv(CAP / "LEGACY_50_FALLBACK_PROJECT_AUDIT.csv", encoding="utf-8-sig")
40     bedside = pd.read_csv(CAP / "bedside_project_capability_audit.csv", encoding="utf-8-sig")
41     checks: dict[str, dict[str, object]] = {}
42
43     def record(name: str, violations: int, detail: object) -> None:
44         checks[name] = {"passed": int(violations) == 0, "violation_count": int(violations), "detail": detail}
45
46     valid_evidence = evidence[evidence["match_level"].isin(["A", "B", "C"])]
47     evidence_rooms = valid_evidence.groupby("project_norm")["machine_id"].agg(lambda values: set(values.astype(str))).to_dict()
48     invalid_room_rows = 0
49     for row in items.iter_tuples(index=False):
50         allowed = evidence_rooms.get(row.project_norm, set())

```

```

50     if not room_set(row.project_compatible_rooms).issubset(allowed):
51         invalid_room_rows += 1
52     record("compatible_rooms_trace_to_A_B_C_evidence", invalid_room_rows, "每个项目房间
    均追溯到表1的A/B/C级证据")
53     record("no_blanket_category_fallback", int(evidence["match_rule"].eq("
    same_category_without_project_text_match").sum()), "不得按整个项目族授权")
54     d_projects = set(summary.loc[summary["evidence_level"].eq("D"), "project_norm"])
55     d_with_rooms = items[items["project_norm"].isin(d_projects)]["
    project_compatible_rooms"].fillna("").ne("").sum()
56     record("level_D_excluded_from_hard_capability", int(d_with_rooms), f"D级项目数={len
    (d_projects)}")
57     bedside_bad = 0
58     for row in items[items["item_bedside"].astype(str).str.lower().eq("true")].
    itertuples(index=False):
59         rooms = room_set(row.bedside_compatible_rooms)
60         if rooms and rooms != {"7"}:
61             bedside_bad += 1
62         if rooms and "7" not in evidence_rooms.get(row.project_norm, set()):
63             bedside_bad += 1
64     record("bedside_is_project_capability_intersection", bedside_bad, "床旁可行集只能是
    项目能力与7号机地点能力的交集")
65     record("all_former_fallback_projects_reviewed", abs(len(review) - 50), f"
    review_rows={len(review)}")
66     record("review_has_explicit_treatment", int(review["current_disposition"].isna().
    sum()), "每个原回退项目都有最终处理")
67     record("event_keys_unique", int(events["event_id"].duplicated().sum()), "事件ID唯一
    ")
68     record("item_keys_unique", int(items.duplicated(["event_id", "item_index"]).sum())
    , "事件内项目序号唯一")
69     evaluation = events[events["evaluation_cohort"].astype(str).str.lower().eq("true")
    ].copy()
70     ip_bad = (pd.to_datetime(evaluation.loc[evaluation["source"].eq("住院"), "order_dt
    "], errors="coerce") < pd.Timestamp("2024-04-01")).sum()
71     bg_bad = (pd.to_datetime(evaluation.loc[~evaluation["source"].eq("住院"), "
    planned_date"], errors="coerce") < pd.Timestamp("2024-04-01")).sum()
72     record("train_holdout_boundary", int(ip_bad + bg_bad), "住院按开单日、背景按计划日代理
    进入留出期")
73
74     doctor_frame = pl.load_detail(RAW / "inpatient.parquet", "住院")
75     doctor_frame = doctor_frame[
76         doctor_frame["order_dt"].dt.normalize().lt(pd.Timestamp("2024-04-01"))
77         & doctor_frame["exam_doctor_id"].notna()
78         & ~doctor_frame["non_service_item"]
79     ]
80     doctor_category = (
81         doctor_frame.groupby(["exam_doctor_id", "category"]).size().rename("
    historical_project_rows").reset_index()
82     )
83     doctor_category.to_csv(OUT / "doctor_category_historical_evidence.csv", index=
    False, encoding="utf-8-sig")
84
85     report = {
86         "passed": all(entry["passed"] for entry in checks.values()),
87         "checks": checks,
88         "semantic_counts": {
89             "events": len(events),
90             "items": len(items),
91             "projects": int(summary["project_norm"].nunique()),
92             "evidence_levels": summary["evidence_level"].value_counts().to_dict(),
93             "former_fallback_review_rows": len(review),

```

```

94         "bedside_audit_rows": len(bedside),
95     },
96     "doctor_qualification_boundary": "历史医生-项目族证据可生成，但题面33名未来医生与历史
ID无法一一映射，故主模型只使用总并发容量并假设同一时段内资质可替代，不伪造实名资质矩阵。",
97     "report_completion_boundary": "数据无扫描结束字段；最后任务结束仅作为排程层报告完成代
理。",
98     }
99     (OUT / "input_semantic_verification.json").write_text(
100         json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
101     )
102
103     provenance = """"# TRAIN_HOLDOUT_PROVENANCE_AUDIT
104
105 | 参数 | 使用的数据区间 | 是否允许 | 是否泄漏 | 修复状态 |
106 | --- | --- | --- | --- | --- |
107 | 峰平阈值与代表日 | 2019-03-01至2024-03-31 | 是 | 否 | 已改为训练期分位数，留出期只评价 |
108 | 日历岭回归系数 | 2019-03-01至2024-03-31 | 是 | 否 | 保持 |
109 | 滚动季节朴素预测 | 每个预测时点以前的已观测数据 | 是 | 否 | 明确为rolling-origin |
110 | 项目分类正则与题给时长 | 题面及训练期项目目录 | 是 | 否 | 保持；留出项目只按冻结规则分类 |
111 | 项目-设备能力 | 表1现行设备原文 | 是 | 否 | 不读取留出期完成机房 |
112 | 医生时隙容量 | 2019-03-01至2024-03-31表6 | 是 | 否 | 保持 |
113 | 房间稀缺权重 | 2019-03-01至2024-03-31项目目录 | 是 | 否 | 保持 |
114 | 特殊病例比例 | 题面5%，身份由固定种子抽样 | 是 | 否 | 多seed检验 |
115 | 调度策略与参数 | 题面、训练期及预先声明情景 | 是 | 否 | 留出期不反向调参 |
116 | 最终指标 | 2024-04-01至2025-03-31 | 仅评价 | 否 | 保持 |
117 """"
118     (OUT / "TRAIN_HOLDOUT_PROVENANCE_AUDIT.md").write_text(provenance, encoding="utf-8")
119
120     print(json.dumps(report, ensure_ascii=False, indent=2))
121     if not report["passed"]:
122         raise SystemExit(1)
123
124 if __name__ == "__main__":
125     main()

```

D 排程内核与问题二、三程序

Listing 8 患者级连续日历排程内核

```

1  """Continuous patient-level scheduling of outpatient, physical-exam and inpatient jobs
2
3  The annual replay has one calendar state. Outpatient and physical-exam
4  appointment-day demand is placed first because the problem requires it to be
5  met; inpatient policies are compared on the identical residual resources.
6  The JOINT_SCARCITY policy additionally arranges background demand by project
7  scarcity. It is retained as a Pareto alternative, while the authoritative
8  policy is selected by the competition's inpatient 48-hour KPI.
9  """
10
11 from __future__ import annotations
12
13 import argparse
14 import json
15 import math
16 from dataclasses import dataclass
17 from pathlib import Path

```

```

18 from typing import Any, Iterable
19
20 import numpy as np
21 import pandas as pd
22
23 import build_unified_inputs as source_config
24
25
26 ROOT = Path(__file__).resolve().parents[2]
27 SUPPORTING = ROOT / "supporting_materials"
28 INPUT = SUPPORTING / "processed_data" / "unified_holdout"
29 OUT = SUPPORTING / "results" / "unified_schedule"
30
31 SLOT_MINUTES = 5
32 TRANSFER_MINUTES = 10
33 # The holdout closes at 2025-04-01 00:00. Two calendar days are sufficient
34 # to represent the latest admissible 48-hour inpatient deadline (April 2);
35 # no arbitrary extra scheduling horizon is added.
36 FOLLOWUP_DAYS = 2
37
38 POLICIES = {
39     "FCFS_SHARED": {"background": "fcfs", "inpatient": "fcfs"},
40     "SLACK_GUARD_SHARED": {"background": "fcfs", "inpatient": "slack"},
41     "JOINT_SCARCITY": {"background": "scarcity", "inpatient": "slack"},
42 }
43 AUTHORITATIVE_POLICY = "FCFS_SHARED"
44
45
46 def as_bool(series: pd.Series) -> pd.Series:
47     if pd.api.types.is_bool_dtype(series):
48         return series.fillna(False)
49     return series.astype("string").str.lower().isin(["true", "1", "yes"])
50
51
52 def room_tuple(value: Any) -> tuple[str, ...]:
53     if pd.isna(value) or not str(value).strip():
54         return ()
55     normalized: set[str] = set()
56     for part in str(value).split("|"):
57         if not part:
58             continue
59         number = float(part)
60         normalized.add(str(int(number)) if number.is_integer() else part)
61     return tuple(sorted(normalized, key=int))
62
63
64 @dataclass(frozen=True)
65 class Task:
66     event_id: str
67     item_index: int
68     project_norm: str
69     category: str
70     duration_slots: int
71     compatible_rooms: tuple[str, ...]
72     capability_level: str
73     fasting: bool
74     bladder: bool
75     bedside: bool
76     bladder_ready_dt: pd.Timestamp
77

```

```

78
79 @dataclass
80 class PlanStep:
81     task: Task
82     start_index: int
83     room_id: str
84     sequence_position: int
85     transfer_slots_before: int
86
87
88 class WorkCalendar:
89     def __init__(
90         self,
91         start_date: pd.Timestamp,
92         end_date_exclusive: pd.Timestamp,
93         room_ids: list[str],
94         slot_capacity: pd.DataFrame,
95         capacity_column: str,
96         room_scarcity: dict[str, float],
97         room_mode: str,
98         transfer_minutes: int,
99     ) -> None:
100         dates = pd.date_range(start_date, end_date_exclusive - pd.Timedelta(days=1),
101                                freq="D")
102         timestamps: list[pd.Timestamp] = []
103         slot_in_day: list[int] = []
104         block: list[int] = []
105         for date in dates:
106             for offset in range(48):
107                 timestamps.append(date + pd.Timedelta(hours=8, minutes=offset *
108                 SLOT_MINUTES))
109                 slot_in_day.append(offset)
110                 block.append(0)
111             for offset in range(48):
112                 timestamps.append(date + pd.Timedelta(hours=13, minutes=offset *
113                 SLOT_MINUTES))
114                 slot_in_day.append(48 + offset)
115                 block.append(1)
116         self.times = pd.DatetimeIndex(timestamps)
117         self.slot_in_day = np.asarray(slot_in_day, dtype=np.int16)
118         self.block = np.asarray(block, dtype=np.int8)
119         self.day_values = self.times.normalize().to_numpy()
120         self.room_ids = room_ids
121         self.room_index = {room: index for index, room in enumerate(room_ids)}
122         self.room_busy = np.zeros((len(room_ids), len(self.times)), dtype=bool)
123         self.doctor_load = np.zeros(len(self.times), dtype=np.int16)
124         self.capacity = self._capacity_array(slot_capacity, capacity_column)
125         self.room_work_slots = {room: 0 for room in room_ids}
126         self.room_scarcity = room_scarcity
127         self.room_mode = room_mode
128         self.transfer_slots = transfer_minutes // SLOT_MINUTES
129
130     def clone(self) -> "WorkCalendar":
131         """Copy mutable allocation state while sharing immutable calendar axes."""
132         other = object.__new__(WorkCalendar)
133         other.times = self.times
134         other.slot_in_day = self.slot_in_day
135         other.block = self.block
136         other.day_values = self.day_values
137         other.room_ids = self.room_ids

```

```

135     other.room_index = self.room_index
136     other.room_busy = self.room_busy.copy()
137     other.doctor_load = self.doctor_load.copy()
138     other.capacity = self.capacity
139     other.room_work_slots = self.room_work_slots.copy()
140     other.room_scarcity = self.room_scarcity
141     other.room_mode = self.room_mode
142     other.transfer_slots = self.transfer_slots
143     return other
144
145     def _capacity_array(self, frame: pd.DataFrame, column: str) -> np.ndarray:
146         lookup = {
147             (int(row.weekday), int(row.slot_index)): int(getattr(row, column))
148             for row in frame.itertuples(index=False)
149         }
150         return np.asarray(
151             [lookup[(int(timestamp.dayofweek), int(slot))] for timestamp, slot in zip(
152 self.times, self.slot_in_day)],
153             dtype=np.int16,
154         )
155
156     def search_start(self, timestamp: pd.Timestamp) -> int:
157         return int(self.times.searchsorted(timestamp, side="left"))
158
159     def end_timestamp(self, start_index: int, duration_slots: int) -> pd.Timestamp:
160         return self.times[start_index] + pd.Timedelta(minutes=duration_slots *
161 SLOT_MINUTES)
162
163     def fits_block(self, start_index: int, duration_slots: int) -> bool:
164         if start_index < 0 or start_index + duration_slots > len(self.times):
165             return False
166         end_index = start_index + duration_slots - 1
167         return (
168             self.day_values[start_index] == self.day_values[end_index]
169             and self.block[start_index] == self.block[end_index]
170         )
171
172     def available(self, room_id: str, start_index: int, duration_slots: int) -> bool:
173         if not self.fits_block(start_index, duration_slots):
174             return False
175         room_index = self.room_index[room_id]
176         interval = slice(start_index, start_index + duration_slots)
177         return (
178             not self.room_busy[room_index, interval].any()
179             and np.all(self.doctor_load[interval] < self.capacity[interval])
180         )
181
182     def reserve(self, step: PlanStep) -> None:
183         room_index = self.room_index[step.room_id]
184         interval = slice(step.start_index, step.start_index + step.task.duration_slots)
185
186         self.room_busy[room_index, interval] = True
187         self.doctor_load[interval] += 1
188         self.room_work_slots[step.room_id] += step.task.duration_slots
189
190     def release(self, step: PlanStep) -> None:
191         room_index = self.room_index[step.room_id]
192         interval = slice(step.start_index, step.start_index + step.task.duration_slots)
193
194         self.room_busy[room_index, interval] = False

```

```

191     self.doctor_load[interval] -= 1
192     self.room_work_slots[step.room_id] -= step.task.duration_slots
193
194     def room_choice(self, rooms: list[str], last_room: str | None) -> str:
195         if last_room in rooms:
196             return str(last_room)
197         if self.room_mode == "fcfs":
198             return min(rooms, key=int)
199         return min(
200             rooms,
201             key=lambda room: (
202                 self.room_work_slots[room],
203                 self.room_scarcity.get(room, 0.0),
204                 int(room),
205             ),
206         )
207
208     def find_task(
209         self,
210         task: Task,
211         release: pd.Timestamp,
212         latest_end: pd.Timestamp,
213         last_room: str | None,
214         room_override: str | None = None,
215     ) -> tuple[int, str, int] | None:
216         rooms = [room_override] if room_override is not None else list(task.
compatible_rooms)
217         rooms = [room for room in rooms if room in self.room_index]
218         if not rooms:
219             return None
220         start_index = self.search_start(release)
221         stop_index = min(
222             int(self.times.searchsorted(latest_end, side="left")),
223             len(self.times) - task.duration_slots + 1,
224         )
225         if stop_index <= start_index:
226             return None
227         indices = np.arange(start_index, stop_index, dtype=np.int64)
228         end_indices = indices + task.duration_slots - 1
229         starts = self.times[indices]
230         ends = starts + pd.Timedelta(minutes=task.duration_slots * SLOT_MINUTES)
231         valid = (
232             (self.day_values[indices] == self.day_values[end_indices])
233             & (self.block[indices] == self.block[end_indices])
234             & (ends <= latest_end)
235         )
236         if task.fasting:
237             valid &= np.asarray(
238                 (starts.hour < 10)
239                 & (ends <= starts.normalize() + pd.Timedelta(hours=10)),
240                 dtype=bool,
241             )
242
243         span_end = stop_index + task.duration_slots - 1
244         doctor_blocked = (
245             self.doctor_load[start_index:span_end] >= self.capacity[start_index:
span_end]
246         ).astype(np.int16)
247         doctor_prefix = np.concatenate([[0], np.cumsum(doctor_blocked)])
248         valid &= (

```

```

249         doctor_prefix[task.duration_slots:] - doctor_prefix[:-task.duration_slots]
250     == 0
251     )
252     earliest_by_room: dict[str, int] = {}
253     for room in rooms:
254         room_index = self.room_index[room]
255         room_blocked = self.room_busy[room_index, start_index:span_end].astype(np.
int16)
256         room_prefix = np.concatenate([[0], np.cumsum(room_blocked)])
257         room_free = room_prefix[task.duration_slots:] - room_prefix[:-task.
duration_slots] == 0
258         transfer_slots = 0 if last_room is None or room == last_room else self.
transfer_slots
259         transfer_release = release + pd.Timedelta(minutes=transfer_slots *
SLOT_MINUTES)
260         feasible = valid & room_free & np.asarray(starts >= transfer_release,
dtype=bool)
261         positions = np.flatnonzero(feasible)
262         if positions.size:
263             earliest_by_room[room] = int(indices[positions[0]])
264         if earliest_by_room:
265             earliest = min(earliest_by_room.values())
266             feasible_rooms = [room for room, index in earliest_by_room.items() if
index == earliest]
267             room = self.room_choice(feasible_rooms, last_room)
268             transfer_slots = 0 if last_room is None or room == last_room else self.
transfer_slots
269             return earliest, room, transfer_slots
270         return None
271
272
273 def load_inputs(
274     special_mode: str,
275     capability_mode: str,
276     bladder_minutes: int,
277     preparation_mode: str = "item",
278     special_seed: int | None = None,
279 ) -> tuple[pd.DataFrame, dict[str, list[Task]], pd.DataFrame, list[str], dict[str, str
]]:
280     events = pd.read_csv(
281         INPUT / "events.csv",
282         encoding="utf-8-sig",
283         parse_dates=["order_dt", "planned_date", "base_release_dt", "bladder_ready_dt"
, "release_dt", "deadline_dt"],
284     )
285     for column in [
286         "mandatory_background",
287         "special_case",
288         "evaluation_cohort",
289         "bedside",
290         "fasting",
291         "bladder",
292     ]:
293         events[column] = as_bool(events[column])
294     if special_seed is not None:
295         events["special_case"] = False
296         rng = np.random.default_rng(special_seed)
297         for cohort_value in [False, True]:
298             eligible = events.index[events["evaluation_cohort"].eq(cohort_value)].

```

```

to_numpy()
299         count = int(round(0.05 * len(eligible)))
300         if count:
301             events.loc[rng.choice(eligible, size=count, replace=False), "
special_case"] = True
302         events["bladder_ready_dt"] = (
303             events["order_dt"] + pd.Timedelta(minutes=bladder_minutes)
304         ).map(source_config.round_up_slot)
305         if preparation_mode == "event":
306             events["release_dt"] = source_config.release_times(events, bladder_minutes,
True)
307         elif preparation_mode == "item":
308             events["release_dt"] = source_config.release_times(events, bladder_minutes,
False)
309         else:
310             raise ValueError(f"unknown preparation mode: {preparation_mode}")
311
312         items = pd.read_csv(INPUT / "items.csv", encoding="utf-8-sig")
313         for column in [
314             "item_bedside", "item_fasting", "item_bladder",
315             "event_bedside", "event_fasting", "event_bladder",
316             "uses_fallback_if_nonbedside",
317         ]:
318             items[column] = as_bool(items[column])
319         event_special = events.set_index("event_id")["special_case"].to_dict()
320         event_bladder_ready = events.set_index("event_id")["bladder_ready_dt"].to_dict()
321         tasks: dict[str, list[Task]] = {}
322         for event_id, group in items.groupby("event_id", sort=False):
323             event_tasks: list[Task] = []
324             use_upper = special_mode == "five_percent_upper" and bool(event_special[
event_id])
325             event_ready = pd.Timestamp(event_bladder_ready[event_id])
326             for row in group.sort_values("item_index", kind="stable").itertuples(index=
False):
327                 bedside = bool(row.event_bedside) if preparation_mode == "event" else bool
(row.item_bedside)
328                 fasting = bool(row.event_fasting) if preparation_mode == "event" else bool
(row.item_fasting)
329                 bladder = bool(row.event_bladder) if preparation_mode == "event" else bool
(row.item_bladder)
330                 room_text = row.bedside_compatible_rooms if bedside else row.
project_compatible_rooms
331                 if capability_mode == "strict" and str(row.capability_level) == "C":
332                     rooms: tuple[str, ...] = ()
333                 else:
334                     rooms = room_tuple(room_text)
335                 duration = row.duration_upper_min if use_upper else row.
duration_nominal_min
336                 event_tasks.append(
337                     Task(
338                         event_id=str(event_id),
339                         item_index=int(row.item_index),
340                         project_norm=str(row.project_norm),
341                         category=str(row.category),
342                         duration_slots=max(1, int(math.ceil(float(duration) / SLOT_MINUTES
))),
343                         compatible_rooms=rooms,
344                         capability_level=str(row.capability_level),
345                         fasting=fasting,
346                         bladder=bladder,

```

```

347         bedside=bedside,
348         bladder_ready_dt=event_ready,
349     )
350 )
351 tasks[str(event_id)] = event_tasks
352
353 equipment = pd.read_csv(
354     SUPPORTING / "results" / "capability" / "equipment_source_rows.csv",
355     encoding="utf-8-sig",
356 )
357 equipment["machine_id"] = equipment["machine_id"].astype(str).str.replace(r"\.0$",
358     "", regex=True)
359 room_names = dict(zip(equipment["machine_id"], equipment["current_room"]))
360 room_ids = sorted(room_names, key=int)
361 slot_capacity = pd.read_csv(INPUT / "doctor_slot_capacity.csv", encoding="utf-8-
362 sig")
363 return events, tasks, slot_capacity, room_ids, room_names
364
365 def task_room_scarcity() -> dict[str, float]:
366     """Read room pressure estimated only from the frozen training catalog."""
367     frame = pd.read_csv(INPUT / "room_scarcity_training.csv", encoding="utf-8-sig")
368     return {
369         str(row.room_id): float(row.training_weighted_demand_minutes)
370         for row in frame.itertuples(index=False)
371     }
372
373 def task_order_candidates(event_tasks: list[Task]) -> list[list[Task]]:
374     keys = [
375         lambda task: (not task.fasting, len(task.compatible_rooms), -task.
376 duration_slots, task.item_index),
377         lambda task: (not task.fasting, -task.duration_slots, len(task.
378 compatible_rooms), task.item_index),
379         lambda task: (len(task.compatible_rooms), not task.fasting, task.item_index),
380         lambda task: (task.item_index,),
381     ]
382     candidates: list[list[Task]] = []
383     seen: set[tuple[int, ...]] = set()
384     for key in keys:
385         ordered = sorted(event_tasks, key=key)
386         signature = tuple(task.item_index for task in ordered)
387         if signature not in seen:
388             candidates.append(ordered)
389             seen.add(signature)
390     return candidates
391
392 def tentative_plan(
393     calendar: WorkCalendar,
394     ordered_tasks: list[Task],
395     release: pd.Timestamp,
396     latest_end: pd.Timestamp,
397     room_override: str | None,
398 ) -> list[PlanStep] | None:
399     steps: list[PlanStep] = []
400     cursor = release
401     last_room: str | None = None
402     for position, task in enumerate(ordered_tasks, start=1):
403         task_release = max(cursor, task.bladder_ready_dt) if task.bladder else cursor

```

```

403         found = calendar.find_task(
404             task,
405             task_release,
406             latest_end,
407             last_room,
408             room_override=room_override,
409         )
410         if found is None:
411             for step in reversed(steps):
412                 calendar.release(step)
413             return None
414         start_index, room_id, transfer_slots = found
415         step = PlanStep(task, start_index, room_id, position, transfer_slots)
416         calendar.reserve(step)
417         steps.append(step)
418         cursor = calendar.end_timestamp(start_index, task.duration_slots)
419         last_room = room_id
420     return steps
421
422
423 def plan_event(
424     calendar: WorkCalendar,
425     event_tasks: list[Task],
426     release: pd.Timestamp,
427     latest_end: pd.Timestamp,
428 ) -> list[PlanStep] | None:
429     if any(not task.compatible_rooms for task in event_tasks):
430         return None
431     orders = task_order_candidates(event_tasks)
432     common = set(event_tasks[0].compatible_rooms)
433     for task in event_tasks[1:]:
434         common &= set(task.compatible_rooms)
435     if common:
436         candidate_rooms = sorted(
437             common,
438             key=lambda room: (
439                 calendar.room_work_slots[room],
440                 calendar.room_scarcity.get(room, 0.0),
441                 int(room),
442             ),
443            )[:3]
444         for room in candidate_rooms:
445             steps = tentative_plan(calendar, orders[0], release, latest_end, room)
446             if steps is not None:
447                 return steps
448     for ordered in orders:
449         steps = tentative_plan(calendar, ordered, release, latest_end, None)
450         if steps is not None:
451             return steps
452     return None
453
454
455 def event_priority(
456     events: pd.DataFrame,
457     tasks: dict[str, list[Task]],
458     group: str,
459     strategy: str,
460 ) -> list[str]:
461     subset = events[events["mandatory_background"] if group == "background" else ~
events["mandatory_background"]].copy()

```

```

462 subset["work_slots"] = subset["event_id"].map(
463     lambda event_id: sum(task.duration_slots for task in tasks[event_id])
464 )
465 subset["minimum_room_count"] = subset["event_id"].map(
466     lambda event_id: min((len(task.compatible_rooms) for task in tasks[event_id]),
467                          default=0)
468 )
469 subset["has_fasting_task"] = subset["event_id"].map(
470     lambda event_id: any(task.fasting for task in tasks[event_id])
471 )
472 if group == "background" and strategy == "scarcity":
473     subset = subset.sort_values(
474         ["deadline_dt", "has_fasting_task", "minimum_room_count", "work_slots", "
475         release_dt", "event_id"],
476         ascending=[True, False, True, False, True, True],
477         kind="stable",
478     )
479 elif group == "inpatient" and strategy == "slack":
480     subset["slack_key"] = subset["deadline_dt"] - pd.to_timedelta(
481         subset["work_slots"] * SLOT_MINUTES, unit="min"
482     )
483     subset = subset.sort_values(
484         ["slack_key", "minimum_room_count", "deadline_dt", "event_id"],
485         kind="stable",
486     )
487 else:
488     subset = subset.sort_values(["release_dt", "event_id"], kind="stable")
489 return subset["event_id"].tolist()
490
491 def schedule_group(
492     calendar: WorkCalendar,
493     event_ids: list[str],
494     event_lookup: pd.DataFrame,
495     tasks: dict[str, list[Task]],
496 ) -> tuple[dict[str, list[PlanStep]], dict[str, str]]:
497     plans: dict[str, list[PlanStep]] = {}
498     failure: dict[str, str] = {}
499     for count, event_id in enumerate(event_ids, start=1):
500         event = event_lookup.loc[event_id]
501         steps = plan_event(calendar, tasks[event_id], event.release_dt, event.
502         deadline_dt)
503         if steps is None:
504             if any(not task.compatible_rooms for task in tasks[event_id]):
505                 failure[event_id] = "no_compatible_room_under_scenario"
506             else:
507                 failure[event_id] = "no_complete_plan_by_deadline"
508         else:
509             plans[event_id] = steps
510             if count % 25000 == 0:
511                 print(f" on-time pass {count:}/{len(event_ids):}", flush=True)
512     print(
513         f" deadline pass complete: scheduled={len(plans):}, unallocated={len(failure
514         ):}, ",
515         flush=True,
516     )
517     return plans, failure
518
519 def plan_to_schedule(

```

```

518     calendar: WorkCalendar,
519     plans: dict[str, list[PlanStep]],
520     events: pd.DataFrame,
521     policy: str,
522     room_names: dict[str, str],
523 ) -> pd.DataFrame:
524     event_meta = events.set_index("event_id")
525     rows: list[dict[str, Any]] = []
526     for event_id, steps in plans.items():
527         event = event_meta.loc[event_id]
528         ordered = sorted(steps, key=lambda step: step.sequence_position)
529         for step in ordered:
530             start = calendar.times[step.start_index]
531             end = calendar.end_timestamp(step.start_index, step.task.duration_slots)
532             rows.append(
533                 {
534                     "policy": policy,
535                     "event_id": event_id,
536                     "source": event.source,
537                     "patient_id": event.patient_id,
538                     "item_index": step.task.item_index,
539                     "sequence_position": step.sequence_position,
540                     "project_norm": step.task.project_norm,
541                     "category": step.task.category,
542                     "start_dt": start,
543                     "end_dt": end,
544                     "room_id": step.room_id,
545                     "room_name": room_names.get(step.room_id, step.room_id),
546                     "duration_minutes": step.task.duration_slots * SLOT_MINUTES,
547                     "transfer_minutes_before": step.transfer_slots_before *
548                     SLOT_MINUTES,
549                     "room_switch_from_previous": step.transfer_slots_before > 0,
550                     "capability_level": step.task.capability_level,
551                     "fasting": step.task.fasting,
552                     "bladder": step.task.bladder,
553                     "bedside": step.task.bedside,
554                     "release_dt": event.release_dt,
555                     "deadline_dt": event.deadline_dt,
556                     "evaluation_cohort": event.evaluation_cohort,
557                 }
558             )
559     return pd.DataFrame(rows)
560
561 def outcomes_from_schedule(
562     events: pd.DataFrame,
563     schedule: pd.DataFrame,
564     failure: dict[str, str],
565     policy: str,
566 ) -> pd.DataFrame:
567     if schedule.empty:
568         aggregates = pd.DataFrame(columns=["event_id"])
569     else:
570         aggregates = (
571             schedule.groupby("event_id", as_index=False)
572             .agg(
573                 scheduled_items=("item_index", "size"),
574                 first_start_dt=("start_dt", "min"),
575                 completion_dt=("end_dt", "max"),
576                 room_count=("room_id", "nunique"),

```

```

577         room_switch_count=("room_switch_from_previous", "sum"),
578         transfer_minutes=("transfer_minutes_before", "sum"),
579         fallback_task_count=("capability_level", lambda values: values.eq("
category_fallback").sum()),
580     )
581 )
582 result = events.merge(aggregates, on="event_id", how="left", validate="one_to_one"
)
583 result["scheduled_items"] = result["scheduled_items"].fillna(0).astype(int)
584 result["complete_event"] = result["scheduled_items"].eq(result["
service_project_count"])
585 result["deadline_met"] = result["complete_event"] & result["completion_dt"].le(
result["deadline_dt"])
586 result["waiting_hours"] = (
587     result["first_start_dt"] - result["release_dt"]
588 ).dt.total_seconds() / 3600
589 result["failure_reason"] = result["event_id"].map(failure)
590 result.loc[
591     ~result["deadline_met"] & result["failure_reason"].isna(), "failure_reason"
592 ] = "completed_after_deadline"
593 result.loc[result["deadline_met"], "failure_reason"] = ""
594 result["policy"] = policy
595 return result
596
597
598 def metrics_from_outcomes(
599     outcomes: pd.DataFrame,
600     schedule: pd.DataFrame,
601     calendar: WorkCalendar,
602     policy: str,
603     scenario: str,
604 ) -> dict[str, Any]:
605     cohort = outcomes[outcomes["evaluation_cohort"]]
606     inpatient = cohort[cohort["source"].eq("住院")]
607     background = cohort[~cohort["source"].eq("住院")]
608     scheduled_wait = inpatient["waiting_hours"].dropna()
609     room_minutes = float(schedule["duration_minutes"].sum()) if not schedule.empty
else 0.0
610     staffed_minutes = float(calendar.capacity.sum() * SLOT_MINUTES)
611     return {
612         "scenario": scenario,
613         "policy": policy,
614         "inpatient_events": int(len(inpatient)),
615         "inpatient_complete_48h": int(inpatient["deadline_met"].sum()),
616         "inpatient_48h_rate": float(inpatient["deadline_met"].mean()),
617         "inpatient_scheduled_complete": int(inpatient["complete_event"].sum()),
618         "inpatient_unscheduled": int((~inpatient["complete_event"]).sum()),
619         "inpatient_wait_p50_hours_conditional": float(scheduled_wait.quantile(0.50)),
620         "inpatient_wait_p90_hours_conditional": float(scheduled_wait.quantile(0.90)),
621         "inpatient_wait_p95_hours_conditional": float(scheduled_wait.quantile(0.95)),
622         "inpatient_room_switch_rate_multi": float(
623             inpatient.loc[inpatient["service_project_count"].gt(1), "room_switch_count"
624             ].fillna(0).gt(0).mean()
625         ),
626         "background_events": int(len(background)),
627         "background_on_planned_day": int(background["deadline_met"].sum()),
628         "background_on_time_rate": float(background["deadline_met"].mean()),
629         "background_scheduled_complete": int(background["complete_event"].sum()),
630         "fallback_scheduled_task_share": float(schedule["capability_level"].eq("
category_fallback").mean()),

```

```

630         "scheduled_minutes": room_minutes,
631         "staffed_capacity_utilization": room_minutes / staffed_minutes if
staffed_minutes else np.nan,
632     }
633
634
635 def daily_outputs(
636     outcomes: pd.DataFrame,
637     schedule: pd.DataFrame,
638     calendar: WorkCalendar,
639     policy: str,
640 ) -> tuple[pd.DataFrame, pd.DataFrame]:
641     working = outcomes[outcomes["evaluation_cohort"]].copy()
642     working["metric_date"] = working["order_dt"].where(
643         working["source"].eq("住院"), working["planned_date"]
644     ).dt.normalize()
645     daily = (
646         working.groupby(["metric_date", "source"], as_index=False)
647         .agg(
648             events=("event_id", "size"),
649             deadline_met=("deadline_met", "sum"),
650             scheduled_complete=("complete_event", "sum"),
651             demand_nominal_minutes=("nominal_minutes", "sum"),
652             demand_upper_minutes=("upper_minutes", "sum"),
653         )
654     )
655     daily["deadline_rate"] = daily["deadline_met"] / daily["events"]
656     daily["policy"] = policy
657
658     if schedule.empty:
659         utilization = pd.DataFrame()
660     else:
661         schedule = schedule.copy()
662         schedule["date"] = schedule["start_dt"].dt.normalize()
663         utilization = (
664             schedule.groupby(["date", "room_id", "room_name"], as_index=False)["
duration_minutes"]
665             .sum()
666         )
667         utilization["room_utilization"] = utilization["duration_minutes"] / (8 * 60)
668         utilization["policy"] = policy
669     return daily, utilization
670
671
672 def run_policy(
673     events: pd.DataFrame,
674     tasks: dict[str, list[Task]],
675     slot_capacity: pd.DataFrame,
676     room_ids: list[str],
677     room_names: dict[str, str],
678     policy: str,
679     capacity_column: str,
680     scenario: str,
681     transfer_minutes: int = TRANSFER_MINUTES,
682     background_state: tuple[WorkCalendar, dict[str, list[PlanStep]], dict[str, str]] |
None = None,
683 ) -> tuple[
684     pd.DataFrame,
685     pd.DataFrame,
686     dict[str, Any],

```

```

687 pd.DataFrame,
688 pd.DataFrame,
689 tuple[WorkCalendar, dict[str, list[PlanStep]], dict[str, str]],
690 ]:
691     config = POLICIES[policy]
692     scarcity = task_room_scarcity()
693     simulation_start = source_config.SIMULATION_START
694     simulation_end = source_config.HOLDOUT_END_EXCLUSIVE + pd.Timedelta(days=
        FOLLOWUP_DAYS)
695     lookup = events.set_index("event_id")
696     inpatient_order = event_priority(events, tasks, "inpatient", config["inpatient"])
697     if background_state is None:
698         calendar = WorkCalendar(
699             simulation_start,
700             simulation_end,
701             room_ids,
702             slot_capacity,
703             capacity_column,
704             scarcity,
705             config["background"],
706             transfer_minutes,
707         )
708         background_order = event_priority(events, tasks, "background", config["
        background"])
709         print(f"{policy}: scheduling {len(background_order):,} outpatient/physical-
        exam events", flush=True)
710         background_plans, background_failure = schedule_group(
711             calendar, background_order, lookup, tasks
712         )
713         reusable_background = (
714             calendar.clone(),
715             background_plans,
716             background_failure,
717         )
718     else:
719         stored_calendar, background_plans, background_failure = background_state
720         calendar = stored_calendar.clone()
721         reusable_background = background_state
722         print(f"{policy}: reusing identical background allocation", flush=True)
723     print(f"{policy}: scheduling {len(inpatient_order):,} inpatient events", flush=
        True)
724     inpatient_plans, inpatient_failure = schedule_group(
725         calendar, inpatient_order, lookup, tasks
726     )
727     plans = {**background_plans, **inpatient_plans}
728     failure = {**background_failure, **inpatient_failure}
729     schedule = plan_to_schedule(calendar, plans, events, policy, room_names)
730     outcomes = outcomes_from_schedule(events, schedule, failure, policy)
731     metrics = metrics_from_outcomes(outcomes, schedule, calendar, policy, scenario)
732     metrics["transfer_minutes_assumption"] = transfer_minutes
733     daily, utilization = daily_outputs(outcomes, schedule, calendar, policy)
734     print(
735         f"{policy}: inpatient48h={metrics['inpatient_48h_rate']:.4%}, "
736         f"background={metrics['background_on_time_rate']:.4%}, tasks={len(schedule):,}
737     ",
738         flush=True,
739     )
740     return schedule, outcomes, metrics, daily, utilization, reusable_background
741

```

```

742 def recommendation_table(schedule: pd.DataFrame, outcomes: pd.DataFrame) -> pd.
    DataFrame:
743     evaluation = as_bool(outcomes["evaluation_cohort"])
744     inpatient_ids = set(
745         outcomes.loc[outcomes["source"].eq("住院") & evaluation, "event_id"]
746     )
747     result = schedule[schedule["event_id"].isin(inpatient_ids)].copy()
748     result = result.sort_values(["start_dt", "room_id", "event_id", "sequence_position"], kind="stable")
749     event_rank = (
750         result[["event_id"]]
751         .drop_duplicates()
752         .assign(first_start=result.groupby("event_id", sort=False)["start_dt"].first()
753             .values)
754         .sort_values(["first_start", "event_id"], kind="stable")
755     )
756     event_rank["recommended_patient_rank"] = np.arange(1, len(event_rank) + 1)
757     result = result.merge(
758         event_rank[["event_id", "recommended_patient_rank"]], on="event_id", how="left"
759     )
760     columns = [
761         "recommended_patient_rank",
762         "event_id",
763         "patient_id",
764         "sequence_position",
765         "project_norm",
766         "category",
767         "start_dt",
768         "end_dt",
769         "room_id",
770         "room_name",
771         "transfer_minutes_before",
772         "fasting",
773         "bladder",
774         "bedside",
775         "capability_level",
776     ]
777     return result[columns].sort_values(
778         ["recommended_patient_rank", "sequence_position"], kind="stable"
779     )
780
781 def main() -> None:
782     parser = argparse.ArgumentParser()
783     parser.add_argument("--output-dir", type=Path, default=OUT)
784     parser.add_argument("--main-only", action="store_true")
785     parser.add_argument("--sensitivity-only", action="store_true")
786     parser.add_argument("--main-policy", choices=list(POLICIES))
787     parser.add_argument("--sensitivity-scenario")
788     args = parser.parse_args()
789     if args.main_only and args.sensitivity_only:
790         parser.error("--main-only and --sensitivity-only cannot be used together")
791     if args.sensitivity_scenario and not args.sensitivity_only:
792         parser.error("--sensitivity-scenario requires --sensitivity-only")
793     if args.main_policy and args.sensitivity_only:
794         parser.error("--main-policy cannot be combined with --sensitivity-only")
795     args.output_dir.mkdir(parents=True, exist_ok=True)
796
797     metrics_rows: list[dict[str, Any]] = []

```

```

798 daily_rows: list[pd.DataFrame] = []
799 authoritative_schedule: pd.DataFrame | None = None
800 authoritative_outcomes: pd.DataFrame | None = None
801 authoritative_utilization: pd.DataFrame | None = None
802 if args.sensitivity_only:
803     existing_metrics = pd.read_csv(args.output_dir / "policy_scenario_metrics.csv"
804 )
805     existing_metrics = existing_metrics[existing_metrics["scenario"].eq("
806 main_5pct_upper")]
807     metrics_rows.extend(existing_metrics.to_dict(orient="records"))
808     existing_daily = pd.read_csv(args.output_dir / "daily_service_metrics.csv",
809 low_memory=False)
810     if "scenario" not in existing_daily.columns:
811         existing_daily["scenario"] = "main_5pct_upper"
812     existing_daily["scenario"] = existing_daily["scenario"].fillna("
813 main_5pct_upper")
814     daily_rows.append(existing_daily[existing_daily["scenario"].eq("
815 main_5pct_upper")])
816 else:
817     main_events, main_tasks, slot_capacity, room_ids, room_names = load_inputs(
818 "five_percent_upper", "hierarchical", source_config.
819 BLADDER_PREPARATION_MINUTES
820 )
821     background_cache: dict[
822 str, tuple[WorkCalendar, dict[str, list[PlanStep]], dict[str, str]]
823 ] = {}
824     selected_policies = [args.main_policy] if args.main_policy else list(POLICIES)
825     for policy in selected_policies:
826         background_key = POLICIES[policy]["background"]
827         schedule, outcomes, metrics, daily, utilization, reusable_background =
828 run_policy(
829     main_events,
830     main_tasks,
831     slot_capacity,
832     room_ids,
833     room_names,
834     policy,
835     "capacity_main",
836     "main_5pct_upper",
837     TRANSFER_MINUTES,
838     background_cache.get(background_key),
839 )
840     background_cache.setdefault(background_key, reusable_background)
841     metrics_rows.append(metrics)
842     daily_rows.append(daily.assign(scenario="main_5pct_upper"))
843     if policy == AUTHORITATIVE_POLICY:
844         authoritative_schedule = schedule
845         authoritative_outcomes = outcomes
846         authoritative_utilization = utilization
847
848 sensitivity_specs = [] if args.main_only else [
849     ("nominal_no_special", "five_percent_upper", "hierarchical", "capacity_main",
850 60, False, 10),
851     ("low_doctor_q25", "five_percent_upper", "hierarchical", "capacity_low", 60,
852 True, 10),
853     ("strict_capability", "five_percent_upper", "strict", "capacity_main", 60,
854 True, 10),
855     ("bladder_45min", "five_percent_upper", "hierarchical", "capacity_main", 45,
856 True, 10),
857     ("bladder_90min", "five_percent_upper", "hierarchical", "capacity_main", 90,

```

```

True, 10),
847     ("transfer_5min", "five_percent_upper", "hierarchical", "capacity_main", 60,
True, 5),
848     ("transfer_15min", "five_percent_upper", "hierarchical", "capacity_main", 60,
True, 15),
849 ]
850 if args.sensitivity_scenario:
851     sensitivity_specs = [
852         spec for spec in sensitivity_specs if spec[0] == args.sensitivity_scenario
853     ]
854     if not sensitivity_specs:
855         parser.error(f"unknown sensitivity scenario: {args.sensitivity_scenario}")
856 # The first tuple uses a flag to switch all events to nominal duration.
857 for name, special_mode, capability_mode, capacity_column, bladder_minutes,
use_special, transfer_minutes in sensitivity_specs:
858     effective_special = special_mode if use_special else "all_nominal"
859     events, tasks, capacity, rooms, names = load_inputs(
860         effective_special, capability_mode, bladder_minutes
861     )
862     _, _, metrics, daily, _, _ = run_policy(
863         events,
864         tasks,
865         capacity,
866         rooms,
867         names,
868         AUTHORITATIVE_POLICY,
869         capacity_column,
870         name,
871         transfer_minutes,
872     )
873     metrics_rows.append(metrics)
874     daily_rows.append(daily.assign(scenario=name))
875
876 metrics_frame = pd.DataFrame(metrics_rows)
877 metrics_frame.to_csv(args.output_dir / "policy_scenario_metrics.csv", index=False,
encoding="utf-8-sig")
878 pd.concat(daily_rows, ignore_index=True).to_csv(
879     args.output_dir / "daily_service_metrics.csv", index=False, encoding="utf-8-
sig"
880 )
881 if not args.sensitivity_only:
882     assert authoritative_schedule is not None
883     assert authoritative_outcomes is not None
884     assert authoritative_utilization is not None
885     authoritative_schedule.to_csv(
886         args.output_dir / "final_patient_task_schedule.csv", index=False, encoding
="utf-8-sig"
887     )
888     authoritative_outcomes.to_csv(
889         args.output_dir / "final_patient_outcomes.csv", index=False, encoding="utf
-8-sig"
890     )
891     authoritative_utilization.to_csv(
892         args.output_dir / "final_room_utilization_daily.csv", index=False,
encoding="utf-8-sig"
893     )
894     recommendation_table(authoritative_schedule, authoritative_outcomes).to_csv(
895         args.output_dir / "inpatient_recommended_schedule.csv", index=False,
encoding="utf-8-sig"
896     )

```

```

897     summary = {
898         "authoritative_policy": AUTHORITATIVE_POLICY,
899         "annual_state_reset_count": 1,
900         "simulation_start": str(source_config.SIMULATION_START.date()),
901         "simulation_end_exclusive": str(
902             (source_config.HOLDOUT_END_EXCLUSIVE + pd.Timedelta(days=FOLLOWUP_DAYS)).
date()
903         ),
904         "evaluation_start": str(source_config.HOLDOUT_START.date()),
905         "evaluation_end_exclusive": str(source_config.HOLDOUT_END_EXCLUSIVE.date()),
906         "background_interpretation": "retrospective plan-day scenario reconstructed
from report date, with release no earlier than the observed order time; report
time/room/doctor dispatch are excluded",
907         "official_kpi_boundary": "scheduled service completion is used as report-
submission completion proxy because scan-end timestamp is absent",
908         "transfer_minutes_between_rooms": TRANSFER_MINUTES,
909         "metrics": metrics_frame.to_dict(orient="records"),
910     }
911     (args.output_dir / "summary.json").write_text(
912         json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
913     )
914     print(metrics_frame.to_string(index=False), flush=True)
915
916
917 if __name__ == "__main__":
918     main()

```

Listing 9 P2 纯住院、P3 网格与敏感性场景运行器

```

1  """Run the pre-freeze P2 and P3 scenarios on the audited scheduling kernel."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  import math
8  from pathlib import Path
9  from typing import Any
10
11  import numpy as np
12  import pandas as pd
13
14  import run_unified_scheduler as scheduler
15
16
17  ROOT = Path(__file__).resolve().parents[2]
18  OUT = ROOT / "supporting_materials" / "results" / "final_frozen"
19  ALPHAS = [0.0, 0.30, 0.40, 0.50, 0.55, 0.60, 0.65, 0.70]
20
21
22  def save_json(path: Path, payload: dict[str, Any]) -> None:
23      path.parent.mkdir(parents=True, exist_ok=True)
24      path.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
25
26
27  def run_p2() -> None:
28      events, tasks, capacity, rooms, room_names = scheduler.load_inputs(
29          "five_percent_upper", "hierarchical", 60, preparation_mode="item"
30      )

```

```

31 events = events[~events["mandatory_background"]].copy()
32 tasks = {event_id: tasks[event_id] for event_id in events["event_id"]}
33 output = OUT / "p2_inpatient_only"
34 output.mkdir(parents=True, exist_ok=True)
35 candidates: list[tuple[str, pd.DataFrame, pd.DataFrame, dict[str, Any], pd.
DataFrame, pd.DataFrame]] = []
36 for policy in ["FCFS_SHARED", "SLACK_GUARD_SHARED"]:
37     schedule, outcomes, metrics, daily, utilization, _ = scheduler.run_policy(
38         events, tasks, capacity, rooms, room_names, policy, "capacity_main",
39         "P2_INPATIENT_ONLY", scheduler.TRANSFER_MINUTES,
40     )
41     candidates.append((policy, schedule, outcomes, metrics, daily, utilization))
42 selected = max(
43     candidates,
44     key=lambda item: (
45         item[3]["inpatient_complete_48h"],
46         -item[3]["inpatient_wait_p90_hours_conditional"],
47     ),
48 )
49 policy, schedule, outcomes, metrics, daily, utilization = selected
50 pd.DataFrame([item[3] for item in candidates]).to_csv(
51     output / "policy_metrics.csv", index=False, encoding="utf-8-sig"
52 )
53 schedule.to_csv(output / "final_patient_task_schedule.csv", index=False, encoding=
"utf-8-sig")
54 outcomes.to_csv(output / "final_patient_outcomes.csv", index=False, encoding="utf
-8-sig")
55 daily.to_csv(output / "daily_service_metrics.csv", index=False, encoding="utf-8-
sig")
56 utilization.to_csv(output / "room_utilization_daily.csv", index=False, encoding="
utf-8-sig")
57 scheduler.recommendation_table(schedule, outcomes).to_csv(
58     output / "inpatient_recommended_schedule.csv", index=False, encoding="utf-8-
sig"
59 )
60 failure = (
61     outcomes.loc[~outcomes["deadline_met"], "failure_reason"]
62     .fillna("unknown")
63     .value_counts()
64     .rename_axis("failure_reason")
65     .reset_index(name="event_count")
66 )
67 failure.to_csv(output / "failure_reason_breakdown.csv", index=False, encoding="utf
-8-sig")
68 save_json(
69     output / "summary.json",
70     {
71         "scenario": "P2_INPATIENT_ONLY",
72         "selected_policy": policy,
73         "preparation_granularity": "item",
74         "metrics": metrics,
75     },
76 )
77
78
79 def schedule_quota(
80     calendar: scheduler.WorkCalendar,
81     event_ids: list[str],
82     lookup: pd.DataFrame,
83     tasks: dict[str, list[scheduler.Task]],

```

```

84     alpha: float,
85 ) -> tuple[dict[str, list[scheduler.PlanStep]], dict[str, str], list[str], dict[str,
Any]]:
86     plans: dict[str, list[scheduler.PlanStep]] = {}
87     failure: dict[str, str] = {}
88     remaining: list[str] = []
89     frame = lookup.loc[event_ids].copy()
90     warmup_ids = frame.index[~frame["evaluation_cohort"]].tolist()
91     warmup_plans, warmup_failure = scheduler.schedule_group(calendar, warmup_ids,
lookup, tasks)
92     plans.update(warmup_plans)
93     failure.update(warmup_failure)
94
95     daily_rows: list[dict[str, Any]] = []
96     evaluation = frame[frame["evaluation_cohort"]].copy()
97     evaluation["quota_date"] = evaluation["planned_date"].dt.normalize()
98     order_rank = {event_id: rank for rank, event_id in enumerate(event_ids)}
99     for date, group in evaluation.groupby("quota_date", sort=True):
100         ids = sorted(group.index.tolist(), key=order_rank.get)
101         target = int(math.ceil(alpha * len(ids)))
102         achieved = 0
103         attempted = 0
104         for position, event_id in enumerate(ids):
105             if achieved >= target:
106                 remaining.extend(ids[position:])
107                 break
108             event = lookup.loc[event_id]
109             steps = scheduler.plan_event(calendar, tasks[event_id], event.release_dt,
event.deadline_dt)
110             attempted += 1
111             if steps is None:
112                 failure[event_id] = "no_compatible_room_under_scenario" if any(
113                     not task.compatible_rooms for task in tasks[event_id]
114                 ) else "no_complete_plan_by_deadline"
115             else:
116                 plans[event_id] = steps
117                 achieved += 1
118         else:
119             remaining.extend([])
120         daily_rows.append(
121             {
122                 "date": str(pd.Timestamp(date).date()),
123                 "events": len(ids),
124                 "target": target,
125                 "achieved_before_inpatient": achieved,
126                 "attempted": attempted,
127                 "target_met": achieved >= target,
128             }
129         )
130     daily = pd.DataFrame(daily_rows)
131     diagnostics = {
132         "alpha": alpha,
133         "quota_days": int(len(daily)),
134         "quota_shortfall_days": int((~daily["target_met"]).sum()) if len(daily) else
0,
135         "quota_target_events": int(daily["target"].sum()) if len(daily) else 0,
136         "quota_achieved_before_inpatient": int(daily["achieved_before_inpatient"].sum
()) if len(daily) else 0,
137         "daily_rows": daily_rows,
138     }

```

```

139     return plans, failure, remaining, diagnostics
140
141
142 def run_p3_point(
143     alpha: float,
144     save_detail: bool,
145     preparation_mode: str = "item",
146     special_seed: int | None = None,
147     label: str | None = None,
148     capability_mode: str = "hierarchical",
149     capacity_column: str = "capacity_main",
150     bladder_minutes: int = 60,
151     transfer_minutes: int = scheduler.TRANSFER_MINUTES,
152 ) -> None:
153     p2_summary = json.loads((OUT / "p2_inpatient_only" / "summary.json").read_text(
154         encoding="utf-8"))
155     policy = p2_summary["selected_policy"]
156     events, tasks, capacity, rooms, room_names = scheduler.load_inputs(
157         "five_percent_upper", capability_mode, bladder_minutes,
158         preparation_mode=preparation_mode, special_seed=special_seed,
159     )
160     lookup = events.set_index("event_id")
161     scarcity = scheduler.task_room_scarcity()
162     calendar = scheduler.WorkCalendar(
163         scheduler.source_config.SIMULATION_START,
164         scheduler.source_config.HOLDOUT_END_EXCLUSIVE + pd.Timedelta(days=scheduler.
165         FOLLOWUP_DAYS),
166         rooms, capacity, capacity_column, scarcity, "scarcity", transfer_minutes,
167     )
168     background_order = scheduler.event_priority(events, tasks, "background", "scarcity")
169     background_plans, background_failure, remaining, quota = schedule_quota(
170         calendar, background_order, lookup, tasks, alpha
171     )
172     inpatient_order = scheduler.event_priority(
173         events, tasks, "inpatient", scheduler.POLICIES[policy]["inpatient"]
174     )
175     inpatient_plans, inpatient_failure = scheduler.schedule_group(
176         calendar, inpatient_order, lookup, tasks
177     )
178     residual_plans, residual_failure = scheduler.schedule_group(
179         calendar, remaining, lookup, tasks
180     )
181     plans = {**background_plans, **inpatient_plans, **residual_plans}
182     failure = {**background_failure, **inpatient_failure, **residual_failure}
183     schedule = scheduler.plan_to_schedule(calendar, plans, events, policy, room_names)
184     outcomes = scheduler.outcomes_from_schedule(events, schedule, failure, policy)
185     metrics = scheduler.metrics_from_outcomes(
186         outcomes, schedule, calendar, policy, f"P3_ALPHA_{alpha:.2f}"
187     )
188     metrics.update(
189         {
190             "background_target_alpha": alpha,
191             "quota_shortfall_days": quota["quota_shortfall_days"],
192             "quota_target_events": quota["quota_target_events"],
193             "quota_achieved_before_inpatient": quota["quota_achieved_before_inpatient"]
194         },
195         {
196             "preparation_granularity": preparation_mode,
197             "special_seed": special_seed if special_seed is not None else scheduler.
198             source_config.SEED,
199         }
200     )

```

```

194         "capability_mode": capability_mode,
195         "capacity_column": capacity_column,
196         "bladder_minutes": bladder_minutes,
197         "transfer_minutes": transfer_minutes,
198     }
199 )
200 shard = (
201     OUT / "p3_shards" / f"alpha_{alpha:.2f}"
202     if label is None
203     else OUT / "sensitivity_shards" / label
204 )
205 shard.mkdir(parents=True, exist_ok=True)
206 pd.DataFrame([metrics]).to_csv(shard / "metrics.csv", index=False, encoding="utf-8-sig")
207 save_json(shard / "quota_diagnostics.json", quota)
208 if save_detail:
209     schedule.to_csv(shard / "final_patient_task_schedule.csv", index=False,
210 encoding="utf-8-sig")
211     outcomes.to_csv(shard / "final_patient_outcomes.csv", index=False, encoding="utf-8-sig")
212     daily, utilization = scheduler.daily_outputs(outcomes, schedule, calendar,
213 policy)
214     daily.to_csv(shard / "daily_service_metrics.csv", index=False, encoding="utf-8-sig")
215     utilization.to_csv(shard / "room_utilization_daily.csv", index=False, encoding="utf-8-sig")
216     scheduler.recommendation_table(schedule, outcomes).to_csv(
217         shard / "inpatient_recommended_schedule.csv", index=False, encoding="utf-8-sig")
218 )
219 def finalize_p3() -> None:
220     parts = [
221         pd.read_csv(OUT / "p3_shards" / f"alpha_{alpha:.2f}" / "metrics.csv")
222         for alpha in ALPHAS
223     ]
224     metrics = pd.concat(parts, ignore_index=True).sort_values("background_target_alpha")
225     # The epsilon constraint is defined on the holdout-wide background
226     # completion rate, matching the competition KPI. Daily quota shortfalls
227     # remain a diagnostic for heterogeneous demand/capability and are not a
228     # reason to discard a point that satisfies the declared annual epsilon.
229     feasible = metrics[
230         metrics["background_on_time_rate"] + 1e-12
231         >= metrics["background_target_alpha"]
232     ].copy()
233     if feasible.empty:
234         raise RuntimeError("all P3 epsilon constraints are infeasible")
235     nondominated = []
236     for row in feasible.itertuples(index=False):
237         dominated = (
238             (feasible["background_on_time_rate"] >= row.background_on_time_rate)
239             & (feasible["inpatient_48h_rate"] >= row.inpatient_48h_rate)
240             & (
241                 (feasible["background_on_time_rate"] > row.background_on_time_rate)
242                 | (feasible["inpatient_48h_rate"] > row.inpatient_48h_rate)
243             )
244         ).any()
245     nondominated.append(not dominated)

```

```

246 feasible["pareto_nondominated"] = nondominated
247 front = feasible[feasible["pareto_nondominated"]].copy()
248 bg_min, bg_max = front["background_on_time_rate"].min(), front["
background_on_time_rate"].max()
249 ip_min, ip_max = front["inpatient_48h_rate"].min(), front["inpatient_48h_rate"].
max()
250 front["distance_to_ideal"] = np.sqrt(
251     ((bg_max - front["background_on_time_rate"]) / max(bg_max - bg_min, 1e-12)) **
2
252     + ((ip_max - front["inpatient_48h_rate"]) / max(ip_max - ip_min, 1e-12)) ** 2
253 )
254 selected = front.sort_values(
255     ["distance_to_ideal", "background_on_time_rate", "inpatient_48h_rate"],
256     ascending=[True, False, False],
257 ).iloc[0]
258 metrics = metrics.merge(
259     feasible[["background_target_alpha", "pareto_nondominated"]],
260     on="background_target_alpha", how="left",
261 )
262 output = OUT / "p3_joint"
263 output.mkdir(parents=True, exist_ok=True)
264 metrics.to_csv(output / "pareto_metrics.csv", index=False, encoding="utf-8-sig")
265 save_json(
266     output / "selection.json",
267     {
268         "selection_rule": "minimum normalized Euclidean distance to the attainable
Pareto ideal",
269         "selected_alpha": float(selected["background_target_alpha"]),
270         "selected_metrics": selected.to_dict(),
271     },
272 )
273 print(f"SELECTED_ALPHA={selected['background_target_alpha']:.2f}")
274
275
276 def main() -> None:
277     parser = argparse.ArgumentParser()
278     parser.add_argument("--stage", choices=["p2", "p3-point", "finalize-p3"], required
=True)
279     parser.add_argument("--alpha", type=float)
280     parser.add_argument("--save-detail", action="store_true")
281     parser.add_argument("--preparation-mode", choices=["item", "event"], default="item
")
282     parser.add_argument("--special-seed", type=int)
283     parser.add_argument("--label")
284     parser.add_argument("--capability-mode", choices=["hierarchical", "strict"],
default="hierarchical")
285     parser.add_argument("--capacity-column", choices=["capacity_main", "capacity_low"
], default="capacity_main")
286     parser.add_argument("--bladder-minutes", type=int, default=60)
287     parser.add_argument("--transfer-minutes", type=int, default=scheduler.
TRANSFER_MINUTES)
288     args = parser.parse_args()
289     if args.stage == "p2":
290         run_p2()
291     elif args.stage == "p3-point":
292         if args.alpha is None:
293             parser.error("--alpha is required for p3-point")
294         run_p3_point(
295             args.alpha, args.save_detail, args.preparation_mode, args.special_seed,
296             args.label, args.capability_mode, args.capacity_column,

```

```

297         args.bladder_minutes, args.transfer_minutes,
298     )
299     else:
300         finalize_p3()
301
302
303 if __name__ == "__main__":
304     main()

```

E 精确对照与现实校准程序

Listing 10 代表性小窗口 CP-SAT 精确对照

```

1  """Compare the annual construction heuristic with CP-SAT on small P2 windows."""
2
3  from __future__ import annotations
4
5  import json
6  import time
7  from concurrent.futures import ThreadPoolExecutor
8  from pathlib import Path
9
10 import numpy as np
11 import pandas as pd
12 from ortools.sat.python import cp_model
13
14 import run_unified_scheduler as scheduler
15
16
17 ROOT = Path(__file__).resolve().parents[2]
18 OUT = ROOT / "supporting_materials" / "results" / "exact_benchmark"
19 MAX_EVENTS = 18
20 TIME_LIMIT_SECONDS = 60.0
21
22
23 def representative_cases(events: pd.DataFrame, tasks: dict[str, list[scheduler.Task]])
24     -> list[tuple[str, pd.Timestamp, list[str]]]:
25     inpatient = events[
26         (~events["mandatory_background"]) & events["evaluation_cohort"]
27     ].copy()
28     inpatient["date"] = inpatient["order_dt"].dt.normalize()
29     inpatient["multi"] = inpatient["service_project_count"].gt(1)
30     inpatient["bedside_task"] = inpatient["event_id"].map(
31         lambda event_id: any(task.bedside for task in tasks[event_id])
32     )
33     inpatient["obstetric_task"] = inpatient["event_id"].map(
34         lambda event_id: any(task.category == "产科III/IV级" for task in tasks[
35             event_id])
36     )
37     daily = inpatient.groupby("date").agg(
38         events=("event_id", "size"),
39         multi_share=("multi", "mean"),
40         bedside_count=("bedside_task", "sum"),
41         obstetric_count=("obstetric_task", "sum"),
42     )
43     weekday = daily[daily.index.dayofweek < 5]
44     weekend = daily[daily.index.dayofweek >= 5]
45     normal_date = (weekday["events"] - weekday["events"].median()).abs().idxmin()

```

```

44     dates = {
45         "normal_weekday": normal_date,
46         "peak_weekday": weekday["events"].idxmax(),
47         "weekend": weekend["events"].idxmax(),
48         "multi_project_peak": daily["multi_share"].idxmax(),
49         "bedside_peak": daily["bedside_count"].idxmax(),
50         "obstetric_peak": daily["obstetric_count"].idxmax(),
51     }
52     cases = []
53     for name, date in dates.items():
54         group = inpatient[inpatient["date"].eq(date)].copy()
55         if name == "bedside_peak":
56             group = group.sort_values(["bedside_task", "service_project_count", "
order_dt"], ascending=[False, False, True])
57         elif name == "obstetric_peak":
58             group = group.sort_values(["obstetric_task", "service_project_count", "
order_dt"], ascending=[False, False, True])
59         elif name == "multi_project_peak":
60             group = group.sort_values(["service_project_count", "order_dt"], ascending
=[False, True])
61         else:
62             group = group.sort_values(["order_dt", "event_id"])
63             case_limit = 10 if name == "multi_project_peak" else MAX_EVENTS
64             cases.append((name, pd.Timestamp(date), group["event_id"].head(case_limit).
tolist()))
65     return cases
66
67
68 def feasible_options(
69     calendar: scheduler.WorkCalendar,
70     task: scheduler.Task,
71     release: pd.Timestamp,
72     deadline: pd.Timestamp,
73 ) -> list[tuple[str, int]]:
74     release = max(release, task.bladder_ready_dt) if task.bladder else release
75     start = calendar.search_start(release)
76     stop = min(
77         int(calendar.times.searchsorted(deadline, side="left")),
78         len(calendar.times) - task.duration_slots + 1,
79     )
80     options: list[tuple[str, int]] = []
81     for index in range(start, stop):
82         if not calendar.fits_block(index, task.duration_slots):
83             continue
84         end = calendar.end_timestamp(index, task.duration_slots)
85         if end > deadline:
86             continue
87         if task.fasting and end > calendar.times[index].normalize() + pd.Timedelta(
hours=10):
88             continue
89         for room in task.compatible_rooms:
90             if room in calendar.room_index:
91                 options.append((room, index))
92     return options
93
94
95 def solve_case(
96     name: str,
97     date: pd.Timestamp,
98     event_ids: list[str],

```

```

99     events: pd.DataFrame,
100     tasks: dict[str, list[scheduler.Task]],
101     capacity: pd.DataFrame,
102     rooms: list[str],
103     policy: str,
104 ) -> dict[str, object]:
105     lookup = events.set_index("event_id")
106     start_date = date
107     end_date = date + pd.Timedelta(days=3)
108     calendar = scheduler.WorkCalendar(
109         start_date, end_date, rooms, capacity, "capacity_main", scheduler.
110         task_room_scarcity(),
111         "scarcity", scheduler.TRANSFER_MINUTES,
112     )
113     subset = events[events["event_id"].isin(event_ids)].copy()
114     heuristic_order = scheduler.event_priority(
115         subset, tasks, "inpatient", scheduler.POLICIES[policy]["inpatient"]
116     )
117     heuristic_calendar = calendar.clone()
118     heuristic_plans, _ = scheduler.schedule_group(
119         heuristic_calendar, heuristic_order, lookup, tasks
120     )
121     heuristic_value = len(heuristic_plans)
122
123     model = cp_model.CpModel()
124     event_selected: dict[str, cp_model.IntVar] = {}
125     task_vars: dict[tuple[str, int], dict[str, object]] = {}
126     room_intervals: dict[str, list[cp_model.IntervalVar]] = {room: [] for room in
127     rooms}
128     doctor_slot_vars: dict[int, list[cp_model.IntVar]] = {}
129
130     for event_id in event_ids:
131         event = lookup.loc[event_id]
132         selected = model.new_bool_var(f"y_{event_id}")
133         event_selected[event_id] = selected
134         local_tasks = tasks[event_id]
135         for task in local_tasks:
136             options = feasible_options(calendar, task, event.release_dt, event.
137             deadline_dt)
138             option_vars: list[cp_model.IntVar] = []
139             room_selected: dict[str, cp_model.IntVar] = {}
140             start_var = model.new_int_var(0, len(calendar.times), f"s_{event_id}_{task
141             .item_index}")
142             for room in task.compatible_rooms:
143                 room_selected[room] = model.new_bool_var(f"r_{event_id}_{task.
144                 item_index}_{room}")
145             by_room: dict[str, list[cp_model.IntVar]] = {room: [] for room in task.
146             compatible_rooms}
147             start_terms = []
148             for room, start in options:
149                 x = model.new_bool_var(f"x_{event_id}_{task.item_index}_{room}_{start}
150                 ")
151                 option_vars.append(x)
152                 by_room[room].append(x)
153                 start_terms.append(start * x)
154                 interval = model.new_optional_fixed_size_interval_var(
155                     start, task.duration_slots, x,
156                     f"iv_{event_id}_{task.item_index}_{room}_{start}",
157                 )
158                 room_intervals[room].append(interval)

```

```

152         for slot in range(start, start + task.duration_slots):
153             doctor_slot_vars.setdefault(slot, []).append(x)
154     if option_vars:
155         model.add(sum(option_vars) == selected)
156         model.add(start_var == sum(start_terms))
157         for room, room_vars in by_room.items():
158             model.add(room_selected[room] == sum(room_vars))
159     else:
160         model.add(selected == 0)
161         for room in room_selected:
162             model.add(room_selected[room] == 0)
163     task_vars[(event_id, task.item_index)] = {
164         "task": task,
165         "start": start_var,
166         "rooms": room_selected,
167     }
168
169     for left_index in range(len(local_tasks)):
170         for right_index in range(left_index + 1, len(local_tasks)):
171             left = task_vars[(event_id, local_tasks[left_index].item_index)]
172             right = task_vars[(event_id, local_tasks[right_index].item_index)]
173             left_before = model.new_bool_var(f"ord_{event_id}_{left_index}_{
right_index}")
174             for left_room, left_room_var in left["rooms"].items():
175                 for right_room, right_room_var in right["rooms"].items():
176                     gap = scheduler.TRANSFER_MINUTES // scheduler.SLOT_MINUTES if
left_room != right_room else 0
177                     model.add(
178                         right["start"] >= left["start"] + left["task"].
duration_slots + gap
179                     ).only_enforce_if([selected, left_before, left_room_var,
right_room_var])
180                     model.add(
181                         left["start"] >= right["start"] + right["task"].
duration_slots + gap
182                     ).only_enforce_if([selected, left_before.negated(),
left_room_var, right_room_var])
183
184     for room, intervals in room_intervals.items():
185         if intervals:
186             model.add_no_overlap(intervals)
187     for slot, variables in doctor_slot_vars.items():
188         model.add(sum(variables) <= int(calendar.capacity[slot]))
189     model.maximize(sum(event_selected.values()))
190     solver = cp_model.CpSolver()
191     solver.parameters.max_time_in_seconds = TIME_LIMIT_SECONDS
192     # Three windows are solved concurrently below; two CP-SAT workers per
193     # window keep aggregate CPU use bounded.
194     solver.parameters.num_search_workers = 2
195     started = time.perf_counter()
196     status = solver.solve(model)
197     elapsed = time.perf_counter() - started
198     exact_value = int(round(solver.objective_value)) if status in (cp_model.OPTIMAL,
cp_model.FEASIBLE) else 0
199     bound = float(solver.best_objective_bound) if status in (cp_model.OPTIMAL,
cp_model.FEASIBLE) else float(len(event_ids))
200     return {
201         "case": name,
202         "date": str(date.date()),
203         "event_count": len(event_ids),

```

```

204         "heuristic_complete": heuristic_value,
205         "exact_complete": exact_value,
206         "best_bound": bound,
207         "heuristic_gap_to_exact": (exact_value - heuristic_value) / max(exact_value,
1),
208         "heuristic_gap_to_bound": (bound - heuristic_value) / max(bound, 1),
209         "solver_status": solver.status_name(status),
210         "solve_seconds": elapsed,
211     }
212
213
214 def write_report(frame: pd.DataFrame) -> None:
215     columns = list(frame.columns)
216     markdown = [
217         "|" + "|".join(columns) + "|",
218         "|" + "|".join(["---"] * len(columns)) + "|",
219     ]
220     for row in frame.itertuples(index=False, name=None):
221         markdown.append("|" + "|".join(str(value).replace("|", "/") for value in row)
+ "|")
222     lines = [
223         "## Exact vs Heuristic Benchmark",
224         "",
225         "代表性小规模住院窗口使用与主模型相同的项目时长、设备集合、医生总并发、准备条件、患者互斥和
跨室转运约束。CP-SAT 在每个窗口内最大化48小时完整完成患者数；全年模型仍采用构造算法。",
226         "",
227         "\n".join(markdown),
228         "",
229         f"最大启发式相对精确解差距: {frame['heuristic_gap_to_exact'].max():.4%}。",
230         f"最大启发式相对最好界差距: {frame['heuristic_gap_to_bound'].max():.4%}。",
231     ]
232     (OUT / "EXACT_VS_HEURISTIC_REPORT.md").write_text("\n".join(lines), encoding="utf
-8")
233
234
235 def main() -> None:
236     OUT.mkdir(parents=True, exist_ok=True)
237     events, tasks, capacity, rooms, _ = scheduler.load_inputs(
238         "five_percent_upper", "hierarchical", 60, preparation_mode="item"
239     )
240     p2 = json.loads(
241         (ROOT / "supporting_materials" / "results" / "final_frozen" / "
p2_inpatient_only" / "summary.json").read_text(encoding="utf-8")
242     )
243     policy = p2["selected_policy"]
244     cases = representative_cases(events, tasks)
245     with ThreadPoolExecutor(max_workers=3) as pool:
246         rows = list(
247             pool.map(
248                 lambda case: solve_case(*case, events, tasks, capacity, rooms, policy)
249             ,
250             cases,
251         )
252     )
253     frame = pd.DataFrame(rows)
254     frame.to_csv(OUT / "exact_vs_heuristic_metrics.csv", index=False, encoding="utf-8-
sig")
255     write_report(frame)
256     print(frame.to_string(index=False))

```

```

257
258 if __name__ == "__main__":
259     main()

```

Listing 11 C0-C7 历史现实校准

```

1  """Run the staged historical-to-model calibration decomposition."""
2
3  from __future__ import annotations
4
5  import argparse
6  import json
7  from dataclasses import replace
8  from pathlib import Path
9
10 import pandas as pd
11
12 import analyze_p1 as p1
13 import run_unified_scheduler as scheduler
14
15
16 ROOT = Path(__file__).resolve().parents[2]
17 RAW = ROOT / "supporting_materials" / "processed_data" / "raw_parquet"
18 OUT = ROOT / "supporting_materials" / "results" / "calibration"
19
20
21 def historical_holdout_rate() -> tuple[int, int, float]:
22     frame = p1.load_detail(RAW / "inpatient.parquet", "住院")
23     frame = frame[
24         frame["order_dt"].between(pd.Timestamp("2024-04-01"), pd.Timestamp("2025-04-01"), inclusive="left")
25     ].copy()
26     frame = frame.sort_values("source_row", kind="stable")
27     duplicate_columns = ["patient_id", "order_dt", "ordered_project", "report_dt", "exam_doctor_id", "machine_id"]
28     frame = frame[~frame.duplicated(duplicate_columns, keep="first")]
29     grouped = frame.groupby(["patient_id", "order_dt"], dropna=False).agg(
30         rows=("source_row", "size"), reports=("report_dt", "count"), completion=("report_dt", "max")
31     ).reset_index()
32     grouped["complete"] = grouped["reports"].eq(grouped["rows"]) & grouped["completion"]
33     grouped["order_dt"] + pd.Timedelta(hours=48)
34     ) & grouped["completion"].ge(grouped["order_dt"])
35     return len(grouped), int(grouped["complete"].sum()), float(grouped["complete"].mean())
36
37
38 def run_variant(name: str) -> None:
39     events, tasks, capacity, rooms, room_names = scheduler.load_inputs(
40         "five_percent_upper", "hierarchical", 60, preparation_mode="item"
41     )
42     events = events[~events["mandatory_background"]].copy()
43     tasks = {event_id: tasks[event_id] for event_id in events["event_id"]}
44     if name in {"C2_standard_hours", "C3_doctor_capacity":
45         tasks = {
46             event_id: [
47                 replace(task, compatible_rooms=tuple(rooms), fasting=False, bladder=False, bedside=False)
48                 for task in event_tasks

```

```

49         ]
50         for event_id, event_tasks in tasks.items()
51     }
52     elif name == "C4_project_capability":
53         tasks = {
54             event_id: [replace(task, fasting=False, bladder=False) for task in
event_tasks]
55             for event_id, event_tasks in tasks.items()
56         }
57     if name == "C2_standard_hours":
58         capacity = capacity.copy()
59         capacity["capacity_main"] = len(rooms)
60     schedule, outcomes, metrics, _, _, _ = scheduler.run_policy(
61         events, tasks, capacity, rooms, room_names, "SLACK_GUARD_SHARED",
62         "capacity_main", name, scheduler.TRANSFER_MINUTES,
63     )
64     OUT.mkdir(parents=True, exist_ok=True)
65     pd.DataFrame([metrics]).to_csv(OUT / f"{name}.csv", index=False, encoding="utf-8-
sig")
66
67
68 def finalize() -> None:
69     total, completed, rate = historical_holdout_rate()
70     events = pd.read_csv(
71         ROOT / "supporting_materials" / "processed_data" / "unified_holdout" / "events
.csv",
72         encoding="utf-8-sig",
73     )
74     inpatient = events[(events["source"] == "住院") & events["evaluation_cohort"].
astype(str).str.lower().eq("true")]
75     c1_complete = int((inpatient["upper_minutes"] <= 48 * 60).sum())
76     rows = [
77         {"scenario": "C0_historical_report", "completed": completed, "total": total, "
rate": rate, "added_constraint": "observed order-to-report result"},
78         {"scenario": "C1_duration_only", "completed": c1_complete, "total": len(
inpatient), "rate": c1_complete / len(inpatient), "added_constraint": "problem
upper durations only"},
79     ]
80     labels = {
81         "C2_standard_hours": "8-12 and 13-17 work blocks; all rooms interchangeable;
22-doctor cap",
82         "C3_doctor_capacity": "training-derived weekday-slot doctor capacity",
83         "C4_project_capability": "reviewed project-machine compatibility without
preparation",
84     }
85     for name, description in labels.items():
86         metric = pd.read_csv(OUT / f"{name}.csv").iloc[0]
87         rows.append({
88             "scenario": name,
89             "completed": int(metric["inpatient_complete_48h"]),
90             "total": int(metric["inpatient_events"]),
91             "rate": float(metric["inpatient_48h_rate"]),
92             "added_constraint": description,
93         })
94     p2 = json.loads((ROOT / "supporting_materials" / "results" / "final_frozen" / "
p2_inpatient_only" / "summary.json").read_text(encoding="utf-8"))
95     rows.append({
96         "scenario": "C5_item_preparation",
97         "completed": p2["metrics"]["inpatient_complete_48h"],
98         "total": p2["metrics"]["inpatient_events"],

```

```

99         "rate": p2["metrics"]["inpatient_48h_rate"],
100         "added_constraint": "item-level fasting, bladder readiness and bedside
semantics",
101     })
102     selection = json.loads((ROOT / "supporting_materials" / "results" / "final_frozen"
/ "p3_joint" / "selection.json").read_text(encoding="utf-8"))
103     selected = selection["selected_metrics"]
104     rows.append({
105         "scenario": "C6_joint_background",
106         "completed": int(selected["inpatient_complete_48h"]),
107         "total": int(selected["inpatient_events"]),
108         "rate": float(selected["inpatient_48h_rate"]),
109         "added_constraint": f"outpatient/physical-exam epsilon target alpha={selection
['selected_alpha']}",
110     })
111     rows.append({
112         "scenario": "C7_formal_heuristic",
113         "completed": int(selected["inpatient_complete_48h"]),
114         "total": int(selected["inpatient_events"]),
115         "rate": float(selected["inpatient_48h_rate"]),
116         "added_constraint": "selected deterministic large-scale construction policy",
117     })
118     frame = pd.DataFrame(rows)
119     frame["change_from_previous_pp"] = frame["rate"].diff() * 100
120     frame.to_csv(OUT / "calibration_metrics.csv", index=False, encoding="utf-8-sig")
121     frame[["scenario", "change_from_previous_pp"]].to_csv(
122         OUT / "calibration_waterfall.csv", index=False, encoding="utf-8-sig"
123     )
124     columns = list(frame.columns)
125     markdown = [
126         "|" + "|".join(columns) + "|",
127         "|" + "|".join(["---"] * len(columns)) + "|",
128     ]
129     for row in frame.itertuples(index=False, name=None):
130         markdown.append("|" + "|".join(str(value).replace("|", "/") for value in row)
+ "|")
131     text = [
132         "# 历史现实校准报告", "",
133         "历史报告完成率与标准化排程率口径不同。下表在同一留出期事件定义上逐层加入题给时长、标准班
次、医生并发、设备能力、准备条件和背景负荷。", "",
134         "\n".join(markdown), "",
135         "C0使用真实报告提交时刻；C1-C7使用排程中最后任务结束作为报告完成代理。两者不能直接作因果比
较，分层结果用于定位模型下降来自哪一类约束。",
136     ]
137     (OUT / "CALIBRATION_REPORT.md").write_text("\n".join(text), encoding="utf-8")
138
139
140 def main() -> None:
141     parser = argparse.ArgumentParser()
142     parser.add_argument("--scenario", choices=["C2_standard_hours", "
C3_doctor_capacity", "C4_project_capability", "finalize"], required=True)
143     args = parser.parse_args()
144     if args.scenario == "finalize":
145         finalize()
146     else:
147         run_variant(args.scenario)
148
149
150 if __name__ == "__main__":
151     main()

```

F 独立验证与结果冻结程序

Listing 12 不调用调度内核的排程可行性验证

```
1  """Independent feasibility checks for the authoritative final schedule.
2
3  This verifier deliberately does not import the scheduler. It reconstructs
4  duration, compatibility, interval and capacity checks from the exported input
5  and result tables so that implementation errors are not silently shared.
6  """
7
8  from __future__ import annotations
9
10 import argparse
11 import json
12 import math
13 from pathlib import Path
14
15 import numpy as np
16 import pandas as pd
17
18
19 ROOT = Path(__file__).resolve().parents[2]
20 INPUT = ROOT / "supporting_materials" / "processed_data" / "unified_holdout"
21 DEFAULT_RESULT = ROOT / "supporting_materials" / "results" / "unified_schedule"
22 SLOT_MINUTES = 5
23 TRANSFER_MINUTES = 10
24
25
26 def as_bool(series: pd.Series) -> pd.Series:
27     if pd.api.types.is_bool_dtype(series):
28         return series.fillna(False)
29     return series.astype("string").str.lower().isin(["true", "1", "yes"])
30
31
32 def room_set(value: object) -> set[str]:
33     if pd.isna(value):
34         return set()
35     result: set[str] = set()
36     for part in str(value).split("|"):
37         if not part:
38             continue
39         number = float(part)
40         result.add(str(int(number)) if number.is_integer() else part)
41     return result
42
43
44 def slot_index(timestamp: pd.Timestamp) -> int | None:
45     minute = timestamp.hour * 60 + timestamp.minute
46     if 8 * 60 <= minute < 12 * 60:
47         return (minute - 8 * 60) // SLOT_MINUTES
48     if 13 * 60 <= minute < 17 * 60:
49         return 48 + (minute - 13 * 60) // SLOT_MINUTES
50     return None
51
52
53 def main() -> None:
```

```

54 parser = argparse.ArgumentParser()
55 parser.add_argument("--result-dir", type=Path, default=DEFAULT_RESULT)
56 parser.add_argument("--preparation-mode", choices=["item", "event"], default="item")
57 parser.add_argument("--transfer-minutes", type=int, default=TRANSFER_MINUTES)
58 args = parser.parse_args()
59 result_dir = args.result_dir
60 events = pd.read_csv(
61     INPUT / "events.csv",
62     encoding="utf-8-sig",
63     parse_dates=["order_dt", "planned_date", "base_release_dt", "bladder_ready_dt",
64     "release_dt", "deadline_dt"],
65 )
66 items = pd.read_csv(INPUT / "items.csv", encoding="utf-8-sig", low_memory=False)
67 schedule = pd.read_csv(
68     result_dir / "final_patient_task_schedule.csv",
69     encoding="utf-8-sig",
70     parse_dates=["start_dt", "end_dt", "release_dt", "deadline_dt"],
71     dtype={"room_id": "string"},
72     low_memory=False,
73 )
74 outcomes = pd.read_csv(
75     result_dir / "final_patient_outcomes.csv",
76     encoding="utf-8-sig",
77     parse_dates=["release_dt", "deadline_dt", "first_start_dt", "completion_dt"],
78 )
79 capacity = pd.read_csv(INPUT / "doctor_slot_capacity.csv", encoding="utf-8-sig")
80 for frame, columns in [
81     (events, ["special_case", "evaluation_cohort", "bedside", "fasting", "bladder"]),
82     (items, ["item_bedside", "item_fasting", "item_bladder", "event_bedside", "event_fasting", "event_bladder"]),
83     (schedule, ["bedside", "fasting", "bladder", "room_switch_from_previous"]),
84     (outcomes, ["complete_event", "deadline_met", "evaluation_cohort"]),
85 ]:
86     for column in columns:
87         frame[column] = as_bool(frame[column])
88
89 # P2 intentionally exports the inpatient cohort only, whereas P3 exports
90 # all three sources. Scope the independent verifier to the cohort named
91 # by the outcome table so that an inpatient-only result is not compared
92 # against unrelated outpatient and physical-exam events.
93 result_event_ids = set(outcomes["event_id"].astype(str))
94 events = events[events["event_id"].astype(str).isin(result_event_ids)].copy()
95 items = items[items["event_id"].astype(str).isin(result_event_ids)].copy()
96
97 checks: dict[str, dict[str, object]] = {}
98
99 def record(name: str, violation_count: int, detail: object = "") -> None:
100     checks[name] = {
101         "passed": int(violation_count) == 0,
102         "violation_count": int(violation_count),
103         "detail": detail,
104     }
105
106 record(
107     "unique_exported_task_key",
108     int(schedule.duplicated(["event_id", "item_index"]).sum()),
109     "each scheduled task must occur exactly once",
110 )

```

```

110 item_keys = items[["event_id", "item_index"]].drop_duplicates()
111 merged = schedule.merge(
112     items,
113     on=["event_id", "item_index"],
114     how="left",
115     suffixes=("_scheduled", "_input"),
116     indicator=True,
117     validate="many_to_one",
118 )
119 record("scheduled_task_exists_in_input", int(merged["_merge"].ne("both").sum()))
120 prefix = "item" if args.preparation_mode == "item" else "event"
121 merged["bedside_input"] = merged[f"{prefix}_bedside"]
122 merged["fasting_input"] = merged[f"{prefix}_fasting"]
123 merged["bladder_input"] = merged[f"{prefix}_bladder"]
124
125 event_meta = events.set_index("event_id")
126 merged["special_case_input"] = merged["event_id"].map(event_meta["special_case"])
127 merged["expected_duration_minutes"] = np.where(
128     merged["special_case_input"],
129     merged["duration_upper_min"],
130     merged["duration_nominal_min"],
131 )
132 merged["expected_duration_minutes"] = (
133     np.ceil(merged["expected_duration_minutes"].astype(float) / SLOT_MINUTES)
134     .clip(lower=1)
135     * SLOT_MINUTES
136 )
137 duration_bad = (
138     merged["duration_minutes"].ne(merged["expected_duration_minutes"])
139     | (merged["end_dt"] - merged["start_dt"]).dt.total_seconds().div(60).ne(
140         merged["duration_minutes"]
141     )
142 )
143 record("duration_matches_5pct_scenario", int(duration_bad.sum()))
144
145 compat_bad = [
146     room not in room_set(rooms)
147     for room, rooms in zip(
148         merged["room_id"].astype(str),
149         np.where(merged["bedside_input"], merged["bedside_compatible_rooms"],
150 merged["project_compatible_rooms"]),
151     )
152 ]
153 record("project_room_compatibility", int(sum(compat_bad)))
154 record(
155     "bedside_uses_room_7",
156     int((merged["bedside_input"] & merged["room_id"].ne("7")).sum()),
157 )
158
159 release_column = "base_release_dt" if args.preparation_mode == "item" else "
release_dt"
160 release = schedule["event_id"].map(event_meta[release_column])
161 deadline = schedule["event_id"].map(event_meta["deadline_dt"])
162 record("task_not_before_release", int(schedule["start_dt"].lt(release).sum()))
163 record("task_complete_by_event_deadline", int(schedule["end_dt"].gt(deadline).sum
164 ()))
165 bladder_ready = schedule["event_id"].map(pd.to_datetime(event_meta["
bladder_ready_dt"]))
166 record(
167     "bladder_task_after_readiness",

```

```

166         int((merged["bladder_input"] & merged["start_dt"].lt(bladder_ready.to_numpy())
167     ).sum()),
168     )
169     record(
170         "five_minute_grid",
171         int(
172             (
173                 schedule["start_dt"].dt.minute.mod(SLOT_MINUTES).ne(0)
174                 | schedule["end_dt"].dt.minute.mod(SLOT_MINUTES).ne(0)
175                 | schedule["start_dt"].dt.second.ne(0)
176                 | schedule["end_dt"].dt.second.ne(0)
177             ).sum()
178         ),
179     )
180     block_bad = 0
181     for row in schedule[["start_dt", "end_dt"]].itertuples(index=False):
182         morning = row.start_dt.hour < 12 and row.end_dt <= row.start_dt.normalize() +
183         pd.Timedelta(hours=12)
184         afternoon = row.start_dt.hour >= 13 and row.end_dt <= row.start_dt.normalize()
185         + pd.Timedelta(hours=17)
186         if row.start_dt.normalize() != row.end_dt.normalize() or not (morning or
187         afternoon):
188             block_bad += 1
189     record("inside_single_work_block", block_bad)
190
191     fasting_bad = merged["fasting_input"] & merged["end_dt"].gt(
192         merged["start_dt"].dt.normalize() + pd.Timedelta(hours=10)
193     )
194     record("fasting_complete_by_10", int(fasting_bad.sum()))
195
196     room_overlap = 0
197     for _, group in schedule.sort_values(["room_id", "start_dt"]).groupby("room_id",
198     sort=False):
199         room_overlap += int(group["start_dt"].iloc[1:].reset_index(drop=True).lt(
200             group["end_dt"].iloc[:-1].reset_index(drop=True)
201         ).sum())
202     record("no_room_overlap", room_overlap)
203
204     event_overlap = 0
205     transfer_bad = 0
206     sequence_bad = 0
207     for _, group in schedule.sort_values(["event_id", "sequence_position"]).groupby("
208     event_id", sort=False):
209         expected_sequence = np.arange(1, len(group) + 1)
210         sequence_bad += int((group["sequence_position"].to_numpy() !=
211         expected_sequence).sum())
212         if len(group) < 2:
213             continue
214         starts = group["start_dt"].iloc[1:].reset_index(drop=True)
215         ends = group["end_dt"].iloc[:-1].reset_index(drop=True)
216         prior_rooms = group["room_id"].iloc[:-1].reset_index(drop=True)
217         current_rooms = group["room_id"].iloc[1:].reset_index(drop=True)
218         switches = current_rooms.ne(prior_rooms)
219         event_overlap += int(starts.lt(ends).sum())
220         transfer_bad += int((switches & starts.lt(ends + pd.Timedelta(minutes=args.
221         transfer_minutes))).sum())
222     record("contiguous_event_sequence_numbers", sequence_bad)
223     record("no_same_patient_task_overlap", event_overlap)
224     record("cross_room_transfer_at_least_10min", transfer_bad)

```

```

218 # Reconstruct doctor concurrency by expanding each scheduled interval to
219 # its occupied 5-minute starts, then compare with weekday-slot capacity.
220 occupied_parts: list[pd.DataFrame] = []
221 for slots, group in schedule.groupby(schedule["duration_minutes"].floordiv(
222     SLOT_MINUTES).astype(int)):
223     base = group[["start_dt"]].reset_index(drop=True)
224     repeated = pd.DataFrame(
225         {
226             "slot_dt": np.concatenate(
227                 [
228                     (base["start_dt"] + pd.Timedelta(minutes=offset * SLOT_MINUTES
229                     for offset in range(int(slots)))
230                 ]
231             )
232         }
233     )
234     occupied_parts.append(repeated)
235 occupied = pd.concat(occupied_parts, ignore_index=True)
236 load = occupied.groupby("slot_dt", as_index=False).size().rename(columns={"size":
237     "doctor_load"})
238 load["weekday"] = load["slot_dt"].dt.dayofweek
239 load["slot_index"] = load["slot_dt"].map(slot_index)
240 cap = capacity[["weekday", "slot_index", "capacity_main"]]
241 load = load.merge(cap, on=["weekday", "slot_index"], how="left", validate="
242     many_to_one")
243 record("doctor_capacity_defined", int(load["capacity_main"].isna().sum()))
244 record(
245     "doctor_concurrency_within_capacity",
246     int(load["doctor_load"].gt(load["capacity_main"]).sum()),
247     {
248         "max_observed_load": int(load["doctor_load"].max()),
249         "max_capacity": int(load["capacity_main"].max()),
250     },
251 )
252 scheduled_count = schedule.groupby("event_id").size()
253 expected_count = events.set_index("event_id")["service_project_count"]
254 computed_complete = scheduled_count.reindex(expected_count.index, fill_value=0).eq
255 (expected_count)
256 outcome_complete = outcomes.set_index("event_id")["complete_event"].reindex(
257     expected_count.index)
258 record("outcome_complete_flag_consistency", int(computed_complete.ne(
259     outcome_complete).sum()))
260 computed_deadline = computed_complete.copy()
261 completion = schedule.groupby("event_id")["end_dt"].max().reindex(expected_count.
262     index)
263 computed_deadline &= completion.le(events.set_index("event_id")["deadline_dt"])
264 outcome_deadline = outcomes.set_index("event_id")["deadline_met"].reindex(
265     expected_count.index)
266 record("outcome_deadline_flag_consistency", int(computed_deadline.ne(
267     outcome_deadline).sum()))
268
269 passed = all(entry["passed"] for entry in checks.values())
270 report = {
271     "passed": passed,
272     "schedule_rows": int(len(schedule)),
273     "scheduled_events": int(schedule["event_id"].nunique()),
274     "input_events": int(len(events)),

```

```

268     "checks": checks,
269 }
270 result_dir.mkdir(parents=True, exist_ok=True)
271 (result_dir / "independent_verification.json").write_text(
272     json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
273 )
274 print(json.dumps(report, ensure_ascii=False, indent=2))
275 if not passed:
276     raise SystemExit(1)
277
278
279 if __name__ == "__main__":
280     main()

```

Listing 13 技术审计、唯一结果与冻结报告生成

```

1  """Assemble audit outputs, freeze manifest and the pre-freeze technical report."""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  import shutil
8  from pathlib import Path
9
10 import pandas as pd
11
12
13 ROOT = Path(__file__).resolve().parents[2]
14 RESULTS = ROOT / "supporting_materials" / "results"
15 FROZEN = RESULTS / "final_frozen"
16 VALIDATION = RESULTS / "validation"
17
18
19 def load_json(path: Path) -> dict:
20     return json.loads(path.read_text(encoding="utf-8"))
21
22
23 def sha256(path: Path) -> str:
24     digest = hashlib.sha256()
25     with path.open("rb") as handle:
26         for block in iter(lambda: handle.read(1024 * 1024), b""):
27             digest.update(block)
28     return digest.hexdigest()
29
30
31 def copy_selected_p3() -> float:
32     selection = load_json(FROZEN / "p3_joint" / "selection.json")
33     alpha = float(selection["selected_alpha"])
34     source = FROZEN / "p3_shards" / f"alpha_{alpha:.2f}"
35     target = FROZEN / "p3_joint"
36     required = [
37         "final_patient_task_schedule.csv", "final_patient_outcomes.csv",
38         "daily_service_metrics.csv", "room_utilization_daily.csv",
39         "inpatient_recommended_schedule.csv", "quota_diagnostics.json",
40     ]
41     for name in required:
42         shutil.copy2(source / name, target / name)
43     return alpha
44

```

```

45
46 def sensitivity_outputs() -> tuple[pd.DataFrame, dict, pd.DataFrame]:
47     shard_root = FROZEN / "sensitivity_shards"
48     frames = []
49     for path in sorted(shard_root.glob("*/metrics.csv")):
50         frame = pd.read_csv(path)
51         frame.insert(0, "audit_scenario", path.parent.name)
52         frames.append(frame)
53     metrics = pd.concat(frames, ignore_index=True)
54     metrics.to_csv(FROZEN / "sensitivity_metrics.csv", index=False, encoding="utf-8-
sig")
55     seed_rows = metrics[metrics["audit_scenario"].str.startswith("seed_").copy()
56     seed_summary = {
57         "seed_count": len(seed_rows),
58         "inpatient_48h_rate": {
59             "mean": float(seed_rows["inpatient_48h_rate"].mean()),
60             "std": float(seed_rows["inpatient_48h_rate"].std(ddof=1)),
61             "min": float(seed_rows["inpatient_48h_rate"].min()),
62             "max": float(seed_rows["inpatient_48h_rate"].max()),
63         },
64         "background_rate": {
65             "mean": float(seed_rows["background_on_time_rate"].mean()),
66             "std": float(seed_rows["background_on_time_rate"].std(ddof=1)),
67             "min": float(seed_rows["background_on_time_rate"].min()),
68             "max": float(seed_rows["background_on_time_rate"].max()),
69         },
70         "waiting_p90_hours": seed_rows["inpatient_wait_p90_hours_conditional"].tolist
71     ),
72         "scheduled_minutes": seed_rows["scheduled_minutes"].tolist(),
73     }
74     (FROZEN / "special_seed_summary.json").write_text(
75         json.dumps(seed_summary, ensure_ascii=False, indent=2), encoding="utf-8"
76     )
77     prep = metrics[metrics["audit_scenario"].isin(["preparation_item", "
preparation_event"])]
78     prep.to_csv(FROZEN / "preparation_granularity_sensitivity.csv", index=False,
encoding="utf-8-sig")
79     return metrics, seed_summary, prep
80
81 def main() -> None:
82     alpha = copy_selected_p3()
83     sensitivity, seed_summary, prep = sensitivity_outputs()
84     p2 = load_json(FROZEN / "p2_inpatient_only" / "summary.json")
85     p3 = load_json(FROZEN / "p3_joint" / "selection.json")
86     semantic = load_json(VALIDATION / "input_semantic_verification.json")
87     p2_verify = load_json(FROZEN / "p2_inpatient_only" / "independent_verification.
json")
88     p3_verify = load_json(FROZEN / "p3_joint" / "independent_verification.json")
89     exact = pd.read_csv(RESULTS / "exact_benchmark" / "exact_vs_heuristic_metrics.csv"
)
90     calibration = pd.read_csv(RESULTS / "calibration" / "calibration_metrics.csv")
91     fallback = pd.read_csv(RESULTS / "capability" / "LEGACY_50_FALLBACK_PROJECT_AUDIT.
csv")
92     bedside = pd.read_csv(RESULTS / "capability" / "bedside_project_capability_audit.
csv")
93     peak = pd.read_csv(RESULTS / "p1" / "peak_flat_summary.csv")
94
95     freeze_checks = {
96         "train_holdout_no_leakage": bool(peak["threshold_data_role"].eq("training_only

```



```

153 )
154
155 original_ip = 0.6396546608073929
156 original_bg = 0.5969951898048852
157 p3m = p3["selected_metrics"]
158 report = f"""# 模型与结果冻结前技术审计报告
159
160 ## 1. 最终结论
161
162 技术审计后的冻结判断为：`MODEL_AND_RESULTS_FROZEN = {'TRUE' if frozen else 'FALSE'}`。冻结
    检查逐项结果见本报告第14节和 `FINAL_RESULT_MANIFEST.json`。
163
164 ## 2. 原始模型存在的问题
165
166 原模型存在四项会改变结果的硬问题：50个项目采用整族能力回退；床旁标记直接覆盖项目专项能力；问题三把模
    型自身约59.70%的背景完成率写成“满足需求”；问题一峰平阈值使用留出期分位数。另发现项目单元格中的
    显式 `][` 拼接没有拆成独立任务。上述问题均已在上游代码修复并重跑。
167
168 ## 3. 设备能力整改
169
170 50个原回退项目已逐项列入 `LEGACY_50_FALLBACK_PROJECT_AUDIT.csv`，A/B/C级映射进入硬能力集合，
    D级项目排除；4个旧复合单元已按源字段边界拆分。整族回退已删除。当前目录D级项目 {semantic['
    semantic_counts']['evidence_levels'].get('D', 0)} 个。床旁审计共 {len(bedside)} 个项目
    一来源组合；7号机只有在项目证据和床旁地点证据同时成立时才进入可行集。
171
172 ## 4. P1整改
173
174 平峰、高峰、压力阈值和代表日全部改由训练期确定；留出期只统计阈值外推表现。日历岭回归保持 `log(1+N)
    ` 拟合并经 `expm1` 返回原尺度；滚动基准只使用预测时点之前的已观测数据。参数来源逐项见 `
    TRAIN_HOLDOUT_PROVENANCE_AUDIT.md`。
175
176 ## 5. P2结果
177
178 纯住院场景共 {p2['metrics']['inpatient_events'],} 个评价事件，48小时完整完成 {p2['metrics'
    ']['inpatient_complete_48h'],} 个，完成率 {p2['metrics']['inpatient_48h_rate
    ']:.4%}；选用策略为 {p2['selected_policy']}。P50/P90/P95等待分别为 {p2['metrics']['
    inpatient_wait_p50_hours_conditional']:.2f}/{p2['metrics']['
    inpatient_wait_p90_hours_conditional']:.2f}/{p2['metrics']['
    inpatient_wait_p95_hours_conditional']:.2f} 小时。
179
180 ## 6. P3结果
181
182 联合场景按全留出期背景计划日代理完成率构造 $\epsilon$ 约束，日级配额短缺另作异质性诊断；自动选择的运行点为
    alpha={alpha:.2f}。该点住院48小时完整完成 {int(p3m['inpatient_complete_48h']):,} 个，
    完成率 {p3m['inpatient_48h_rate']:.4%}；门诊与体检计划日代理完成率 {p3m['
    background_on_time_rate']:.4%}。相对原63.95%/59.70%，分别变化 {(p3m['
    inpatient_48h_rate']-original_ip)*100:+.2f} 和 {(p3m['background_on_time_rate']-
    original_bg)*100:+.2f} 个百分点。该背景率只表示回顾性计划日压力场景，不称为全部需求已满足。
183
184 ## 7. Pareto分析
185
186 `p3_joint/pareto_metrics.csv` 给出背景服务水平与住院48小时率的可行前沿。主点按“到可达理想点的
    归一化欧氏距离最小”确定，不用当前结果反向设阈值。
187
188 ## 8. 历史现实校准
189
190 `calibration/CALIBRATION_REPORT.md` 从历史真实报告完成率开始，逐层加入题给时长、标准班次、医生
    并发、项目能力、准备条件和背景负荷。历史率与模型率口径不同；分解结果用于识别下降来源，而不把差值
    解释为模型直接造成的临床损失。
191

```

```

192 ## 9. 精确求解验证
193
194 CP-SAT代表性小窗口共 {len(exact)} 个，启发式相对精确解的最大差距为 {exact['
    heuristic_gap_to_exact'].max():.4%}，相对最好界最大差距为 {exact['
    heuristic_gap_to_bound'].max():.4%}。全年结果因此仍称为可行近似解，不宣称全局最优。
195
196 ## 10. 敏感性
197
198 已完成医生容量、A/B能力边界、跨室5/15分钟、憋尿45/90分钟、项目级/事件级准备及5个特殊病例seed。多
    seed住院率均值/标准差为 {seed_summary['inpatient_48h_rate']['mean']:.4%}/{
    seed_summary['inpatient_48h_rate']['std']:.4%}。
199
200 ## 11. 模型假设合理性
201
202 A类为题面直接给定（22台设备、33名医生、时长区间、5%特殊病例、48小时口径）；B类为训练期标定（需求、
    医生时隙容量、项目目录）；C类为数据不能唯一确定但已做敏感性（憋尿60分钟、跨室10分钟、报告完成代
    理、计划日代理）；原整族回退和床旁绕过专项能力属于D类，已删除或修正。
203
204 ## 12. 最终冻结参数
205
206 时隙5分钟；工作时段08:00-12:00、13:00-17:00；跨室10分钟；憋尿60分钟；准备约束按项目粒度；设备能
    力为表1 A/B/C级逐项目映射，D级不进入主情景；医生为训练期星期×5分钟总并发代理；特殊病例比例5%，
    固定主seed 20260824；事件键为来源+患者ID+精确开单时刻；住院分母保留全部评价事件，最后任务结束
    作为报告完成代理。
207
208 ## 13. 最终 authoritative 文件
209
210 唯一权威根目录：`supporting_materials/results/final_frozen/`。P2、P3、敏感性和 manifest 均
    在该目录；精确对照和校准分别位于 `supporting_materials/results/exact_benchmark/` 与 `
    supporting_materials/results/calibration/`。
211
212 ## 14. 最终冻结判断
213
214 {chr(10).join(f'- [{x} if value else " "] {key}' for key, value in freeze_checks.
    items())}
215
216 `MODEL_AND_RESULTS_FROZEN = {'TRUE' if frozen else 'FALSE'}`
217 """
218     (ROOT / "PRE_FREEZE_MODEL_AUDIT_REPORT.md").write_text(report, encoding="utf-8")
219
220     validation_report = f"""# FINAL_MODEL_VALIDATION_REPORT
221
222 ## Level 1 Raw-data semantic audit
223
224 原始时间范围、字段、事件键、重复记录和报告延迟由 `tables/audit/data_audit_metrics.json` 与 `
    bundle_key_sensitivity.json` 核查。
225
226 ## Level 2 Model-input semantic audit
227
228 输入语义验证结果：{'通过' if semantic['passed'] else '未通过'}。设备能力、50项复核、床旁交集、
    训练/留出边界、医生资质边界和报告代理见 `input_semantic_verification.json`。
229
230 ## Level 3 Schedule feasibility audit
231
232 P2独立检查：{'通过' if p2_verify['passed'] else '未通过'}；P3独立检查：{'通过' if p3_verify
    ['passed'] else '未通过'}。检查覆盖任务唯一性、时长、项目房间兼容、床旁、释放/截止、工作时
    段、空腹、憋尿准备、设备互斥、患者互斥、跨室转运、医生并发和结果聚合。
233
234 最终结论：{'三层均通过。' if frozen else '仍有未通过项，见manifest。'}
235 """

```

```

236 (VALIDATION / "FINAL_MODEL_VALIDATION_REPORT.md").write_text(validation_report,
encoding="utf-8")
237 print(json.dumps({"MODEL_AND_RESULTS_FROZEN": frozen, "checks": freeze_checks},
ensure_ascii=False, indent=2))
238 if not frozen:
239     raise SystemExit(1)
240
241
242 if __name__ == "__main__":
243     main()

```

G 论文图表生成程序

Listing 14 论文十幅矢量图生成

```

1  """Create vector figures from the frozen result chain."""
2  from __future__ import annotations
3  from pathlib import Path
4  import json
5  import matplotlib.dates as mdates
6  import matplotlib.pyplot as plt
7  from matplotlib import font_manager
8  from matplotlib.patches import Patch
9  import numpy as np
10 import pandas as pd
11
12 ROOT = Path(__file__).resolve().parents[2]
13 P1 = ROOT / "supporting_materials/results/p1"
14 INPUT = ROOT / "supporting_materials/processed_data/unified_holdout"
15 FROZEN = ROOT / "supporting_materials/results/final_frozen"
16 CAL = ROOT / "supporting_materials/results/calibration"
17 OUT = ROOT / "supporting_materials/figures/paper_final"
18 BLUE, ORANGE, GREEN, RED, PURPLE, GRAY = "#276FBF", "#F28E2B", "#3A9D5D", "#C93C3C", "
    #7A5195", "#6B7280"
19 LIGHT, GRID = "#E8EEF5", "#D7DEE8"
20
21 def configure() -> None:
22     available = {font.name for font in font_manager.fontManager.ttflist}
23     family = next((name for name in ["Microsoft YaHei", "SimHei", "Noto Sans CJK SC"]
if name in available), "DejaVu Sans")
24     plt.rcParams.update({"font.family": family, "axes.unicode_minus": False, "axes.
spines.top": False, "axes.spines.right": False, "figure.dpi": 150, "savefig.dpi":
300})
25     OUT.mkdir(parents=True, exist_ok=True)
26
27 def save(fig: plt.Figure, name: str) -> None:
28     fig.savefig(OUT / f"{name}.pdf", bbox_inches="tight")
29     plt.close(fig)
30
31 def monthly_demand() -> None:
32     frame = pd.read_csv(P1 / "monthly_trends.csv", parse_dates=["period"])
33     frame = frame[frame["basis"].eq("开单日")]
34     colors = {"住院": BLUE, "门诊": ORANGE, "体检": GREEN}
35     fig, ax = plt.subplots(figsize=(10.2, 4.3))
36     for source, group in frame.groupby("source", sort=False):
37         ax.plot(group["period"], group["mean_daily_episodes"], lw=1.7, color=colors[
source], label=source)
38     split = pd.Timestamp("2024-04-01")

```

```

39 ax.axvspan(split, pd.Timestamp("2025-04-01"), color=LIGHT, alpha=.9)
40 ax.axvline(split, color=GRAY, ls="--", lw=1)
41 ax.text(split + pd.Timedelta(days=10), ax.get_ylim()[1]*.92, "时间顺序留出期", color=GRAY)
42 ax.set(title="三类患者月度日均开单事件与留出边界", xlabel="月份", ylabel="日均事件数")
43 ax.grid(axis="y", color=GRID); ax.legend(frameon=False, ncol=3)
44 fig.tight_layout(); save(fig, "fig01_monthly_demand")
45
46 def doctor_capacity() -> None:
47     frame = pd.read_csv(INPUT / "doctor_slot_capacity.csv")
48     fig, axes = plt.subplots(2, 1, figsize=(10.2, 5.6), sharex=True)
49     labels = ["周一", "周二", "周三", "周四", "周五", "周六", "周日"]
50     for ax, column, title in zip(axes, ["capacity_main", "capacity_low"], ["训练期中位并发（主情景）", "训练期下四分位并发（偏紧情景）"]):
51         for weekday, group in frame.groupby("weekday"):
52             ax.plot(np.arange(len(group)), group[column], lw=1.2, label=labels[int(weekday)])
53             ax.set_ylabel("并发医生数"); ax.set_title(title, loc="left", fontsize=11); ax.grid(axis="y", color=GRID)
54             axes[-1].set_xticks([0,12,24,36,47,48,60,72,84,95], ["08:00", "09:00", "10:00", "11:00", "11:55", "13:00", "14:00", "15:00", "16:00", "16:55"])
55             axes[0].legend(ncol=7, frameon=False, loc="upper center", bbox_to_anchor=(.5,1.32))
56         fig.suptitle("训练期星期一5分钟共享医生容量", y=1.02, fontweight="bold")
57     fig.tight_layout(); save(fig, "fig02_doctor_capacity")
58
59 def capability_coverage() -> None:
60     summary = pd.read_csv(INPUT / "project_machine_summary.csv", encoding="utf-8-sig")
61     items = pd.read_csv(INPUT / "items.csv", encoding="utf-8-sig", low_memory=False)
62     events = pd.read_csv(INPUT / "events.csv", encoding="utf-8-sig", usecols=["event_id", "source"])
63     frame = items[["event_id", "project_norm"]].merge(events, on="event_id").merge(summary[["project_norm", "evidence_level"]], on="project_norm")
64     table = frame.groupby(["source", "evidence_level"]).size().unstack(fill_value=0).reindex(["住院", "门诊", "体检"])
65     table = table.div(table.sum(axis=1), axis=0)
66     fig, ax = plt.subplots(figsize=(8.8, 3.7)); left=np.zeros(len(table))
67     display = {"A": "直接匹配", "B": "语义匹配", "C": "明确广义", "D": "未确认"}
68     for level, color in zip("ABCD", [BLUE, GREEN, ORANGE, RED]):
69         values=table.get(level, pd.Series(0, index=table.index)).to_numpy()
70         bars=ax.barh(table.index, values, left=left, color=color, height=.56, label=display[level])
71         for bar, val, start in zip(bars, values, left):
72             if val>.035: ax.text(start+val/2, bar.get_y()+bar.get_height()/2, f"{val:.1%}", ha="center", va="center", color="white", fontsize=9)
73             left += values
74     ax.set_xlim(0,1); ax.xaxis.set_major_formatter(lambda x, _:f"{x:.0%}"); ax.set_xlabel("项目任务占比"); ax.set_title("项目—设备匹配方式覆盖")
75     ax.legend(ncol=4, frameon=False, loc="lower center", bbox_to_anchor=(.5, -.38))
76     fig.tight_layout(); save(fig, "fig03_capability_coverage")
77
78 def p2_policy() -> None:
79     frame = pd.read_csv(FROZEN / "p2_inpatient_only/policy_metrics.csv")
80     labels=["共享FCFS", "最小松弛度保护"]; x=np.arange(2)
81     fig, axes=plt.subplots(1,2,figsize=(9.2,3.8))
82     axes[0].bar(x, frame["inpatient_48h_rate"], color=[BLUE, ORANGE], width=.56)
83     axes[0].set_xticks(x, labels); axes[0].set_ylim(.84, .86); axes[0].yaxis.set_major_formatter(lambda v, _:f"{v:.1%}"); axes[0].set_title("住院48小时完整完成率")
84     axes[1].bar(x, frame["inpatient_wait_p50_hours_conditional"], color=[BLUE, ORANGE],

```

```

width=.56,label="P50")
85 axes[1].bar(x,frame["inpatient_wait_p90_hours_conditional"]-frame["
inpatient_wait_p50_hours_conditional"],bottom=frame["
inpatient_wait_p50_hours_conditional"],color=[LIGHT,LIGHT],width=.56,label="P50至
P90")
86 axes[1].set_xticks(x,labels); axes[1].set_ylabel("小时"); axes[1].set_title("条件等
待时间"); axes[1].legend(frameon=False)
87 for ax in axes: ax.grid(axis="y",color=GRID,zorder=0)
88 fig.tight_layout(); save(fig, "fig04_p2_policy")
89
90 def pareto() -> None:
91     frame=pd.read_csv(FROZEN/"p3_joint/pareto_metrics.csv")
92     selected=float(json.loads((FROZEN/"p3_joint/selection.json").read_text(encoding="
utf-8"))["selected_alpha"])
93     fig,ax=plt.subplots(figsize=(7.4,5.0))
94     ax.plot(frame["background_on_time_rate"],frame["inpatient_48h_rate"],"-o",color=
BLUE,lw=1.8,ms=6)
95     for row in frame.itertuples(index=False):
96         ax.annotate(f"$\\alpha$={row.background_target_alpha:.2f}",(row.
background_on_time_rate,row.inpatient_48h_rate),xytext=(5,6),textcoords="offset
points",fontsize=8)
97     mark=frame[np.isclose(frame["background_target_alpha"],selected)].iloc[0]
98     ax.scatter(mark["background_on_time_rate"],mark["inpatient_48h_rate"],s=180,color=
RED,edgecolor="white",zorder=5,label="推荐折中点")
99     ax.xaxis.set_major_formatter(lambda v,_:f"{v:.0%}"); ax.yaxis.set_major_formatter(
lambda v,_:f"{v:.0%}")
100 ax.set(xlabel="门诊/体检计划日代理完成率",ylabel="住院48小时完整完成率",title="共享资源下
的住院—背景服务Pareto前沿")
101 ax.grid(color=GRID); ax.legend(frameon=False)
102 fig.tight_layout(); save(fig, "fig05_pareto")
103
104 def calibration() -> None:
105     frame=pd.read_csv(CAL/"calibration_metrics.csv")
106     labels=["历史报告","仅时长","标准班次","医生并发","设备能力","项目准备","背景竞争","正式策
略"]
107     colors=[GRAY,GREEN,GREEN,GREEN,ORANGE,ORANGE,RED,BLUE]
108     fig,ax=plt.subplots(figsize=(10.0,4.2))
109     bars=ax.bar(np.arange(len(frame)),frame["rate"],color=colors,width=.68)
110     ax.set_xticks(np.arange(len(frame)),labels,rotation=18,ha="right"); ax.set_ylim
(.4,1.05); ax.yaxis.set_major_formatter(lambda v,_:f"{v:.0%}")
111     for bar,val in zip(bars,frame["rate"]): ax.text(bar.get_x()+bar.get_width()/2,val
+.015,f"{val:.1%}",ha="center",fontsize=8)
112     ax.set_ylabel("完成率"); ax.set_title("从历史口径到正式联合排程的逐层现实校准"); ax.grid
(axis="y",color=GRID)
113     fig.tight_layout(); save(fig,"fig06_calibration")
114
115 def sensitivity() -> None:
116     frame=pd.read_csv(FROZEN/"sensitivity_metrics.csv").set_index("audit_scenario")
117     base=frame.loc["preparation_item"]
118     order=["preparation_event","doctor_low","capability_AB","bladder45","bladder90","
transfer5","transfer15"]
119     names=["事件级准备","医生容量下四分位","仅直接/语义匹配","憋尿45分钟","憋尿90分钟","转运5分
钟","转运15分钟"]
120     ip=(frame.loc[order,"inpatient_48h_rate"]-base["inpatient_48h_rate"])*100
121     bg=(frame.loc[order,"background_on_time_rate"]-base["background_on_time_rate"]
*100
122     y=np.arange(len(order)); fig,ax=plt.subplots(figsize=(8.8,5.1))
123     ax.axvline(0,color="#263238",lw=1); ax.hlines(y,bg,ip,color="#B7C2CF",lw=2)
124     ax.scatter(ip,y,color=BLUE,s=65,label="住院率"); ax.scatter(bg,y,color=ORANGE,s
=65,label="背景率")

```

```

125 ax.set_yticks(y,names); ax.invert_yaxis(); ax.set_xlabel("相对主情景变化（百分点）");
    ax.set_title("关键边界敏感性")
126 ax.grid(axis="x",color=GRID); ax.legend(frameon=False,ncol=2)
127 fig.tight_layout(); save(fig,"fig07_sensitivity")
128
129 def daily_performance() -> None:
130     daily=pd.read_csv(FROZEN/"p3_joint/daily_service_metrics.csv")
131     daily["metric_date"]=pd.to_datetime(daily["metric_date"],format="mixed").dt.
        normalize()
132     ip=daily[daily["source"].eq("住院")].sort_values("metric_date")
133     bg=daily[~daily["source"].eq("住院")].groupby("metric_date",as_index=False).agg(
        events=("events","sum"),deadline_met=("deadline_met","sum"))
134     bg["deadline_rate"]=bg["deadline_met"]/bg["events"]
135     fig,ax=plt.subplots(figsize=(10.2,4.3))
136     ax.plot(ip["metric_date"],ip["deadline_rate"].rolling(14,min_periods=1).mean(),
        color=BLUE,label="住院48小时率")
137     ax.plot(bg["metric_date"],bg["deadline_rate"].rolling(14,min_periods=1).mean(),
        color=ORANGE,label="背景计划日率")
138     ax.yaxis.set_major_formatter(lambda v,_:f"{v:.0%}"); ax.xaxis.set_major_locator(
        mdates.MonthLocator(interval=2)); ax.xaxis.set_major_formatter(mdates.
        DateFormatter("%Y-%m"))
139     ax.set_xlabel="留出日期",ylabel="14日滚动完成率",title="推荐联合策略的日级稳定性"; ax
        .grid(axis="y",color=GRID); ax.legend(frameon=False,ncol=2)
140     fig.tight_layout(); save(fig,"fig08_daily_performance")
141
142 def gantt() -> None:
143     schedule=pd.read_csv(FROZEN/"p3_joint/final_patient_task_schedule.csv",parse_dates
        =["start_dt","end_dt"],dtype={"room_id":"string"},low_memory=False)
144     schedule["date"]=schedule["start_dt"].dt.normalize()
145     target=schedule.groupby("date")["duration_minutes"].sum().idxmax()
146     day=schedule[schedule["date"].eq(target)].copy()
147     room_order=day.groupby(["room_id","room_name"])["duration_minutes"].sum().
        sort_values().index.tolist()
148     ymap={room:i for i,(room,_) in enumerate(room_order)}; colors={"住院":BLUE,"门诊":
        ORANGE,"体检":GREEN}
149     fig,ax=plt.subplots(figsize=(10.5,6.8)); origin=target+pd.Timedelta(hours=8)
150     for row in day.itertuples(index=False):
151         left=(row.start_dt-origin).total_seconds()/3600; width=(row.end_dt-row.
            start_dt).total_seconds()/3600
152         ax.barh(ymap[str(row.room_id)],width,left=left,height=.7,color=colors[row.
            source],edgecolor="white",linewidth=.2)
153         ax.axvspan(4,5,color="#F0F2F5"); ax.set_yticks(range(len(room_order)),[f"{name}({
            room})" for room,name in room_order],fontsize=8)
154         ax.set_xticks(range(10),[f"{8+h:02d}:00" for h in range(10)]); ax.set_xlim(0,9);
            ax.set_xlabel("时刻"); ax.set_title(f"高负荷代表日 {target.date()} 的设备排程")
155         ax.grid(axis="x",color=GRID); ax.legend(handles=[Patch(color=colors[k],label=k)
            for k in colors],frameon=False,ncol=3)
156         fig.tight_layout(); save(fig,"fig09_representative_gantt")
157
158 def modeling_flow() -> None:
159     fig,ax=plt.subplots(figsize=(10.4,3.6)); ax.axis("off")
160     boxes=[("数据审计\n事件与边界",LIGHT),("P1参数层\n需求·时长·能力", "#DDE9F7"),("P2纯住
        院\n患者级排程", "#DFF1E5"),("P3联合\n约束与Pareto", "#FCE8D2"),("三层验证\n冻结结果", "#
        E9E0F0")]
161     xs=np.linspace(.1,.9,len(boxes))
162     for i,((label,color),x) in enumerate(zip(boxes,xs)):
163         ax.text(x,.58,label,ha="center",va="center",fontsize=10,bbox=dict(boxstyle="
            round,pad=.6",facecolor=color,edgecolor="#607D8B"),transform=ax.transAxes)
164         if i<len(boxes)-1: ax.annotate("",xy=(xs[i+1]-.09,.58),xytext=(x+.09,.58),
            arrowprops=dict(arrowstyle="->",color="#607D8B",lw=1.4),xycoords=ax.transAxes)

```

```

165 ax.text(.5, .16, "训练期冻结参数（至2024-03-31） → 留出期仅评价（2024-04-01至2025-03-31）", ha="center", color=GRAY, transform=ax.transAxes)
166 ax.set_title("统一患者—项目—设备—医生排程建模链", pad=18)
167 fig.tight_layout(); save(fig, "fig10_modeling_flow")
168
169 def main() -> None:
170     configure(); monthly_demand(); doctor_capacity(); capability_coverage(); p2_policy(); pareto(); calibration(); sensitivity(); daily_performance(); gantt(); modeling_flow()
171     print("created 10 vector PDF figures")
172
173 if __name__ == "__main__": main()

```

Listing 15 论文十个数据表生成

```

1  """Generate authoritative LaTeX tables for the rebuilt paper."""
2  from __future__ import annotations
3  import json
4  from pathlib import Path
5  import pandas as pd
6
7  ROOT = Path(__file__).resolve().parents[2]
8  P1 = ROOT / "supporting_materials/results/p1"
9  INPUT = ROOT / "supporting_materials/processed_data/unified_holdout"
10 FROZEN = ROOT / "supporting_materials/results/final_frozen"
11 EXACT = ROOT / "supporting_materials/results/exact_benchmark"
12 CAL = ROOT / "supporting_materials/results/calibration"
13 OUT = ROOT / "paper/tables"
14
15 def esc(value: object) -> str:
16     text = str(value)
17     for old, new in [("\\", r"\textbackslash{}"), ("&", r"\&"), ("% ", r"% "), ("_", r"\_"), ("#", r"\#")]:
18         text = text.replace(old, new)
19     return text
20
21 def pct(value: float) -> str: return f"{100 * float(value):.2f}%\n"
22 def write(name: str, lines: list[str]) -> None: (OUT / name).write_text("\n".join(lines) + "\n", encoding="utf-8")
23
24 def data_scope() -> None:
25     events = pd.read_csv(INPUT / "events.csv", encoding="utf-8-sig")
26     items = pd.read_csv(INPUT / "items.csv", encoding="utf-8-sig", low_memory=False)
27     eval_flag = events["evaluation_cohort"].astype(str).str.lower().eq("true")
28     rows = []
29     for source in ["住院", "门诊", "体检"]:
30         ids = set(events.loc[events["source"].eq(source), "event_id"])
31         rows.append((source, len(ids), int(items["event_id"].isin(ids).sum()), int((eval_flag & events["source"].eq(source)).sum())))
32     lines = [r"\begin{tabular}{lrrr}", r"\toprule", "来源 & 连续模拟事件 & 项目任务 & 留出期评价事件 \\\\", r"\midrule"]
33     lines += [f"{s} & {e:,} & {i:,} & {v:,} \\\\" for s, e, i, v in rows]
34     lines += [r"\midrule", f"合计 & {len(events):,} & {len(items):,} & {int(eval_flag.sum()):,} \\\\", r"\bottomrule", r"\end{tabular}"]
35     write("data_scope.tex", lines)
36
37 def duration_parameters() -> None:
38     frame = pd.read_csv(P1 / "service_duration_parameters.csv")
39     frame = frame[~frame["category"].astype(str).str.startswith("非服务")]
40     lines = [r"\begin{tabular}{lrrrr}", r"\toprule", "项目族 & 下界/min & 名义/min & 上

```

```

41 界/min & 训练代理数 \\\\", r"\\midrule"]
42 for row in frame.itertuples(index=False):
43     n = "--" if pd.isna(row.proxy_n) else f"{int(row.proxy_n):,}"
44     lines.append(f"{esc(row.category)} & {row.duration_lower_min:g} & {row.
45 duration_nominal_min:g} & {row.duration_upper_min:g} & {n} \\\\")
46     lines += [r"\\bottomrule", r"\\end{tabular}"]
47     write("duration_parameters.tex", lines)
48
49 def capability_evidence() -> None:
50     summary = pd.read_csv(INPUT / "project_machine_summary.csv", encoding="utf-8-sig")
51     items = pd.read_csv(INPUT / "items.csv", encoding="utf-8-sig", low_memory=False)
52     joined = items[["project_norm"]].merge(summary[["project_norm", "evidence_level"]
53 ], on="project_norm", how="left", validate="many_to_one")
54     pc, ic = summary["evidence_level"].value_counts(), joined["evidence_level"].
55 value_counts()
56     labels = {"A": "直接匹配", "B": "语义匹配", "C": "明确广义", "D": "未确认"}
57     desc = {"A": "表1项目文本直接命中", "B": "同义、包含或复合部位交集", "C": "表1中的广义项目描
58 述覆盖", "D": "表1无法确认, 主模型排除"}
59     lines = [r"\\begin{tabularx}{\\textwidth}{lrrX}", r"\\toprule", "匹配方式 & 项目数 & 项
60 目任务数 & 使用规则 \\\\", r"\\midrule"]
61     for level in "ABCD": lines.append(f"{labels[level]} & {int(pc.get(level,0))} & {
62 int(ic.get(level,0)):,} & {desc[level]} \\\\")
63     lines += [r"\\bottomrule", r"\\end{tabularx}"]
64     write("capability_evidence.tex", lines)
65
66 def p2_policy() -> None:
67     frame = pd.read_csv(FROZEN / "p2_inpatient_only/policy_metrics.csv", encoding="utf
68 -8-sig")
69     names = {"FCFS_SHARED": "共享FCFS", "SLACK_GUARD_SHARED": "最小松弛度保护"}
70     lines = [r"\\begin{tabular}{lrrrrrr}", r"\\toprule", "策略 & 48h完成数 & 完成率 & P50/h
71 & P90/h & 多项目跨室率 \\\\", r"\\midrule"]
72     for row in frame.itertuples(index=False): lines.append(f"{names[row.policy]} & {
73 row.inpatient_complete_48h:,} & {pct(row.inpatient_48h_rate)} & {row.
74 inpatient_wait_p50_hours_conditional:.2f} & {row.
75 inpatient_wait_p90_hours_conditional:.2f} & {pct(row.
76 inpatient_room_switch_rate_multi)} \\\\")
77     lines += [r"\\bottomrule", r"\\end{tabular}"]
78     write("p2_policy.tex", lines)
79
80 def p3_pareto() -> None:
81     frame = pd.read_csv(FROZEN / "p3_joint/pareto_metrics.csv", encoding="utf-8-sig")
82     selected = float(json.loads((FROZEN / "p3_joint/selection.json").read_text(
83 encoding="utf-8"))["selected_alpha"])
84     lines = [r"\\begin{tabular}{lrrrrrr}", r"\\toprule", "$\\alpha$ & 背景完成数 & 背景率 &
85 住院完成数 & 住院率 & P90/h \\\\", r"\\midrule"]
86     for row in frame.itertuples(index=False):
87         pre, suf = (r"\\textbf{"", ""}) if abs(row.background_target_alpha-selected)<1e
88 -9 else ("", "")
89         lines.append(f"{pre}{row.background_target_alpha:.2f}{suf} & {pre}{row.
90 background_on_planned_day:,}{suf} & {pre}{pct(row.background_on_time_rate)}{suf} &
91 {pre}{row.inpatient_complete_48h:,}{suf} & {pre}{pct(row.inpatient_48h_rate)}{suf}
92 & {pre}{row.inpatient_wait_p90_hours_conditional:.2f}{suf} \\\\")
93     lines += [r"\\bottomrule", r"\\end{tabular}"]
94     write("p3_pareto.tex", lines)
95
96 def calibration() -> None:
97     frame = pd.read_csv(CAL / "calibration_metrics.csv", encoding="utf-8-sig")
98     names = {"C0_historical_report": "历史开单至报告", "C1_duration_only": "仅题给时长", "
99 C2_standard_hours": "加入标准班次", "C3_doctor_capacity": "加入医生并发", "
100 C4_project_capability": "加入项目设备能力", "C5_item_preparation": "加入项目级准备", "

```

```

C6_joint_background":"加入门诊/体检竞争", "C7_formal_heuristic":"正式构造策略"}
80 lines = [r"\begin{tabular}{lrrrr}", r"\toprule", "校准层 & 完成数/分母 & 完成率 & 较上
一层/百分点 \\\\", r"\midrule"]
81 for row in frame.itertuples(index=False):
82     delta = "--" if pd.isna(row.change_from_previous_pp) else f"{row.
change_from_previous_pp:.2f}"
83     lines.append(f"{names[row.scenario]} & {int(row.completed):,}/{int(row.total)
:,} & {pct(row.rate)} & {delta} \\\\")
84 lines += [r"\bottomrule", r"\end{tabular}"]
85 write("calibration.tex", lines)
86
87 def sensitivity() -> None:
88     frame = pd.read_csv(FROZEN / "sensitivity_metrics.csv", encoding="utf-8-sig").
set_index("audit_scenario")
89     order = ["preparation_item", "preparation_event", "doctor_low", "capability_AB", "
bladder45", "bladder90", "transfer5", "transfer15"]
90     names = {"preparation_item": "主设定: 项目级准备", "preparation_event": "事件级准备", "
doctor_low": "医生容量下四分位", "capability_AB": "仅直接/语义匹配", "bladder45": "憋尿45
分钟", "bladder90": "憋尿90分钟", "transfer5": "跨室5分钟", "transfer15": "跨室15分钟"}
91     lines = [r"\begin{tabular}{lrrrrr}", r"\toprule", "情景 & 住院率 & 背景率 & P90/h &
排入分钟 \\\\", r"\midrule"]
92     for name in order:
93         row = frame.loc[name]
94         lines.append(f"{names[name]} & {pct(row.inpatient_48h_rate)} & {pct(row.
background_on_time_rate)} & {row.inpatient_wait_p90_hours_conditional:.2f} & {int(
row.scheduled_minutes):,} \\\\")
95     lines += [r"\bottomrule", r"\end{tabular}"]
96     write("sensitivity_metrics.tex", lines)
97
98 def exact_benchmark() -> None:
99     frame = pd.read_csv(EXACT / "exact_vs_heuristic_metrics.csv", encoding="utf-8-sig"
)
100     names = {"normal_weekday": "普通工作日", "peak_weekday": "高峰工作日", "weekend": "周末"
, "multi_project_peak": "多项目高峰", "bedside_peak": "床旁高峰", "obstetric_peak": "产
科高峰"}
101     lines = [r"\begin{tabular}{lrrrrrl}", r"\toprule", "窗口 & 事件数 & 构造解 & CP-SAT &
最好界 & 状态 \\\\", r"\midrule"]
102     for row in frame.itertuples(index=False): lines.append(f"{names[row.case]} & {row.
event_count} & {row.heuristic_complete} & {row.exact_complete} & {row.best_bound:g
} & {row.solver_status} \\\\")
103     lines += [r"\bottomrule", r"\end{tabular}"]
104     write("exact_benchmark.tex", lines)
105
106 def verification() -> None:
107     reports = [json.loads((FROZEN / path).read_text(encoding="utf-8")) for path in ["
p2_inpatient_only/independent_verification.json", "p3_joint/
independent_verification.json"]]
108     groups = {"键、输入与时长":["unique_exported_task_key", "
scheduled_task_exists_in_input", "duration_matches_5pct_scenario"], "设备能力与床旁"
:["project_room_compatibility", "bedside_uses_room_7"], "时间窗与准备":["
task_not_before_release", "task_complete_by_event_deadline", "
inside_single_work_block", "fasting_complete_by_10", "bladder_task_after_readiness"
], "互斥与转运":["no_room_overlap", "no_same_patient_task_overlap", "
cross_room_transfer_at_least_10min"], "医生容量":["doctor_capacity_defined", "
doctor_concurrency_within_capacity"], "结果聚合":["
outcome_complete_flag_consistency", "outcome_deadline_flag_consistency"]}
109     lines = [r"\begin{tabular}{lrrrl}", r"\toprule", "核验组 & P2违规 & P3违规 & 结论
\\\\", r"\midrule"]
110     for label, keys in groups.items():
111         values = [sum(int(report["checks"][key]["violation_count"]) for key in keys)

```

```

112     for report in reports]
113         lines.append(f"{label} & {values[0]} & {values[1]} & {'通过' if sum(values)==0
114             else '未通过'} \\\\")
115     lines += [r"\bottomrule", r"\end{tabular}"]
116     write("verification.tex", lines)
117
118 def recommendation_sample() -> None:
119     frame = pd.read_csv(FROZEN / "p3_joint/inpatient_recommended_schedule.csv",
120         encoding="utf-8-sig", low_memory=False)
121     sample = frame[frame["event_id"].isin(frame["event_id"].drop_duplicates().head(6))
122         ].head(14)
123     lines = [r"\begin{tabularx}{\textwidth}{r l r X l l}", r"\toprule", "序位 & 事件 &
124         项目序位 & 项目族 & 时段 & 设备 \\\\", r"\midrule"]
125     for row in sample.itertuples(index=False):
126         start, end = pd.Timestamp(row.start_dt).strftime("%m-%d %H:%M"), pd.Timestamp(
127             row.end_dt).strftime("%H:%M")
128         lines.append(f"{row.recommended_patient_rank} & {esc(row.event_id)} & {row.
129             sequence_position} & {esc(row.category)} & {start}--{end} & {esc(row.room_name)}({
130                 row.room_id}) \\\\")
131     lines += [r"\bottomrule", r"\end{tabularx}"]
132     write("recommendation_sample.tex", lines)
133
134 def main() -> None:
135     OUT.mkdir(parents=True, exist_ok=True)
136     data_scope(); duration_parameters(); capability_evidence(); p2_policy(); p3_pareto
137     (); calibration(); sensitivity(); exact_benchmark(); verification();
138     recommendation_sample()
139     print("created 10 frozen-result LaTeX tables")
140
141 if __name__ == "__main__": main()

```

H 关键机器可读结果

Listing 16 问题二纯住院结果摘要

```

1  {
2  "scenario": "P2_INPATIENT_ONLY",
3  "selected_policy": "FCFS_SHARED",
4  "preparation_granularity": "item",
5  "metrics": {
6      "scenario": "P2_INPATIENT_ONLY",
7      "policy": "FCFS_SHARED",
8      "inpatient_events": 70771,
9      "inpatient_complete_48h": 60359,
10     "inpatient_48h_rate": 0.8528775911037006,
11     "inpatient_scheduled_complete": 60359,
12     "inpatient_unscheduled": 10412,
13     "inpatient_wait_p50_hours_conditional": 3.5833333333333335,
14     "inpatient_wait_p90_hours_conditional": 22.083333333333332,
15     "inpatient_wait_p95_hours_conditional": 23.666666666666668,
16     "inpatient_room_switch_rate_multi": 0.5811783598205881,
17     "background_events": 0,
18     "background_on_planned_day": 0,
19     "background_on_time_rate": NaN,
20     "background_scheduled_complete": 0,
21     "fallback_scheduled_task_share": 0.0,
22     "scheduled_minutes": 1344260.0,
23     "staffed_capacity_utilization": 0.5451010208529748,

```

```

24     "transfer_minutes_assumption": 10
25   }
26 }

```

Listing 17 问题三 Pareto 推荐点

```

1 {
2   "selection_rule": "minimum normalized Euclidean distance to the attainable Pareto
3     ideal",
4   "selected_alpha": 0.3,
5   "selected_metrics": {
6     "scenario": "P3_ALPHA_0.30",
7     "policy": "FCFS_SHARED",
8     "inpatient_events": 70771,
9     "inpatient_complete_48h": 42730,
10    "inpatient_48h_rate": 0.6037783838012746,
11    "inpatient_scheduled_complete": 42730,
12    "inpatient_unscheduled": 28041,
13    "inpatient_wait_p50_hours_conditional": 3.0,
14    "inpatient_wait_p90_hours_conditional": 21.75,
15    "inpatient_wait_p95_hours_conditional": 38.583333333333336,
16    "inpatient_room_switch_rate_multi": 0.2503737748491057,
17    "background_events": 96046,
18    "background_on_planned_day": 57177,
19    "background_on_time_rate": 0.5953084980113695,
20    "background_scheduled_complete": 57177,
21    "fallback_scheduled_task_share": 0.0,
22    "scheduled_minutes": 2066410.0,
23    "staffed_capacity_utilization": 0.8379347748953296,
24    "background_target_alpha": 0.3,
25    "quota_shortfall_days": 1,
26    "quota_target_events": 28977,
27    "quota_achieved_before_inpatient": 28970,
28    "preparation_granularity": "item",
29    "special_seed": 20260824,
30    "capability_mode": "hierarchical",
31    "capacity_column": "capacity_main",
32    "bladder_minutes": 60,
33    "transfer_minutes": 10,
34    "pareto_nondominated": true,
35    "distance_to_ideal": 0.915027740483536
36  }
37 }

```

Listing 18 问题二独立验证报告

```

1 {
2   "passed": true,
3   "schedule_rows": 117802,
4   "scheduled_events": 60583,
5   "input_events": 71029,
6   "checks": {
7     "unique_exported_task_key": {
8       "passed": true,
9       "violation_count": 0,
10      "detail": "each scheduled task must occur exactly once"
11    },
12    "scheduled_task_exists_in_input": {
13      "passed": true,
14      "violation_count": 0,

```

```

15     "detail": ""
16 },
17 "duration_matches_5pct_scenario": {
18     "passed": true,
19     "violation_count": 0,
20     "detail": ""
21 },
22 "project_room_compatibility": {
23     "passed": true,
24     "violation_count": 0,
25     "detail": ""
26 },
27 "bedside_uses_room_7": {
28     "passed": true,
29     "violation_count": 0,
30     "detail": ""
31 },
32 "task_not_before_release": {
33     "passed": true,
34     "violation_count": 0,
35     "detail": ""
36 },
37 "task_complete_by_event_deadline": {
38     "passed": true,
39     "violation_count": 0,
40     "detail": ""
41 },
42 "bladder_task_after_readiness": {
43     "passed": true,
44     "violation_count": 0,
45     "detail": ""
46 },
47 "five_minute_grid": {
48     "passed": true,
49     "violation_count": 0,
50     "detail": ""
51 },
52 "inside_single_work_block": {
53     "passed": true,
54     "violation_count": 0,
55     "detail": ""
56 },
57 "fasting_complete_by_10": {
58     "passed": true,
59     "violation_count": 0,
60     "detail": ""
61 },
62 "no_room_overlap": {
63     "passed": true,
64     "violation_count": 0,
65     "detail": ""
66 },
67 "contiguous_event_sequence_numbers": {
68     "passed": true,
69     "violation_count": 0,
70     "detail": ""
71 },
72 "no_same_patient_task_overlap": {
73     "passed": true,
74     "violation_count": 0,

```

```

75     "detail": ""
76   },
77   "cross_room_transfer_at_least_10min": {
78     "passed": true,
79     "violation_count": 0,
80     "detail": ""
81   },
82   "doctor_capacity_defined": {
83     "passed": true,
84     "violation_count": 0,
85     "detail": ""
86   },
87   "doctor_concurrency_within_capacity": {
88     "passed": true,
89     "violation_count": 0,
90     "detail": {
91       "max_observed_load": 20,
92       "max_capacity": 20
93     }
94   },
95   "outcome_complete_flag_consistency": {
96     "passed": true,
97     "violation_count": 0,
98     "detail": ""
99   },
100  "outcome_deadline_flag_consistency": {
101    "passed": true,
102    "violation_count": 0,
103    "detail": ""
104  }
105 }
106 }

```

Listing 19 问题三独立验证报告

```

1  {
2    "passed": true,
3    "schedule_rows": 176739,
4    "scheduled_events": 100303,
5    "input_events": 167361,
6    "checks": {
7      "unique_exported_task_key": {
8        "passed": true,
9        "violation_count": 0,
10       "detail": "each scheduled task must occur exactly once"
11     },
12     "scheduled_task_exists_in_input": {
13       "passed": true,
14       "violation_count": 0,
15       "detail": ""
16     },
17     "duration_matches_5pct_scenario": {
18       "passed": true,
19       "violation_count": 0,
20       "detail": ""
21     },
22     "project_room_compatibility": {
23       "passed": true,
24       "violation_count": 0,
25       "detail": ""

```

```

26     },
27     "bedside_uses_room_7": {
28         "passed": true,
29         "violation_count": 0,
30         "detail": ""
31     },
32     "task_not_before_release": {
33         "passed": true,
34         "violation_count": 0,
35         "detail": ""
36     },
37     "task_complete_by_event_deadline": {
38         "passed": true,
39         "violation_count": 0,
40         "detail": ""
41     },
42     "bladder_task_after_readiness": {
43         "passed": true,
44         "violation_count": 0,
45         "detail": ""
46     },
47     "five_minute_grid": {
48         "passed": true,
49         "violation_count": 0,
50         "detail": ""
51     },
52     "inside_single_work_block": {
53         "passed": true,
54         "violation_count": 0,
55         "detail": ""
56     },
57     "fasting_complete_by_10": {
58         "passed": true,
59         "violation_count": 0,
60         "detail": ""
61     },
62     "no_room_overlap": {
63         "passed": true,
64         "violation_count": 0,
65         "detail": ""
66     },
67     "contiguous_event_sequence_numbers": {
68         "passed": true,
69         "violation_count": 0,
70         "detail": ""
71     },
72     "no_same_patient_task_overlap": {
73         "passed": true,
74         "violation_count": 0,
75         "detail": ""
76     },
77     "cross_room_transfer_at_least_10min": {
78         "passed": true,
79         "violation_count": 0,
80         "detail": ""
81     },
82     "doctor_capacity_defined": {
83         "passed": true,
84         "violation_count": 0,
85         "detail": ""

```

```

86     },
87     "doctor_concurrency_within_capacity": {
88         "passed": true,
89         "violation_count": 0,
90         "detail": {
91             "max_observed_load": 20,
92             "max_capacity": 20
93         }
94     },
95     "outcome_complete_flag_consistency": {
96         "passed": true,
97         "violation_count": 0,
98         "detail": ""
99     },
100    "outcome_deadline_flag_consistency": {
101        "passed": true,
102        "violation_count": 0,
103        "detail": ""
104    }
105 }
106 }

```

Listing 20 五个特殊病例种子汇总

```

1  {
2    "seed_count": 5,
3    "inpatient_48h_rate": {
4        "mean": 0.6036257789207443,
5        "std": 0.0001951281891843176,
6        "min": 0.6033262211922963,
7        "max": 0.6038066439643357
8    },
9    "background_rate": {
10        "mean": 0.5962393020011245,
11        "std": 0.00020370734192191428,
12        "min": 0.5960060804197989,
13        "max": 0.5965058409512108
14    },
15    "waiting_p90_hours": [
16        21.75,
17        21.666666666666668,
18        21.666666666666668,
19        21.75,
20        21.666666666666668
21    ],
22    "scheduled_minutes": [
23        2067430.0,
24        2067540.0,
25        2067530.0,
26        2066490.0,
27        2068475.0
28    ]
29 }

```

I 输出字段说明

文件/字段	含义
events.csv	事件层输入，含来源、释放、截止、计划日代理、项目数、准备标记、复杂病例和评价边界。
items.csv	项目层输入，含规范项目、项目族、时长上下界、匹配方式、项目与事件准备标记和候选设备。
final_patient_task_schedule.csv	已排任务的事件、项目序位、开始、结束、设备、转运、准备和匹配方式。
final_patient_outcomes.csv	全部结果事件的完整完成、截止达标、首检和末检时刻；未排事件亦保留。
pareto_metrics.csv	各背景下限点的住院率、背景率、等待、利用率和配额诊断。
inpatient_recommended_schedule.csv	按首检时刻排序的住院患者一项目可执行推荐表。
independent_verification.json	独立重建约束后的逐项违规数与通过标志。